Raytracer

1.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Directory Documentation 9
5.1 src/cpu Directory Reference . . . 9
5.2 src/gpu Directory Reference . . . 9
5.3 src/hybrid Directory Reference . . . 9
5.4 src/math Directory Reference . . . 9
5.5 src/scene Directory Reference . . . 10
5.6 src Directory Reference . . . 10
5.7 src/utils Directory Reference . . . 10

6 Class Documentation 11
6.1 AABB Class Reference . . . 11
6.1.1 Detailed Description . . . 11
6.1.2 Constructor & Destructor Documentation . . . 12
6.1.2.1 AABB() [1/3] . . . 12
6.1.2.2 AABB() [2/3] . . . 12
6.1.2.3 AABB() [3/3] . . . 12
6.1.3 Member Function Documentation . . . 12
6.1.3.1 axis_interval() . . . 12
6.1.3.2 hit() [1/2] . . . 12
6.1.3.3 hit() [2/2] . . . 12
6.1.3.4 longest_axis() . . . 13
6.1.3.5 max_point() . . . 13
6.1.3.6 min_point() . . . 13
6.1.3.7 surface_area() . . . 13
6.1.4 Member Data Documentation . . . 13
6.1.4.1 empty . . . 13
6.1.4.2 universe . . . 13
6.1.4.3 x . . . 13
6.1.4.4 y . . . 14
6.1.4.5 z . . . 14
6.2 BVHAccel::BuildNode Struct Reference . . . 14
6.2.1 Detailed Description . . . 14

6.2.2 Member Data Documentation . . . 14
6.2.2.1 bbox . . . 14
6.2.2.2 children . . . 14
6.2.2.3 first_prim_offset . . . 15
6.2.2.4 num_primitives . . . 15
6.2.2.5 split_axis . . . 15
6.3 BVHAccel::BuildPrimInfo Struct Reference . . . 15
6.3.1 Detailed Description . . . 15
6.3.2 Member Data Documentation . . . 15
6.3.2.1 bbox . . . 15
6.3.2.2 centroid . . . 16
6.3.2.3 prim_index . . . 16
6.4 BVHAccel Class Reference . . . 16
6.4.1 Detailed Description . . . 17
6.4.2 Constructor & Destructor Documentation . . . 17
6.4.2.1 BVHAccel() . . . 17
6.4.3 Member Function Documentation . . . 17
6.4.3.1 bounding_box() . . . 17
6.4.3.2 delete_tree() . . . 17
6.4.3.3 flatten_tree() . . . 17
6.4.3.4 hit() . . . 18
6.4.3.5 recursive_build() . . . 18
6.4.4 Member Data Documentation . . . 18
6.4.4.1 nodes_ . . . 18
6.4.4.2 ordered_prims_ . . . 18
6.5 BVHNode Class Reference . . . 18
6.5.1 Detailed Description . . . 19
6.5.2 Constructor & Destructor Documentation . . . 19
6.5.2.1 BVHNode() [1/2] . . . 19
6.5.2.2 BVHNode() [2/2] . . . 19
6.5.3 Member Function Documentation . . . 19
6.5.3.1 bounding_box() . . . 19
6.5.3.2 hit() . . . 20
6.5.4 Member Data Documentation . . . 20
6.5.4.1 bbox_ . . . 20
6.5.4.2 left_ . . . 20
6.5.4.3 right_ . . . 20
6.5.4.4 split_axis_ . . . 20
6.6 Camera Class Reference . . . 20
6.6.1 Detailed Description . . . 21
6.6.2 Member Function Documentation . . . 21
6.6.2.1 defocus_disk_sample() . . . 21

6.6.2.2 get_image_height() . . . 21
6.6.2.3 get_image_width() . . . 22
6.6.2.4 get_ray() . . . 22
6.6.2.5 initialize() . . . 22
6.6.2.6 sample_square() . . . 22
6.6.3 Member Data Documentation . . . 22
6.6.3.1 aspect_ratio . . . 22
6.6.3.2 center_ . . . 22
6.6.3.3 defocus_angle . . . 22
6.6.3.4 defocus_disk_u_ . . . 23
6.6.3.5 defocus_disk_v_ . . . 23
6.6.3.6 focus_dist . . . 23
6.6.3.7 image_height_ . . . 23
6.6.3.8 image_width . . . 23
6.6.3.9 lookat . . . 23
6.6.3.10 lookfrom . . . 23
6.6.3.11 max_depth . . . 23
6.6.3.12 pixel00_loc_ . . . 24
6.6.3.13 pixel_delta_u_ . . . 24
6.6.3.14 pixel_delta_v_ . . . 24
6.6.3.15 samples_per_pixel . . . 24
6.6.3.16 u_ . . . 24
6.6.3.17 v_ . . . 24
6.6.3.18 vfov . . . 24
6.6.3.19 vup . . . 24
6.6.3.20 w_ . . . 25
6.7 Checker Class Reference . . . 25
6.7.1 Detailed Description . . . 26
6.7.2 Constructor & Destructor Documentation . . . 26
6.7.2.1 Checker() . . . 26
6.7.3 Member Function Documentation . . . 26
6.7.3.1 color1() . . . 26
6.7.3.2 color2() . . . 26
6.7.3.3 scale() . . . 26
6.7.3.4 scatter() . . . 26
6.7.4 Member Data Documentation . . . 27
6.7.4.1 color1_ . . . 27
6.7.4.2 color2_ . . . 27
6.7.4.3 scale_ . . . 27
6.8 CpuRenderer Class Reference . . . 27
6.8.1 Detailed Description . . . 28
6.8.2 Constructor & Destructor Documentation . . . 28

6.8.2.1 CpuRenderer() . . . 28
6.8.3 Member Function Documentation . . . 28
6.8.3.1 enable_bvh() . . . 28
6.8.3.2 get_world() . . . 28
6.8.3.3 ray_color() . . . 28
6.8.3.4 render() . . . 28
6.8.3.5 render_threaded() . . . 29
6.8.3.6 render_tile() . . . 29
6.8.4 Member Data Documentation . . . 29
6.8.4.1 bvh_root_ . . . 29
6.8.4.2 image_ . . . 29
6.8.4.3 scene_ . . . 29
6.8.4.4 use_bvh_ . . . 29
6.9 Cylinder Class Reference . . . 30
6.9.1 Detailed Description . . . 30
6.9.2 Constructor & Destructor Documentation . . . 30
6.9.2.1 Cylinder() . . . 30
6.9.3 Member Function Documentation . . . 31
6.9.3.1 bounding_box() . . . 31
6.9.3.2 hit() . . . 31
6.9.4 Member Data Documentation . . . 31
6.9.4.1 base_ . . . 31
6.9.4.2 height_ . . . 31
6.9.4.3 material_ . . . 31
6.9.4.4 radius_ . . . 31
6.10 Dielectric Class Reference . . . 32
6.10.1 Detailed Description . . . 32
6.10.2 Constructor & Destructor Documentation . . . 33
6.10.2.1 Dielectric() . . . 33
6.10.3 Member Function Documentation . . . 33
6.10.3.1 reflectance() . . . 33
6.10.3.2 refraction_index() . . . 33
6.10.3.3 scatter() . . . 33
6.10.4 Member Data Documentation . . . 33
6.10.4.1 refraction_index_ . . . 33
6.11 Emissive Class Reference . . . 34
6.11.1 Detailed Description . . . 34
6.11.2 Constructor & Destructor Documentation . . . 35
6.11.2.1 Emissive() . . . 35
6.11.3 Member Function Documentation . . . 35
6.11.3.1 emit_color() . . . 35
6.11.3.2 emitted() . . . 35

6.11.3.3 intensity() . . . 35
6.11.3.4 scatter() . . . 35
6.11.4 Member Data Documentation . . . 35
6.11.4.1 emit_ . . . 35
6.11.4.2 intensity_ . . . 36
6.12 FlatBVHNode Struct Reference . . . 36
6.12.1 Detailed Description . . . 36
6.12.2 Member Data Documentation . . . 36
6.12.2.1 [union] . . . 36
6.12.2.2 bbox . . . 36
6.12.2.3 num_primitives . . . 36
6.12.2.4 pad . . . 37
6.12.2.5 primitives_offset . . . 37
6.12.2.6 second_child_offset . . . 37
6.12.2.7 split_axis . . . 37
6.13 GAABB Struct Reference . . . 37
6.13.1 Detailed Description . . . 37
6.13.2 Constructor & Destructor Documentation . . . 38
6.13.2.1 GAABB() [1/2] . . . 38
6.13.2.2 GAABB() [2/2] . . . 38
6.13.3 Member Function Documentation . . . 38
6.13.3.1 hit() . . . 38
6.13.4 Member Data Documentation . . . 38
6.13.4.1 mn . . . 38
6.13.4.2 mx . . . 38
6.14 GBuildInfo Struct Reference . . . 39
6.14.1 Detailed Description . . . 39
6.14.2 Member Data Documentation . . . 39
6.14.2.1 bbox . . . 39
6.14.2.2 centroid . . . 39
6.14.2.3 sphere_idx . . . 39
6.15 GBVHNode Struct Reference . . . 39
6.15.1 Detailed Description . . . 40
6.15.2 Member Data Documentation . . . 40
6.15.2.1 bbox . . . 40
6.15.2.2 data . . . 40
6.15.2.3 num_prims . . . 40
6.15.2.4 prim_type . . . 40
6.15.2.5 split_axis . . . 40
6.16 GCamera Struct Reference . . . 41
6.16.1 Detailed Description . . . 41
6.16.2 Member Data Documentation . . . 41

6.16.2.1 defocus_angle . . . 41
6.16.2.2 defocus_disk_u . . . 41
6.16.2.3 defocus_disk_v . . . 41
6.16.2.4 image_height . . . 41
6.16.2.5 image_width . . . 42
6.16.2.6 max_depth . . . 42
6.16.2.7 origin . . . 42
6.16.2.8 pixel00_loc . . . 42
6.16.2.9 pixel_delta_u . . . 42
6.16.2.10 pixel_delta_v . . . 42
6.16.2.11 samples_per_pixel . . . 42
6.17 GHitRecord Struct Reference . . . 43
6.17.1 Detailed Description . . . 43
6.17.2 Member Function Documentation . . . 43
6.17.2.1 set_face_normal() . . . 43
6.17.3 Member Data Documentation . . . 43
6.17.3.1 front_face . . . 43
6.17.3.2 material_id . . . 43
6.17.3.3 normal . . . 43
6.17.3.4 p . . . 44
6.17.3.5 t . . . 44
6.18 GMaterial Struct Reference . . . 44
6.18.1 Detailed Description . . . 44
6.18.2 Constructor & Destructor Documentation . . . 44
6.18.2.1 GMaterial() . . . 44
6.18.3 Member Data Documentation . . . 44
6.18.3.1 albedo . . . 44
6.18.3.2 param . . . 45
6.18.3.3 type . . . 45
6.19 GPlane Struct Reference . . . 45
6.19.1 Detailed Description . . . 45
6.19.2 Member Data Documentation . . . 45
6.19.2.1 material_id . . . 45
6.19.2.2 normal . . . 45
6.19.2.3 point . . . 46
6.20 GpuRenderer Class Reference . . . 46
6.20.1 Detailed Description . . . 47
6.20.2 Constructor & Destructor Documentation . . . 47
6.20.2.1 GpuRenderer() . . . 47
6.20.2.2 ~GpuRenderer() . . . 47
6.20.3 Member Function Documentation . . . 47
6.20.3.1 ensure_framebuffer() . . . 47

6.20.3.2 free_framebuffer() . . . 47
6.20.3.3 free_scene() . . . 47
6.20.3.4 last_render_ms() . . . 48
6.20.3.5 render() . . . 48
6.20.3.6 upload_scene() . . . 48
6.20.4 Friends And Related Symbol Documentation . . . 48
6.20.4.1 gpu_realtime_render_frame . . . 48
6.20.4.2 gpu_render_scene_denoised . . . 48
6.20.5 Member Data Documentation . . . 49
6.20.5.1 d_bvh_nodes_ . . . 49
6.20.5.2 d_framebuffer_ . . . 49
6.20.5.3 d_materials_ . . . 49
6.20.5.4 d_planes_ . . . 49
6.20.5.5 d_soa_cx_ . . . 49
6.20.5.6 d_soa_cy_ . . . 49
6.20.5.7 d_soa_cz_ . . . 49
6.20.5.8 d_soa_mat_ . . . 50
6.20.5.9 d_soa_r_ . . . 50
6.20.5.10 d_spheres_ . . . 50
6.20.5.11 fb_height_ . . . 50
6.20.5.12 fb_width_ . . . 50
6.20.5.13 last_ms_ . . . 50
6.20.5.14 num_bvh_nodes_ . . . 50
6.20.5.15 num_materials_ . . . 50
6.20.5.16 num_planes_ . . . 51
6.20.5.17 num_spheres_ . . . 51
6.21 GRay Struct Reference . . . 51
6.21.1 Detailed Description . . . 51
6.21.2 Constructor & Destructor Documentation . . . 51
6.21.2.1 GRay() [1/2] . . . 51
6.21.2.2 GRay() [2/2] . . . 52
6.21.3 Member Function Documentation . . . 52
6.21.3.1 at() . . . 52
6.21.4 Member Data Documentation . . . 52
6.21.4.1 dir . . . 52
6.21.4.2 inv_dir . . . 52
6.21.4.3 orig . . . 52
6.21.4.4 orig_mul_inv . . . 52
6.22 GSphere Struct Reference . . . 53
6.22.1 Detailed Description . . . 53
6.22.2 Member Data Documentation . . . 53
6.22.2.1 center . . . 53

6.22.2.2 material_id . . . 53
6.22.2.3 radius . . . 53
6.23 GSphereSoA Struct Reference . . . 53
6.23.1 Detailed Description . . . 54
6.23.2 Member Data Documentation . . . 54
6.23.2.1 count . . . 54
6.23.2.2 cx . . . 54
6.23.2.3 cy . . . 54
6.23.2.4 cz . . . 54
6.23.2.5 mat . . . 54
6.23.2.6 r . . . 54
6.24 HitRecord Struct Reference . . . 55
6.24.1 Detailed Description . . . 55
6.24.2 Member Function Documentation . . . 55
6.24.2.1 set_face_normal() . . . 55
6.24.3 Member Data Documentation . . . 55
6.24.3.1 front_face . . . 55
6.24.3.2 material . . . 55
6.24.3.3 normal . . . 55
6.24.3.4 p . . . 56
6.24.3.5 t . . . 56
6.25 Hittable Class Reference . . . 56
6.25.1 Detailed Description . . . 56
6.25.2 Constructor & Destructor Documentation . . . 56
6.25.2.1 ~Hittable() . . . 56
6.25.3 Member Function Documentation . . . 57
6.25.3.1 bounding_box() . . . 57
6.25.3.2 hit() . . . 57
6.26 HittableList Class Reference . . . 57
6.26.1 Detailed Description . . . 58
6.26.2 Constructor & Destructor Documentation . . . 58
6.26.2.1 HittableList() . . . 58
6.26.3 Member Function Documentation . . . 58
6.26.3.1 add() . . . 58
6.26.3.2 bounding_box() . . . 58
6.26.3.3 clear() . . . 58
6.26.3.4 hit() . . . 58
6.26.3.5 objects() . . . 59
6.26.4 Member Data Documentation . . . 59
6.26.4.1 bbox_ . . . 59
6.26.4.2 objects_ . . . 59
6.27 ImageBuffer Class Reference . . . 59

6.27.1 Detailed Description . . . 60
6.27.2 Constructor & Destructor Documentation . . . 60
6.27.2.1 ImageBuffer() . . . 60
6.27.3 Member Function Documentation . . . 60
6.27.3.1 get_pixel() . . . 60
6.27.3.2 height() . . . 60
6.27.3.3 index() . . . 60
6.27.3.4 set_pixel() . . . 61
6.27.3.5 to_byte() . . . 61
6.27.3.6 width() . . . 61
6.27.3.7 write_ppm() [1/2] . . . 61
6.27.3.8 write_ppm() [2/2] . . . 61
6.27.3.9 write_ppm_binary() . . . 61
6.27.4 Member Data Documentation . . . 61
6.27.4.1 height_ . . . 61
6.27.4.2 pixels_ . . . 62
6.27.4.3 width_ . . . 62
6.28 Interval Class Reference . . . 62
6.28.1 Detailed Description . . . 62
6.28.2 Constructor & Destructor Documentation . . . 62
6.28.2.1 Interval() [1/2] . . . 62
6.28.2.2 Interval() [2/2] . . . 63
6.28.3 Member Function Documentation . . . 63
6.28.3.1 clamp() . . . 63
6.28.3.2 contains() . . . 63
6.28.3.3 size() . . . 63
6.28.3.4 surrounds() . . . 63
6.28.4 Member Data Documentation . . . 63
6.28.4.1 empty . . . 63
6.28.4.2 max . . . 64
6.28.4.3 min . . . 64
6.28.4.4 universe . . . 64
6.29 Lambertian Class Reference . . . 64
6.29.1 Detailed Description . . . 65
6.29.2 Constructor & Destructor Documentation . . . 65
6.29.2.1 Lambertian() . . . 65
6.29.3 Member Function Documentation . . . 65
6.29.3.1 albedo() . . . 65
6.29.3.2 scatter() . . . 65
6.29.4 Member Data Documentation . . . 66
6.29.4.1 albedo_ . . . 66
6.30 Material Class Reference . . . 66

6.30.1 Detailed Description . . . 66
6.30.2 Member Enumeration Documentation . . . 67
6.30.2.1 Type . . . 67
6.30.3 Constructor & Destructor Documentation . . . 67
6.30.3.1 ~Material() . . . 67
6.30.4 Member Function Documentation . . . 67
6.30.4.1 scatter() . . . 67
6.30.4.2 type() . . . 67
6.30.5 Member Data Documentation . . . 67
6.30.5.1 type_ . . . 67
6.31 Metal Class Reference . . . 68
6.31.1 Detailed Description . . . 68
6.31.2 Constructor & Destructor Documentation . . . 69
6.31.2.1 Metal() . . . 69
6.31.3 Member Function Documentation . . . 69
6.31.3.1 albedo() . . . 69
6.31.3.2 fuzz() . . . 69
6.31.3.3 scatter() . . . 69
6.31.4 Member Data Documentation . . . 69
6.31.4.1 albedo_ . . . 69
6.31.4.2 fuzz_ . . . 70
6.32 Plane Class Reference . . . 70
6.32.1 Detailed Description . . . 70
6.32.2 Constructor & Destructor Documentation . . . 71
6.32.2.1 Plane() . . . 71
6.32.3 Member Function Documentation . . . 71
6.32.3.1 bounding_box() . . . 71
6.32.3.2 hit() . . . 71
6.32.4 Member Data Documentation . . . 71
6.32.4.1 material_ . . . 71
6.32.4.2 normal_ . . . 71
6.32.4.3 point_ . . . 72
6.33 Ray Class Reference . . . 72
6.33.1 Detailed Description . . . 72
6.33.2 Constructor & Destructor Documentation . . . 72
6.33.2.1 Ray() [1/2] . . . 72
6.33.2.2 Ray() [2/2] . . . 72
6.33.3 Member Function Documentation . . . 73
6.33.3.1 at() . . . 73
6.33.3.2 direction() . . . 73
6.33.3.3 origin() . . . 73
6.33.4 Member Data Documentation . . . 73

6.33.4.1 dir . . . 73
6.33.4.2 dir_sign . . . 73
6.33.4.3 inv_dir . . . 73
6.33.4.4 orig . . . 74
6.34 Scene Class Reference . . . 74
6.34.1 Detailed Description . . . 74
6.34.2 Member Function Documentation . . . 75
6.34.2.1 add_material() . . . 75
6.34.2.2 build_cornell_box() . . . 75
6.34.2.3 build_final_scene() . . . 75
6.34.2.4 build_galaxy() . . . 75
6.34.2.5 build_hall() . . . 75
6.34.2.6 build_minimal() . . . 75
6.34.2.7 build_showcase() . . . 75
6.34.2.8 build_three_spheres() . . . 76
6.34.2.9 build_triangle_scene() . . . 76
6.34.3 Member Data Documentation . . . 76
6.34.3.1 camera . . . 76
6.34.3.2 materials_ . . . 76
6.34.3.3 world . . . 76
6.35 Sphere Class Reference . . . 76
6.35.1 Detailed Description . . . 77
6.35.2 Constructor & Destructor Documentation . . . 77
6.35.2.1 Sphere() . . . 77
6.35.3 Member Function Documentation . . . 77
6.35.3.1 bounding_box() . . . 77
6.35.3.2 center() . . . 77
6.35.3.3 hit() . . . 78
6.35.3.4 radius() . . . 78
6.35.4 Member Data Documentation . . . 78
6.35.4.1 center_ . . . 78
6.35.4.2 material_ . . . 78
6.35.4.3 radius_ . . . 78
6.36 Timer Class Reference . . . 78
6.36.1 Detailed Description . . . 79
6.36.2 Constructor & Destructor Documentation . . . 79
6.36.2.1 Timer() . . . 79
6.36.3 Member Function Documentation . . . 79
6.36.3.1 elapsed() . . . 79
6.36.3.2 elapsed_ms() . . . 79
6.36.3.3 reset() . . . 79
6.36.4 Member Data Documentation . . . 80

6.36.4.1 start_ . . . 80
6.37 Triangle Class Reference . . . 80
6.37.1 Detailed Description . . . 80
6.37.2 Constructor & Destructor Documentation . . . 81
6.37.2.1 Triangle() . . . 81
6.37.3 Member Function Documentation . . . 81
6.37.3.1 bounding_box() . . . 81
6.37.3.2 hit() . . . 81
6.37.4 Member Data Documentation . . . 81
6.37.4.1 material_ . . . 81
6.37.4.2 normal_ . . . 81
6.37.4.3 v0_ . . . 82
6.37.4.4 v1_ . . . 82
6.37.4.5 v2_ . . . 82
6.38 Vec3 Class Reference . . . 82
6.38.1 Detailed Description . . . 83
6.38.2 Constructor & Destructor Documentation . . . 83
6.38.2.1 Vec3() [1/2] . . . 83
6.38.2.2 Vec3() [2/2] . . . 83
6.38.3 Member Function Documentation . . . 83
6.38.3.1 length() . . . 83
6.38.3.2 length_squared() . . . 83
6.38.3.3 near_zero() . . . 83
6.38.3.4 operator∗=() . . . 83
6.38.3.5 operator+=() . . . 84
6.38.3.6 operator-() . . . 84
6.38.3.7 operator-=() . . . 84
6.38.3.8 operator/=() . . . 84
6.38.3.9 operator[]() [1/2] . . . 84
6.38.3.10 operator[]() [2/2] . . . 84
6.38.3.11 x() . . . 84
6.38.3.12 y() . . . 85
6.38.3.13 z() . . . 85
6.38.4 Member Data Documentation . . . 85
6.38.4.1 e . . . 85

7 File Documentation **87**
7.1 src/cpu/cpu_renderer.cpp File Reference . . . 87
7.1.1 Function Documentation . . . 87
7.1.1.1 scatter_inline() . . . 87
7.2 cpu_renderer.cpp . . . 88
7.3 src/cpu/cpu_renderer.h File Reference . . . 90

7.4 cpu_renderer.h . . . 91
7.5 src/gpu/gpu_realtime.cpp File Reference . . . 91
7.5.1 Function Documentation . . . 91
7.5.1.1 gpu_realtime_render() . . . 91
7.6 gpu_realtime.cpp . . . 92
7.7 src/gpu/gpu_realtime.h File Reference . . . 94
7.7.1 Function Documentation . . . 94
7.7.1.1 gpu_realtime_render() . . . 94
7.8 gpu_realtime.h . . . 94
7.9 src/gpu/gpu_renderer.cu File Reference . . . 95
7.9.1 Macro Definition Documentation . . . 96
7.9.1.1 BVH_SHARED_NODES . . . 96
7.9.1.2 CUDA_CHECK . . . 97
7.9.1.3 d_camera . . . 97
7.9.2 Typedef Documentation . . . 97
7.9.2.1 GRng . . . 97
7.9.3 Function Documentation . . . 97
7.9.3.1 accumulate_kernel() . . . 97
7.9.3.2 aces_tonemap_gpu() . . . 97
7.9.3.3 build_gpu_bvh() . . . 98
7.9.3.4 convert_camera_only() . . . 98
7.9.3.5 convert_scene() . . . 98
7.9.3.6 denoise_bilateral_kernel() . . . 98
7.9.3.7 gpu_realtime_init() . . . 98
7.9.3.8 gpu_realtime_render_frame() . . . 99
7.9.3.9 gpu_realtime_reset_accum() . . . 99
7.9.3.10 gpu_realtime_tonemap() . . . 99
7.9.3.11 gpu_render_scene() . . . 99
7.9.3.12 gpu_render_scene_denoised() . . . 99
7.9.3.13 grng_float() . . . 99
7.9.3.14 grng_init() . . . 100
7.9.3.15 intersect_plane() . . . 100
7.9.3.16 intersect_sphere() . . . 100
7.9.3.17 intersect_sphere_soa() . . . 100
7.9.3.18 random_in_unit_disk() . . . 100
7.9.3.19 random_in_unit_sphere() . . . 101
7.9.3.20 random_unit_vector() . . . 101
7.9.3.21 ray_color() . . . 101
7.9.3.22 render_kernel() . . . 101
7.9.3.23 scatter_mat() . . . 102
7.9.3.24 tonemap_kernel() . . . 102
7.9.3.25 trace_brute() . . . 102

7.9.3.26 trace_bvh() . . . 102
7.9.4 Variable Documentation . . . 103
7.9.4.1 d_camera_bytes . . . 103
7.9.4.2 s_renderer . . . 103
7.9.4.3 s_rt_acc_grid . . . 103
7.9.4.4 s_rt_accum_count . . . 103
7.9.4.5 s_rt_accum_fb . . . 103
7.9.4.6 s_rt_argb . . . 103
7.9.4.7 s_rt_argb_host . . . 103
7.9.4.8 s_rt_block . . . 104
7.9.4.9 s_rt_config_done . . . 104
7.9.4.10 s_rt_frame_fb . . . 104
7.9.4.11 s_rt_grid . . . 104
7.9.4.12 s_rt_h . . . 104
7.9.4.13 s_rt_start . . . 104
7.9.4.14 s_rt_stop . . . 104
7.9.4.15 s_rt_stream . . . 104
7.9.4.16 s_rt_w . . . 105
7.10 gpu_renderer.cu . . . 105
7.11 src/gpu/gpu_renderer.h File Reference . . . 117
7.11.1 Function Documentation . . . 117
7.11.1.1 gpu_realtime_init() . . . 117
7.11.1.2 gpu_realtime_render_frame() . . . 117
7.11.1.3 gpu_realtime_reset_accum() . . . 118
7.11.1.4 gpu_realtime_tonemap() . . . 118
7.11.1.5 gpu_render_scene() . . . 118
7.11.1.6 gpu_render_scene_denoised() . . . 118
7.12 gpu_renderer.h . . . 118
7.13 src/gpu/gpu_types.cuh File Reference . . . 119
7.13.1 Macro Definition Documentation . . . 120
7.13.1.1 RT_GPU_TYPES_CUH . . . 120
7.13.2 Enumeration Type Documentation . . . 120
7.13.2.1 GMaterialType . . . 120
7.13.3 Function Documentation . . . 121
7.13.3.1 __align__() . . . 121
7.13.3.2 cross() . . . 121
7.13.3.3 dot() . . . 121
7.13.3.4 normalize() . . . 121
7.13.3.5 operator∗() . . . 121
7.13.3.6 reflect() . . . 122
7.13.3.7 refract() . . . 122
7.14 gpu_types.cuh . . . 122

7.15 src/hybrid/hybrid_renderer.cu File Reference . . . 124
7.15.1 Function Documentation . . . 125
7.15.1.1 hybrid_render_scene() . . . 125
7.16 hybrid_renderer.cu . . . 125
7.17 src/hybrid/hybrid_renderer.h File Reference . . . 126
7.17.1 Function Documentation . . . 127
7.17.1.1 hybrid_render_scene() . . . 127
7.18 hybrid_renderer.h . . . 127
7.19 src/main.cpp File Reference . . . 127
7.19.1 Function Documentation . . . 127
7.19.1.1 main() . . . 127
7.20 main.cpp . . . 128
7.21 src/math/aabb.cuh File Reference . . . 130
7.21.1 Macro Definition Documentation . . . 130
7.21.1.1 RT_MATH_AABB_CUH . . . 130
7.21.2 Function Documentation . . . 130
7.21.2.1 surrounding_box() . . . 130
7.22 aabb.cuh . . . 131
7.23 src/math/interval.cuh File Reference . . . 132
7.23.1 Macro Definition Documentation . . . 132
7.23.1.1 RT_MATH_INTERVAL_CUH . . . 132
7.24 interval.cuh . . . 133
7.25 src/math/random.cuh File Reference . . . 133
7.25.1 Macro Definition Documentation . . . 134
7.25.1.1 RT_MATH_RANDOM_CUH . . . 134
7.25.2 Function Documentation . . . 134
7.25.2.1 random_double() [1/2] . . . 134
7.25.2.2 random_double() [2/2] . . . 134
7.25.2.3 random_in_unit_disk() . . . 134
7.25.2.4 random_in_unit_sphere() . . . 134
7.25.2.5 random_on_hemisphere() . . . 135
7.25.2.6 random_unit_vector() . . . 135
7.25.2.7 random_vec3() [1/2] . . . 135
7.25.2.8 random_vec3() [2/2] . . . 135
7.26 random.cuh . . . 135
7.27 src/math/ray.cuh File Reference . . . 136
7.27.1 Macro Definition Documentation . . . 136
7.27.1.1 RT_MATH_RAY_CUH . . . 136
7.28 ray.cuh . . . 137
7.29 src/math/vec3.cuh File Reference . . . 137
7.29.1 Macro Definition Documentation . . . 138
7.29.1.1 RT_HOSTDEV . . . 138

7.29.1.2 RT_MATH_VEC3_CUH . . . 138
7.29.2 Typedef Documentation . . . 138
7.29.2.1 Color . . . 138
7.29.2.2 Point3 . . . 138
7.29.3 Function Documentation . . . 139
7.29.3.1 cross() . . . 139
7.29.3.2 dot() . . . 139
7.29.3.3 operator∗() [1/3] . . . 139
7.29.3.4 operator∗() [2/3] . . . 139
7.29.3.5 operator∗() [3/3] . . . 139
7.29.3.6 operator+() . . . 139
7.29.3.7 operator-() . . . 140
7.29.3.8 operator/() . . . 140
7.29.3.9 operator<<() . . . 140
7.29.3.10 reflect() . . . 140
7.29.3.11 refract() . . . 140
7.29.3.12 unit_vector() . . . 140
7.30 vec3.cuh . . . 141
7.31 src/scene/bvh_builder.cpp File Reference . . . 142
7.32 bvh_builder.cpp . . . 142
7.33 src/scene/bvh_builder.h File Reference . . . 146
7.34 bvh_builder.h . . . 146
7.35 src/scene/bvh_node.cuh File Reference . . . 147
7.35.1 Macro Definition Documentation . . . 147
7.35.1.1 RT_SCENE_BVH_NODE_CUH . . . 147
7.36 bvh_node.cuh . . . 148
7.37 src/scene/camera.cpp File Reference . . . 148
7.38 camera.cpp . . . 148
7.39 src/scene/camera.cuh File Reference . . . 149
7.39.1 Macro Definition Documentation . . . 149
7.39.1.1 RT_SCENE_CAMERA_CUH . . . 149
7.40 camera.cuh . . . 150
7.41 src/scene/material.cpp File Reference . . . 150
7.42 material.cpp . . . 150
7.43 src/scene/material.cuh File Reference . . . 152
7.43.1 Macro Definition Documentation . . . 152
7.43.1.1 RT_SCENE_MATERIAL_CUH . . . 152
7.44 material.cuh . . . 153
7.45 src/scene/primitives.cpp File Reference . . . 154
7.46 primitives.cpp . . . 154
7.47 src/scene/primitives.cuh File Reference . . . 157
7.47.1 Macro Definition Documentation . . . 157

7.47.1.1 RT_SCENE_PRIMITIVES_CUH . . . 157
7.48 primitives.cuh . . . 158
7.49 src/scene/scene.cpp File Reference . . . 159
7.50 scene.cpp . . . 159
7.51 src/scene/scene.h File Reference . . . 164
7.52 scene.h . . . 164
7.53 src/utils/cuda_utils.h File Reference . . . 165
7.54 cuda_utils.h . . . 165
7.55 src/utils/denoise.cpp File Reference . . . 165
7.55.1 Function Documentation . . . 165
7.55.1.1 denoise_bilateral() . . . 165
7.56 denoise.cpp . . . 166
7.57 src/utils/denoise.h File Reference . . . 166
7.57.1 Function Documentation . . . 167
7.57.1.1 denoise_bilateral() . . . 167
7.58 denoise.h . . . 167
7.59 src/utils/image.cpp File Reference . . . 167
7.59.1 Function Documentation . . . 167
7.59.1.1 aces_tonemap() . . . 167
7.60 image.cpp . . . 168
7.61 src/utils/image.h File Reference . . . 168
7.62 image.h . . . 169
7.63 src/utils/timer.h File Reference . . . 169
7.64 timer.h . . . 170

Chapter 1

Directory Hierarchy

1.1 Directories

cpu . . . 9
- cpu_renderer.cpp . . . 87
- cpu_renderer.h . . . 90

gpu . . . 9
- gpu_realtime.cpp . . . 91
- gpu_realtime.h . . . 94
- gpu_renderer.cu . . . 95
- gpu_renderer.h . . . 117
- gpu_types.cuh . . . 119

hybrid . . . 9
- hybrid_renderer.cu . . . 124
- hybrid_renderer.h . . . 126

math . . . 9
- aabb.cuh . . . 130
- interval.cuh . . . 132
- random.cuh . . . 133
- ray.cuh . . . 136
- vec3.cuh . . . 137

scene . . . 10
- bvh_builder.cpp . . . 142
- bvh_builder.h . . . 146
- bvh_node.cuh . . . 147
- camera.cpp . . . 148
- camera.cuh . . . 149
- material.cpp . . . 150
- material.cuh . . . 152
- primitives.cpp . . . 154
- primitives.cuh . . . 157
- scene.cpp . . . 159
- scene.h . . . 164

src . . . 10
- cpu . . . 9
  - cpu_renderer.cpp . . . 87
  - cpu_renderer.h . . . 90
- gpu . . . 9
  - gpu_realtime.cpp . . . 91

- gpu_realtime.h . . . 94
- gpu_renderer.cu . . . 95
- gpu_renderer.h . . . 117
- gpu_types.cuh . . . 119

hybrid . . . 9
- hybrid_renderer.cu . . . 124
- hybrid_renderer.h . . . 126

math . . . 9
- aabb.cuh . . . 130
- interval.cuh . . . 132
- random.cuh . . . 133
- ray.cuh . . . 136
- vec3.cuh . . . 137

scene . . . 10
- bvh_builder.cpp . . . 142
- bvh_builder.h . . . 146
- bvh_node.cuh . . . 147
- camera.cpp . . . 148
- camera.cuh . . . 149
- material.cpp . . . 150
- material.cuh . . . 152
- primitives.cpp . . . 154
- primitives.cuh . . . 157
- scene.cpp . . . 159
- scene.h . . . 164

utils . . . 10
- cuda_utils.h . . . 165
- denoise.cpp . . . 165
- denoise.h . . . 166
- image.cpp . . . 167
- image.h . . . 168
- timer.h . . . 169

main.cpp . . . 127

utils . . . 10
- cuda_utils.h . . . 165
- denoise.cpp . . . 165
- denoise.h . . . 166
- image.cpp . . . 167
- image.h . . . 168
- timer.h . . . 169

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AABB . . . 11
BVHAccel::BuildNode . . . 14
BVHAccel::BuildPrimInfo . . . 15
Camera . . . 20
CpuRenderer . . . 27
FlatBVHNode . . . 36
GAABB . . . 37
GBuildInfo . . . 39
GBVHNode . . . 39
GCamera . . . 41
GHitRecord . . . 43
GMaterial . . . 44
GPlane . . . 45
GpuRenderer . . . 46
GRay . . . 51
GSphere . . . 53
GSphereSoA . . . 53
HitRecord . . . 55
Hittable . . . 56
- BVHAccel . . . 16
- BVHNode . . . 18
- Cylinder . . . 30
- HittableList . . . 57
- Plane . . . 70
- Sphere . . . 76
- Triangle . . . 80

ImageBuffer . . . 59
Interval . . . 62
Material . . . 66
- Checker . . . 25
- Dielectric . . . 32
- Emissive . . . 34
- Lambertian . . . 64
- Metal . . . 68

Ray . . . 72
Scene . . . 74
Timer . . . 78
Vec3 . . . 82

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AABB 11
BVHAccel::BuildNode 14
BVHAccel::BuildPrimInfo 15
BVHAccel 16
BVHNode 18
Camera 20
Checker 25
CpuRenderer 27
Cylinder 30
Dielectric 32
Emissive 34
FlatBVHNode 36
GAABB 37
GBuildInfo 39
GBVHNode 39
GCamera 41
GHitRecord 43
GMaterial 44
GPlane 45
GpuRenderer 46
GRay 51
GSphere 53
GSphereSoA 53
HitRecord 55
Hittable 56
HittableList 57
ImageBuffer 59
Interval 62
Lambertian 64
Material 66
Metal 68
Plane 70
Ray 72
Scene 74
Sphere 76
Timer 78
Triangle 80
Vec3 82

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/main.cpp . . . 127
src/cpu/cpu_renderer.cpp . . . 87
src/cpu/cpu_renderer.h . . . 90
src/gpu/gpu_realtime.cpp . . . 91
src/gpu/gpu_realtime.h . . . 94
src/gpu/gpu_renderer.cu . . . 95
src/gpu/gpu_renderer.h . . . 117
src/gpu/gpu_types.cuh . . . 119
src/hybrid/hybrid_renderer.cu . . . 124
src/hybrid/hybrid_renderer.h . . . 126
src/math/aabb.cuh . . . 130
src/math/interval.cuh . . . 132
src/math/random.cuh . . . 133
src/math/ray.cuh . . . 136
src/math/vec3.cuh . . . 137
src/scene/bvh_builder.cpp . . . 142
src/scene/bvh_builder.h . . . 146
src/scene/bvh_node.cuh . . . 147
src/scene/camera.cpp . . . 148
src/scene/camera.cuh . . . 149
src/scene/material.cpp . . . 150
src/scene/material.cuh . . . 152
src/scene/primitives.cpp . . . 154
src/scene/primitives.cuh . . . 157
src/scene/scene.cpp . . . 159
src/scene/scene.h . . . 164
src/utils/cuda_utils.h . . . 165
src/utils/denoise.cpp . . . 165
src/utils/denoise.h . . . 166
src/utils/image.cpp . . . 167
src/utils/image.h . . . 168
src/utils/timer.h . . . 169

Chapter 5

Directory Documentation

5.1 src/cpu Directory Reference

Files

- file cpu_renderer.cpp
- file cpu_renderer.h

5.2 src/gpu Directory Reference

Files

- file gpu_realtime.cpp
- file gpu_realtime.h
- file gpu_renderer.cu
- file gpu_renderer.h
- file gpu_types.cuh

5.3 src/hybrid Directory Reference

Files

- file hybrid_renderer.cu
- file hybrid_renderer.h

5.4 src/math Directory Reference

Files

- file aabb.cuh
- file interval.cuh
- file random.cuh
- file ray.cuh
- file vec3.cuh

5.5 src/scene Directory Reference

Files

- file bvh_builder.cpp
- file bvh_builder.h
- file bvh_node.cuh
- file camera.cpp
- file camera.cuh
- file material.cpp
- file material.cuh
- file primitives.cpp
- file primitives.cuh
- file scene.cpp
- file scene.h

5.6 src Directory Reference

Directories

- directory cpu
- directory gpu
- directory hybrid
- directory math
- directory scene
- directory utils

Files

- file main.cpp

5.7 src/utils Directory Reference

Files

- file cuda_utils.h
- file denoise.cpp
- file denoise.h
- file image.cpp
- file image.h
- file timer.h

Chapter 6

Class Documentation

6.1 AABB Class Reference

Public Member Functions

- RT_HOSTDEV AABB ()
- RT_HOSTDEV AABB (const Interval &x, const Interval &y, const Interval &z)
- RT_HOSTDEV AABB (const Point3 &a, const Point3 &b)
- RT_HOSTDEV const Interval & axis_interval (int n) const
- RT_HOSTDEV Point3 min_point () const
- RT_HOSTDEV Point3 max_point () const
- RT_HOSTDEV bool hit (const Ray &r, double t_min, double t_max) const
- RT_HOSTDEV bool hit (const Ray &r, Interval ray_t) const
- RT_HOSTDEV double surface_area () const
- RT_HOSTDEV int longest_axis () const

Public Attributes

- Interval x
- Interval y
- Interval z

Static Public Attributes

- static const AABB empty = AABB(Interval::empty, Interval::empty, Interval::empty)
- static const AABB universe = AABB(Interval::universe, Interval::universe, Interval::universe)

6.1.1 Detailed Description

Definition at line 16 of file aabb.cuh.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 AABB() [1/3]

```
RT_HOSTDEV AABB::AABB () [inline]
```

Definition at line 20 of file aabb.cuh.

6.1.2.2 AABB() [2/3]

```
RT_HOSTDEV AABB::AABB (
            const Interval & x,
            const Interval & y,
            const Interval & z) [inline]
```

Definition at line 22 of file aabb.cuh.

6.1.2.3 AABB() [3/3]

```
RT_HOSTDEV AABB::AABB (
            const Point3 & a,
            const Point3 & b) [inline]
```

Definition at line 26 of file aabb.cuh.

6.1.3 Member Function Documentation

6.1.3.1 axis_interval()

```
RT_HOSTDEV const Interval & AABB::axis_interval (
            int n) const [inline]
```

Definition at line 34 of file aabb.cuh.

6.1.3.2 hit() [1/2]

```
RT_HOSTDEV bool AABB::hit (
            const Ray & r,
            double t_min,
            double t_max) const [inline]
```

Definition at line 47 of file aabb.cuh.

6.1.3.3 hit() [2/2]

```
RT_HOSTDEV bool AABB::hit (
            const Ray & r,
            Interval ray_t) const [inline]
```

Definition at line 79 of file aabb.cuh.

6.1.3.4 longest_axis()

```
RT_HOSTDEV int AABB::longest_axis () const  [inline]
```

Definition at line 92 of file aabb.cuh.

6.1.3.5 max_point()

```
RT_HOSTDEV Point3 AABB::max_point () const  [inline]
```

Definition at line 41 of file aabb.cuh.

6.1.3.6 min_point()

```
RT_HOSTDEV Point3 AABB::min_point () const  [inline]
```

Definition at line 40 of file aabb.cuh.

6.1.3.7 surface_area()

```
RT_HOSTDEV double AABB::surface_area () const  [inline]
```

Definition at line 86 of file aabb.cuh.

6.1.4 Member Data Documentation

6.1.4.1 empty

```
const AABB AABB::empty = AABB(Interval::empty, Interval::empty, Interval::empty)  [inline],
[static]
```

Definition at line 98 of file aabb.cuh.

6.1.4.2 universe

```
const AABB AABB::universe = AABB(Interval::universe, Interval::universe, Interval::universe)
[inline], [static]
```

Definition at line 99 of file aabb.cuh.

6.1.4.3 x

```
Interval AABB::x
```

Definition at line 18 of file aabb.cuh.

6.1.4.4 y

```
Interval AABB::y
```

Definition at line 18 of file aabb.cuh.

6.1.4.5 z

```
Interval AABB::z
```

Definition at line 18 of file aabb.cuh.

The documentation for this class was generated from the following file:

- src/math/aabb.cuh

6.2 BVHAccel::BuildNode Struct Reference

Public Attributes

- AABB bbox
- BuildNode ∗ children [2] = {nullptr, nullptr}
- int split_axis = 0
- int first_prim_offset = 0
- int num_primitives = 0

6.2.1 Detailed Description

Definition at line 28 of file bvh_builder.h.

6.2.2 Member Data Documentation

6.2.2.1 bbox

```
AABB BVHAccel::BuildNode::bbox
```

Definition at line 29 of file bvh_builder.h.

6.2.2.2 children

```
BuildNode* BVHAccel::BuildNode::children[2] = {nullptr, nullptr}
```

Definition at line 30 of file bvh_builder.h.

6.2.2.3 first_prim_offset

```
int BVHAccel::BuildNode::first_prim_offset = 0
```

Definition at line 32 of file bvh_builder.h.

6.2.2.4 num_primitives

```
int BVHAccel::BuildNode::num_primitives = 0
```

Definition at line 33 of file bvh_builder.h.

6.2.2.5 split_axis

```
int BVHAccel::BuildNode::split_axis = 0
```

Definition at line 31 of file bvh_builder.h.

The documentation for this struct was generated from the following file:

- src/scene/bvh_builder.h

6.3 BVHAccel::BuildPrimInfo Struct Reference

Public Attributes

- size_t prim_index
- AABB bbox
- Point3 centroid

6.3.1 Detailed Description

Definition at line 22 of file bvh_builder.h.

6.3.2 Member Data Documentation

6.3.2.1 bbox

```
AABB BVHAccel::BuildPrimInfo::bbox
```

Definition at line 24 of file bvh_builder.h.

6.3.2.2 centroid

```
Point3 BVHAccel::BuildPrimInfo::centroid
```

Definition at line 25 of file bvh_builder.h.

6.3.2.3 prim_index

```
size_t BVHAccel::BuildPrimInfo::prim_index
```

Definition at line 23 of file bvh_builder.h.

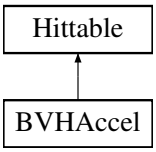
The documentation for this struct was generated from the following file:

- src/scene/bvh_builder.h

6.4 BVHAccel Class Reference

```
#include <bvh_builder.h>
```

Inheritance diagram for BVHAccel:



Classes

- struct BuildPrimInfo
- struct BuildNode

Public Member Functions

- BVHAccel (const HittableList &list)
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override

Public Member Functions inherited from Hittable

- virtual ~Hittable ()=default

Private Member Functions

- BuildNode * recursive_build (std::vector< BuildPrimInfo > &prim_info, int start, int end, std::vector< std::↩ shared_ptr< Hittable > > &src_prims, int &total_nodes)
- int flatten_tree (BuildNode *node, int &offset)
- void delete_tree (BuildNode *node)

Private Attributes

- std::vector< FlatBVHNode > nodes_
- std::vector< std::shared_ptr< Hittable > > ordered_prims_

6.4.1 Detailed Description

Definition at line 11 of file bvh_builder.h.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 BVHAccel()

```
BVHAccel::BVHAccel (
            const HittableList & list)
```

Definition at line 11 of file bvh_builder.cpp.

6.4.3 Member Function Documentation

6.4.3.1 bounding_box()

```
AABB BVHAccel::bounding_box () const  [override], [virtual]
```

Implements Hittable.

Definition at line 250 of file bvh_builder.cpp.

6.4.3.2 delete_tree()

```
void BVHAccel::delete_tree (
            BuildNode * node)  [private]
```

Definition at line 196 of file bvh_builder.cpp.

6.4.3.3 flatten_tree()

```
int BVHAccel::flatten_tree (
            BuildNode * node,
            int & offset)  [private]
```

Definition at line 174 of file bvh_builder.cpp.

6.4.3.4 hit()

```
bool BVHAccel::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [override], [virtual]
```

Implements Hittable.

Definition at line 203 of file bvh_builder.cpp.

6.4.3.5 recursive_build()

```
BVHAccel::BuildNode * BVHAccel::recursive_build (
            std::vector< BuildPrimInfo > & prim_info,
            int start,
            int end,
            std::vector< std::shared_ptr< Hittable > > & src_prims,
            int & total_nodes)  [private]
```

Definition at line 37 of file bvh_builder.cpp.

6.4.4 Member Data Documentation

6.4.4.1 nodes_

```
std::vector<FlatBVHNode> BVHAccel::nodes_  [private]
```

Definition at line 19 of file bvh_builder.h.

6.4.4.2 ordered_prims_

```
std::vector<std::shared_ptr<Hittable> > BVHAccel::ordered_prims_  [private]
```

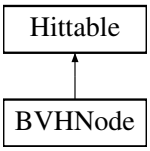
Definition at line 20 of file bvh_builder.h.

The documentation for this class was generated from the following files:

- src/scene/bvh_builder.h
- src/scene/bvh_builder.cpp

6.5 BVHNode Class Reference

Inheritance diagram for BVHNode:



Public Member Functions

- BVHNode (std::vector< std::shared_ptr< Hittable > > &objects, size_t start, size_t end)
- BVHNode (const class HittableList &list)
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override

Public Member Functions inherited from Hittable

- virtual ~Hittable ()=default

Private Attributes

- std::shared_ptr< Hittable > left_
- std::shared_ptr< Hittable > right_
- AABB bbox_
- int split_axis_ = 0

6.5.1 Detailed Description

Definition at line 23 of file bvh_node.cuh.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 BVHNode() [1/2]

```
BVHNode::BVHNode (
            std::vector< std::shared_ptr< Hittable > > & objects,
            size_t start,
            size_t end)
```

Definition at line 264 of file bvh_builder.cpp.

6.5.2.2 BVHNode() [2/2]

```
BVHNode::BVHNode (
            const class HittableList & list)
```

6.5.3 Member Function Documentation

6.5.3.1 bounding_box()

```
AABB BVHNode::bounding_box () const  [override], [virtual]
```

Implements Hittable.

Definition at line 308 of file bvh_builder.cpp.

6.5.3.2 hit()

```
bool BVHNode::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [override], [virtual]
```

Implements Hittable.

Definition at line 293 of file bvh_builder.cpp.

6.5.4 Member Data Documentation

6.5.4.1 bbox_

```
AABB BVHNode::bbox_  [private]
```

Definition at line 34 of file bvh_node.cuh.

6.5.4.2 left_

```
std::shared_ptr<Hittable> BVHNode::left_  [private]
```

Definition at line 32 of file bvh_node.cuh.

6.5.4.3 right_

```
std::shared_ptr<Hittable> BVHNode::right_  [private]
```

Definition at line 33 of file bvh_node.cuh.

6.5.4.4 split_axis_

```
int BVHNode::split_axis_ = 0  [private]
```

Definition at line 35 of file bvh_node.cuh.

The documentation for this class was generated from the following files:

- src/scene/bvh_node.cuh
- src/scene/bvh_builder.cpp

6.6 Camera Class Reference

Public Member Functions

- void initialize ()
- int get_image_width () const
- int get_image_height () const
- Ray get_ray (int i, int j) const

Public Attributes

- double aspect_ratio = 16.0 / 9.0
- int image_width = 400
- int samples_per_pixel = 100
- int max_depth = 50
- double vfov = 90.0
- Point3 lookfrom = Point3(0, 0, 0)
- Point3 lookat = Point3(0, 0, -1)
- Vec3 vup = Vec3(0, 1, 0)
- double defocus_angle = 0.0
- double focus_dist = 10.0

Private Member Functions

- Vec3 sample_square () const
- Point3 defocus_disk_sample () const

Private Attributes

- int image_height_
- Point3 center_
- Point3 pixel00_loc_
- Vec3 pixel_delta_u_
- Vec3 pixel_delta_v_
- Vec3 u_
- Vec3 v_
- Vec3 w_
- Vec3 defocus_disk_u_
- Vec3 defocus_disk_v_

6.6.1 Detailed Description

Definition at line 7 of file camera.cuh.

6.6.2 Member Function Documentation

6.6.2.1 defocus_disk_sample()

```
Point3 Camera::defocus_disk_sample () const  [private]
```

Definition at line 58 of file camera.cpp.

6.6.2.2 get_image_height()

```
int Camera::get_image_height () const  [inline]
```

Definition at line 28 of file camera.cuh.

6.6.2.3 get_image_width()

```
int Camera::get_image_width () const  [inline]
```

Definition at line 27 of file camera.cuh.

6.6.2.4 get_ray()

```
Ray Camera::get_ray (
            int i,
            int j) const
```

Definition at line 42 of file camera.cpp.

6.6.2.5 initialize()

```
void Camera::initialize ()
```

Definition at line 6 of file camera.cpp.

6.6.2.6 sample_square()

```
Vec3 Camera::sample_square () const  [private]
```

Definition at line 54 of file camera.cpp.

6.6.3 Member Data Documentation

6.6.3.1 aspect_ratio

```
double Camera::aspect_ratio = 16.0 / 9.0
```

Definition at line 10 of file camera.cuh.

6.6.3.2 center_

```
Point3 Camera::center_  [private]
```

Definition at line 35 of file camera.cuh.

6.6.3.3 defocus_angle

```
double Camera::defocus_angle = 0.0
```

Definition at line 20 of file camera.cuh.

6.6.3.4 defocus_disk_u_

```
Vec3 Camera::defocus_disk_u_ [private]
```

Definition at line 40 of file camera.cuh.

6.6.3.5 defocus_disk_v_

```
Vec3 Camera::defocus_disk_v_ [private]
```

Definition at line 41 of file camera.cuh.

6.6.3.6 focus_dist

```
double Camera::focus_dist = 10.0
```

Definition at line 21 of file camera.cuh.

6.6.3.7 image_height_

```
int Camera::image_height_ [private]
```

Definition at line 34 of file camera.cuh.

6.6.3.8 image_width

```
int Camera::image_width = 400
```

Definition at line 11 of file camera.cuh.

6.6.3.9 lookat

```
Point3 Camera::lookat = Point3(0, 0, -1)
```

Definition at line 17 of file camera.cuh.

6.6.3.10 lookfrom

```
Point3 Camera::lookfrom = Point3(0, 0, 0)
```

Definition at line 16 of file camera.cuh.

6.6.3.11 max_depth

```
int Camera::max_depth = 50
```

Definition at line 13 of file camera.cuh.

6.6.3.12 pixel00_loc_

```
Point3 Camera::pixel00_loc_ [private]
```

Definition at line 36 of file camera.cuh.

6.6.3.13 pixel_delta_u_

```
Vec3 Camera::pixel_delta_u_ [private]
```

Definition at line 37 of file camera.cuh.

6.6.3.14 pixel_delta_v_

```
Vec3 Camera::pixel_delta_v_ [private]
```

Definition at line 38 of file camera.cuh.

6.6.3.15 samples_per_pixel

```
int Camera::samples_per_pixel = 100
```

Definition at line 12 of file camera.cuh.

6.6.3.16 u_

```
Vec3 Camera::u_ [private]
```

Definition at line 39 of file camera.cuh.

6.6.3.17 v_

```
Vec3 Camera::v_ [private]
```

Definition at line 39 of file camera.cuh.

6.6.3.18 vfov

```
double Camera::vfov = 90.0
```

Definition at line 15 of file camera.cuh.

6.6.3.19 vup

```
Vec3 Camera::vup = Vec3(0, 1, 0)
```

Definition at line 18 of file camera.cuh.

6.6.3.20 w_

```
Vec3 Camera::w_ [private]
```

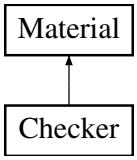
Definition at line 39 of file camera.cuh.

The documentation for this class was generated from the following files:

- src/scene/camera.cuh
- src/scene/camera.cpp

6.7 Checker Class Reference

Inheritance diagram for Checker:



Public Member Functions

- Checker (const Color &color1, const Color &color2, double scale=1.0)
- bool scatter (const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered) const override
- const Color & color1 () const
- const Color & color2 () const
- double scale () const

Public Member Functions inherited from Material

- virtual ~Material ()=default
- Type type () const

Private Attributes

- Color color1_
- Color color2_
- double scale_

Additional Inherited Members

Public Types inherited from Material

- enum class Type : uint8_t {
  Lambertian , Metal , Dielectric , Checker ,
  Emissive , Unknown }

Protected Attributes inherited from Material

- Type type_ = Type::Unknown

6.7.1 Detailed Description

Definition at line 80 of file material.cuh.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 Checker()

```
Checker::Checker (
            const Color & color1,
            const Color & color2,
            double scale = 1.0)
```

Definition at line 89 of file material.cpp.

6.7.3 Member Function Documentation

6.7.3.1 color1()

```
const Color & Checker::color1 () const  [inline]
```

Definition at line 87 of file material.cuh.

6.7.3.2 color2()

```
const Color & Checker::color2 () const  [inline]
```

Definition at line 88 of file material.cuh.

6.7.3.3 scale()

```
double Checker::scale () const  [inline]
```

Definition at line 89 of file material.cuh.

6.7.3.4 scatter()

```
bool Checker::scatter (
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered) const  [override], [virtual]
```

Implements Material.

Definition at line 94 of file material.cpp.

6.7.4 Member Data Documentation

6.7.4.1 color1_

```
Color Checker::color1_ [private]
```

Definition at line 92 of file material.cuh.

6.7.4.2 color2_

```
Color Checker::color2_ [private]
```

Definition at line 92 of file material.cuh.

6.7.4.3 scale_

```
double Checker::scale_ [private]
```

Definition at line 93 of file material.cuh.

The documentation for this class was generated from the following files:

- src/scene/material.cuh
- src/scene/material.cpp

6.8 CpuRenderer Class Reference

```
#include <cpu_renderer.h>
```

Public Member Functions

- CpuRenderer (const Scene &scene, ImageBuffer &image)
- void render ()
- void render_threaded (int num_threads=0)
- void enable_bvh ()
- void render_tile (int x0, int y0, int x1, int y1) const

Private Member Functions

- const Hittable & get_world () const
- Color ray_color (const Ray &r, const Hittable &world, int depth) const

Private Attributes

- const Scene & scene_
- ImageBuffer & image_
- std::shared_ptr< Hittable > bvh_root_
- bool use_bvh_ = false

6.8.1 Detailed Description

Definition at line 10 of file cpu_renderer.h.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 CpuRenderer()

```
CpuRenderer::CpuRenderer (
            const Scene & scene,
            ImageBuffer & image)
```

Definition at line 14 of file cpu_renderer.cpp.

6.8.3 Member Function Documentation

6.8.3.1 enable_bvh()

```
void CpuRenderer::enable_bvh ()
```

Definition at line 17 of file cpu_renderer.cpp.

6.8.3.2 get_world()

```
const Hittable & CpuRenderer::get_world () const  [private]
```

Definition at line 22 of file cpu_renderer.cpp.

6.8.3.3 ray_color()

```
Color CpuRenderer::ray_color (
            const Ray & r,
            const Hittable & world,
            int depth) const  [private]
```

Definition at line 188 of file cpu_renderer.cpp.

6.8.3.4 render()

```
void CpuRenderer::render ()
```

Definition at line 75 of file cpu_renderer.cpp.

6.8.3.5 render_threaded()

```
void CpuRenderer::render_threaded (
            int num_threads = 0)
```

Definition at line 130 of file cpu_renderer.cpp.

6.8.3.6 render_tile()

```
void CpuRenderer::render_tile (
            int x0,
            int y0,
            int x1,
            int y1) const
```

Definition at line 98 of file cpu_renderer.cpp.

6.8.4 Member Data Documentation

6.8.4.1 bvh_root_

```
std::shared_ptr<Hittable> CpuRenderer::bvh_root_ [private]
```

Definition at line 29 of file cpu_renderer.h.

6.8.4.2 image_

```
ImageBuffer& CpuRenderer::image_ [private]
```

Definition at line 28 of file cpu_renderer.h.

6.8.4.3 scene_

```
const Scene& CpuRenderer::scene_ [private]
```

Definition at line 27 of file cpu_renderer.h.

6.8.4.4 use_bvh_

```
bool CpuRenderer::use_bvh_ = false [private]
```

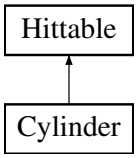
Definition at line 30 of file cpu_renderer.h.

The documentation for this class was generated from the following files:

- src/cpu/cpu_renderer.h
- src/cpu/cpu_renderer.cpp

6.9 Cylinder Class Reference

Inheritance diagram for Cylinder:



Public Member Functions

- Cylinder (const Point3 &base, double radius, double height, const Material ∗material)
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override

Public Member Functions inherited from Hittable

- virtual ∼Hittable ()=default

Private Attributes

- Point3 base_
- double radius_
- double height_
- const Material ∗ material_

6.9.1 Detailed Description

Definition at line 74 of file primitives.cuh.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 Cylinder()

```
Cylinder::Cylinder (
            const Point3 & base,
            double radius,
            double height,
            const Material * material)
```

Definition at line 95 of file primitives.cpp.

6.9.3 Member Function Documentation

6.9.3.1 bounding_box()

```
AABB Cylinder::bounding_box () const  [override], [virtual]
```

Implements Hittable.

Definition at line 134 of file primitives.cpp.

6.9.3.2 hit()

```
bool Cylinder::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [override], [virtual]
```

Implements Hittable.

Definition at line 98 of file primitives.cpp.

6.9.4 Member Data Documentation

6.9.4.1 base_

```
Point3 Cylinder::base_  [private]
```

Definition at line 82 of file primitives.cuh.

6.9.4.2 height_

```
double Cylinder::height_  [private]
```

Definition at line 84 of file primitives.cuh.

6.9.4.3 material_

```
const Material* Cylinder::material_  [private]
```

Definition at line 85 of file primitives.cuh.

6.9.4.4 radius_

```
double Cylinder::radius_  [private]
```

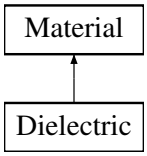
Definition at line 83 of file primitives.cuh.

The documentation for this class was generated from the following files:

- src/scene/primitives.cuh
- src/scene/primitives.cpp

6.10 Dielectric Class Reference

Inheritance diagram for Dielectric:



Public Member Functions

- Dielectric (double refraction_index)
- bool scatter (const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered) const override
- double refraction_index () const

Public Member Functions inherited from Material

- virtual ~Material ()=default
- Type type () const

Static Public Member Functions

- static double reflectance (double cosine, double refraction_index)

Private Attributes

- double refraction_index_

Additional Inherited Members

Public Types inherited from Material

- enum class Type : uint8_t {
  Lambertian , Metal , Dielectric , Checker ,
  Emissive , Unknown }

Protected Attributes inherited from Material

- Type type_ = Type::Unknown

6.10.1 Detailed Description

Definition at line 62 of file material.cuh.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 Dielectric()

```
Dielectric::Dielectric (
            double refraction_index)  [explicit]
```

Definition at line 50 of file material.cpp.

6.10.3 Member Function Documentation

6.10.3.1 reflectance()

```
double Dielectric::reflectance (
            double cosine,
            double refraction_index)  [static]
```

Definition at line 76 of file material.cpp.

6.10.3.2 refraction_index()

```
double Dielectric::refraction_index () const  [inline]
```

Definition at line 69 of file material.cuh.

6.10.3.3 scatter()

```
bool Dielectric::scatter (
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered) const  [override], [virtual]
```

Implements Material.

Definition at line 55 of file material.cpp.

6.10.4 Member Data Documentation

6.10.4.1 refraction_index_

```
double Dielectric::refraction_index_  [private]
```

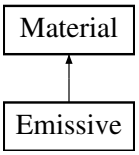
Definition at line 75 of file material.cuh.

The documentation for this class was generated from the following files:

- src/scene/material.cuh
- src/scene/material.cpp

6.11 Emissive Class Reference

Inheritance diagram for Emissive:



Public Member Functions

- Emissive (const Color &emit_color, double intensity=1.0)
- bool scatter (const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered) const override
- Color emitted () const
- const Color & emit_color () const
- double intensity () const

Public Member Functions inherited from Material

- virtual ~Material ()=default
- Type type () const

Private Attributes

- Color emit_
- double intensity_

Additional Inherited Members

Public Types inherited from Material

- enum class Type : uint8_t {
  Lambertian , Metal , Dielectric , Checker ,
  Emissive , Unknown }

Protected Attributes inherited from Material

- Type type_ = Type::Unknown

6.11.1 Detailed Description

Definition at line 98 of file material.cuh.

6.11.2 Constructor & Destructor Documentation

6.11.2.1 Emissive()

```
Emissive::Emissive (
            const Color & emit_color,
            double intensity = 1.0)
```

Definition at line 115 of file material.cpp.

6.11.3 Member Function Documentation

6.11.3.1 emit_color()

```
const Color & Emissive::emit_color () const  [inline]
```

Definition at line 106 of file material.cuh.

6.11.3.2 emitted()

```
Color Emissive::emitted () const  [inline]
```

Definition at line 105 of file material.cuh.

6.11.3.3 intensity()

```
double Emissive::intensity () const  [inline]
```

Definition at line 107 of file material.cuh.

6.11.3.4 scatter()

```
bool Emissive::scatter (
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered) const  [override], [virtual]
```

Implements Material.

Definition at line 120 of file material.cpp.

6.11.4 Member Data Documentation

6.11.4.1 emit_

```
Color Emissive::emit_  [private]
```

Definition at line 110 of file material.cuh.

6.11.4.2 intensity_

```
double Emissive::intensity_ [private]
```

Definition at line 111 of file material.cuh.

The documentation for this class was generated from the following files:

- src/scene/material.cuh
- src/scene/material.cpp

6.12 FlatBVHNode Struct Reference

Public Attributes

- AABB bbox
- union {
  uint32_t primitives_offset
  uint32_t second_child_offset
  };
- uint16_t num_primitives
- uint8_t split_axis
- uint8_t pad

6.12.1 Detailed Description

Definition at line 11 of file bvh_node.cuh.

6.12.2 Member Data Documentation

6.12.2.1 [union]

```
union { ... } FlatBVHNode
```

6.12.2.2 bbox

```
AABB FlatBVHNode::bbox
```

Definition at line 12 of file bvh_node.cuh.

6.12.2.3 num_primitives

```
uint16_t FlatBVHNode::num_primitives
```

Definition at line 17 of file bvh_node.cuh.

6.12.2.4 pad

```
uint8_t FlatBVHNode::pad
```

Definition at line 19 of file bvh_node.cuh.

6.12.2.5 primitives_offset

```
uint32_t FlatBVHNode::primitives_offset
```

Definition at line 14 of file bvh_node.cuh.

6.12.2.6 second_child_offset

```
uint32_t FlatBVHNode::second_child_offset
```

Definition at line 15 of file bvh_node.cuh.

6.12.2.7 split_axis

```
uint8_t FlatBVHNode::split_axis
```

Definition at line 18 of file bvh_node.cuh.

The documentation for this struct was generated from the following file:

- src/scene/bvh_node.cuh

6.13 GAABB Struct Reference

Public Member Functions

- __host__ __device__ GAABB ()
- __host__ __device__ GAABB (const GVec3 &a, const GVec3 &b)
- __host__ __device__ bool hit (const GRay &r, float t_min, float t_max) const

Public Attributes

- GVec3 mn
- GVec3 mx

6.13.1 Detailed Description

Definition at line 77 of file gpu_types.cuh.

6.13.2 Constructor & Destructor Documentation

6.13.2.1 GAABB() [1/2]

```
__host__ __device__ GAABB::GAABB () [inline]
```

Definition at line 80 of file gpu_types.cuh.

6.13.2.2 GAABB() [2/2]

```
__host__ __device__ GAABB::GAABB (
            const GVec3 & a,
            const GVec3 & b) [inline]
```

Definition at line 81 of file gpu_types.cuh.

6.13.3 Member Function Documentation

6.13.3.1 hit()

```
__host__ __device__ bool GAABB::hit (
            const GRay & r,
            float t_min,
            float t_max) const [inline]
```

Definition at line 83 of file gpu_types.cuh.

6.13.4 Member Data Documentation

6.13.4.1 mn

```
GVec3 GAABB::mn
```

Definition at line 78 of file gpu_types.cuh.

6.13.4.2 mx

```
GVec3 GAABB::mx
```

Definition at line 78 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.14 GBuildInfo Struct Reference

Public Attributes

- int sphere_idx
- GAABB bbox
- GVec3 centroid

6.14.1 Detailed Description

Definition at line 723 of file gpu_renderer.cu.

6.14.2 Member Data Documentation

6.14.2.1 bbox

```
GAABB GBuildInfo::bbox
```

Definition at line 723 of file gpu_renderer.cu.

6.14.2.2 centroid

```
GVec3 GBuildInfo::centroid
```

Definition at line 723 of file gpu_renderer.cu.

6.14.2.3 sphere_idx

```
int GBuildInfo::sphere_idx
```

Definition at line 723 of file gpu_renderer.cu.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_renderer.cu

6.15 GBVHNode Struct Reference

Public Attributes

- GAABB bbox
- uint32_t data
- uint16_t num_prims
- uint8_t split_axis
- uint8_t prim_type

6.15.1 Detailed Description

Definition at line 160 of file gpu_types.cuh.

6.15.2 Member Data Documentation

6.15.2.1 bbox

```
GAABB GBVHNode::bbox
```

Definition at line 161 of file gpu_types.cuh.

6.15.2.2 data

```
uint32_t GBVHNode::data
```

Definition at line 162 of file gpu_types.cuh.

6.15.2.3 num_prims

```
uint16_t GBVHNode::num_prims
```

Definition at line 163 of file gpu_types.cuh.

6.15.2.4 prim_type

```
uint8_t GBVHNode::prim_type
```

Definition at line 165 of file gpu_types.cuh.

6.15.2.5 split_axis

```
uint8_t GBVHNode::split_axis
```

Definition at line 164 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.16 GCamera Struct Reference

Public Attributes

- GVec3 origin
- GVec3 pixel00_loc
- GVec3 pixel_delta_u
- GVec3 pixel_delta_v
- GVec3 defocus_disk_u
- GVec3 defocus_disk_v
- float defocus_angle
- int image_width
- int image_height
- int samples_per_pixel
- int max_depth

6.16.1 Detailed Description

Definition at line 171 of file gpu_types.cuh.

6.16.2 Member Data Documentation

6.16.2.1 defocus_angle

```
float GCamera::defocus_angle
```

Definition at line 178 of file gpu_types.cuh.

6.16.2.2 defocus_disk_u

```
GVec3 GCamera::defocus_disk_u
```

Definition at line 176 of file gpu_types.cuh.

6.16.2.3 defocus_disk_v

```
GVec3 GCamera::defocus_disk_v
```

Definition at line 177 of file gpu_types.cuh.

6.16.2.4 image_height

```
int GCamera::image_height
```

Definition at line 180 of file gpu_types.cuh.

6.16.2.5 image_width

```
int GCamera::image_width
```

Definition at line 179 of file gpu_types.cuh.

6.16.2.6 max_depth

```
int GCamera::max_depth
```

Definition at line 182 of file gpu_types.cuh.

6.16.2.7 origin

```
GVec3 GCamera::origin
```

Definition at line 172 of file gpu_types.cuh.

6.16.2.8 pixel00_loc

```
GVec3 GCamera::pixel00_loc
```

Definition at line 173 of file gpu_types.cuh.

6.16.2.9 pixel_delta_u

```
GVec3 GCamera::pixel_delta_u
```

Definition at line 174 of file gpu_types.cuh.

6.16.2.10 pixel_delta_v

```
GVec3 GCamera::pixel_delta_v
```

Definition at line 175 of file gpu_types.cuh.

6.16.2.11 samples_per_pixel

```
int GCamera::samples_per_pixel
```

Definition at line 181 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.17 GHitRecord Struct Reference

Public Member Functions

- __host__ __device__ void set_face_normal (const GRay &r, const GVec3 &outward_normal)

Public Attributes

- GVec3 p
- GVec3 normal
- float t
- int material_id
- bool front_face

6.17.1 Detailed Description

Definition at line 144 of file gpu_types.cuh.

6.17.2 Member Function Documentation

6.17.2.1 set_face_normal()

```
__host__ __device__ void GHitRecord::set_face_normal (
            const GRay & r,
            const GVec3 & outward_normal)  [inline]
```

Definition at line 151 of file gpu_types.cuh.

6.17.3 Member Data Documentation

6.17.3.1 front_face

```
bool GHitRecord::front_face
```

Definition at line 149 of file gpu_types.cuh.

6.17.3.2 material_id

```
int GHitRecord::material_id
```

Definition at line 148 of file gpu_types.cuh.

6.17.3.3 normal

```
GVec3 GHitRecord::normal
```

Definition at line 146 of file gpu_types.cuh.

6.17.3.4 p

```
GVec3 GHitRecord::p
```

Definition at line 145 of file gpu_types.cuh.

6.17.3.5 t

```
float GHitRecord::t
```

Definition at line 147 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.18 GMaterial Struct Reference

Public Member Functions

- __host__ __device__ GMaterial ()

Public Attributes

- GVec3 albedo
- float param
- GMaterialType type

6.18.1 Detailed Description

Definition at line 104 of file gpu_types.cuh.

6.18.2 Constructor & Destructor Documentation

6.18.2.1 GMaterial()

```
__host__ __device__ GMaterial::GMaterial () [inline]
```

Definition at line 109 of file gpu_types.cuh.

6.18.3 Member Data Documentation

6.18.3.1 albedo

```
GVec3 GMaterial::albedo
```

Definition at line 105 of file gpu_types.cuh.

6.18.3.2 param

```
float GMaterial::param
```

Definition at line 106 of file gpu_types.cuh.

6.18.3.3 type

```
GMaterialType GMaterial::type
```

Definition at line 107 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.19 GPlane Struct Reference

Public Attributes

- GVec3 point
- GVec3 normal
- int material_id

6.19.1 Detailed Description

Definition at line 135 of file gpu_types.cuh.

6.19.2 Member Data Documentation

6.19.2.1 material_id

```
int GPlane::material_id
```

Definition at line 138 of file gpu_types.cuh.

6.19.2.2 normal

```
GVec3 GPlane::normal
```

Definition at line 137 of file gpu_types.cuh.

6.19.2.3 point

```
GVec3 GPlane::point
```

Definition at line 136 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.20 GpuRenderer Class Reference

```
#include <gpu_renderer.h>
```

Public Member Functions

- GpuRenderer ()
- ~GpuRenderer ()
- void upload_scene (const GSphere *spheres, int num_spheres, const GPlane *planes, int num_planes, const GMaterial *materials, int num_materials, const GBVHNode *bvh_nodes, int num_bvh_nodes)
- void render (const GCamera &camera, float *h_output, int width, int height)
- float last_render_ms () const
- void ensure_framebuffer (int width, int height)

Private Member Functions

- void free_scene ()
- void free_framebuffer ()

Private Attributes

- GSphere * d_spheres_ = nullptr
- float * d_soa_cx_ = nullptr
- float * d_soa_cy_ = nullptr
- float * d_soa_cz_ = nullptr
- float * d_soa_r_ = nullptr
- int * d_soa_mat_ = nullptr
- GPlane * d_planes_ = nullptr
- GMaterial * d_materials_ = nullptr
- GBVHNode * d_bvh_nodes_ = nullptr
- float * d_framebuffer_ = nullptr
- int num_spheres_ = 0
- int num_planes_ = 0
- int num_materials_ = 0
- int num_bvh_nodes_ = 0
- int fb_width_ = 0
- int fb_height_ = 0
- float last_ms_ = 0.0f

Friends

- float gpu_realtime_render_frame (const Scene &scene)
- void gpu_render_scene_denoised (const Scene &scene, ImageBuffer &image, float &kernel_ms, float sigma_color=0.1f)

6.20.1 Detailed Description

Definition at line 18 of file gpu_renderer.h.

6.20.2 Constructor & Destructor Documentation

6.20.2.1 GpuRenderer()

```
GpuRenderer::GpuRenderer ()
```

Definition at line 443 of file gpu_renderer.cu.

6.20.2.2 ~GpuRenderer()

```
GpuRenderer::~GpuRenderer ()
```

Definition at line 444 of file gpu_renderer.cu.

6.20.3 Member Function Documentation

6.20.3.1 ensure_framebuffer()

```
void GpuRenderer::ensure_framebuffer (
            int width,
            int height)
```

Definition at line 463 of file gpu_renderer.cu.

6.20.3.2 free_framebuffer()

```
void GpuRenderer::free_framebuffer () [private]
```

Definition at line 458 of file gpu_renderer.cu.

6.20.3.3 free_scene()

```
void GpuRenderer::free_scene () [private]
```

Definition at line 446 of file gpu_renderer.cu.

6.20.3.4 last_render_ms()

```
float GpuRenderer::last_render_ms () const  [inline]
```

Definition at line 33 of file gpu_renderer.h.

6.20.3.5 render()

```
void GpuRenderer::render (
            const GCamera & camera,
            float * h_output,
            int width,
            int height)
```

Definition at line 521 of file gpu_renderer.cu.

6.20.3.6 upload_scene()

```
void GpuRenderer::upload_scene (
            const GSphere * spheres,
            int num_spheres,
            const GPlane * planes,
            int num_planes,
            const GMaterial * materials,
            int num_materials,
            const GBVHNode * bvh_nodes,
            int num_bvh_nodes)
```

Definition at line 470 of file gpu_renderer.cu.

6.20.4 Friends And Related Symbol Documentation

6.20.4.1 gpu_realtime_render_frame

```
float gpu_realtime_render_frame (
            const Scene & scene)  [friend]
```

Definition at line 948 of file gpu_renderer.cu.

6.20.4.2 gpu_render_scene_denoised

```
void gpu_render_scene_denoised (
            const Scene & scene,
            ImageBuffer & image,
            float & kernel_ms,
            float sigma_color = 0.1f)  [friend]
```

Definition at line 861 of file gpu_renderer.cu.

6.20.5 Member Data Documentation

6.20.5.1 d_bvh_nodes_

```
GBVHNode* GpuRenderer::d_bvh_nodes_ = nullptr  [private]
```

Definition at line 49 of file gpu_renderer.h.

6.20.5.2 d_framebuffer_

```
float* GpuRenderer::d_framebuffer_ = nullptr  [private]
```

Definition at line 50 of file gpu_renderer.h.

6.20.5.3 d_materials_

```
GMaterial* GpuRenderer::d_materials_ = nullptr  [private]
```

Definition at line 48 of file gpu_renderer.h.

6.20.5.4 d_planes_

```
GPlane* GpuRenderer::d_planes_ = nullptr  [private]
```

Definition at line 47 of file gpu_renderer.h.

6.20.5.5 d_soa_cx_

```
float* GpuRenderer::d_soa_cx_ = nullptr  [private]
```

Definition at line 42 of file gpu_renderer.h.

6.20.5.6 d_soa_cy_

```
float* GpuRenderer::d_soa_cy_ = nullptr  [private]
```

Definition at line 43 of file gpu_renderer.h.

6.20.5.7 d_soa_cz_

```
float* GpuRenderer::d_soa_cz_ = nullptr  [private]
```

Definition at line 44 of file gpu_renderer.h.

6.20.5.8 d_soa_mat_

```
int* GpuRenderer::d_soa_mat_ = nullptr  [private]
```

Definition at line 46 of file gpu_renderer.h.

6.20.5.9 d_soa_r_

```
float* GpuRenderer::d_soa_r_ = nullptr  [private]
```

Definition at line 45 of file gpu_renderer.h.

6.20.5.10 d_spheres_

```
GSphere* GpuRenderer::d_spheres_ = nullptr  [private]
```

Definition at line 40 of file gpu_renderer.h.

6.20.5.11 fb_height_

```
int GpuRenderer::fb_height_ = 0  [private]
```

Definition at line 57 of file gpu_renderer.h.

6.20.5.12 fb_width_

```
int GpuRenderer::fb_width_ = 0  [private]
```

Definition at line 56 of file gpu_renderer.h.

6.20.5.13 last_ms_

```
float GpuRenderer::last_ms_ = 0.0f  [private]
```

Definition at line 59 of file gpu_renderer.h.

6.20.5.14 num_bvh_nodes_

```
int GpuRenderer::num_bvh_nodes_ = 0  [private]
```

Definition at line 55 of file gpu_renderer.h.

6.20.5.15 num_materials_

```
int GpuRenderer::num_materials_ = 0  [private]
```

Definition at line 54 of file gpu_renderer.h.

6.20.5.16 num_planes_

```
int GpuRenderer::num_planes_ = 0  [private]
```

Definition at line 53 of file gpu_renderer.h.

6.20.5.17 num_spheres_

```
int GpuRenderer::num_spheres_ = 0  [private]
```

Definition at line 52 of file gpu_renderer.h.

The documentation for this class was generated from the following files:

- src/gpu/gpu_renderer.h
- src/gpu/gpu_renderer.cu

6.21 GRay Struct Reference

Public Member Functions

- __host__ __device__ GRay ()
- __host__ __device__ GRay (const GVec3 &o, const GVec3 &d)
- __host__ __device__ GVec3 at (float t) const

Public Attributes

- GVec3 orig
- GVec3 dir
- GVec3 inv_dir
- GVec3 orig_mul_inv

6.21.1 Detailed Description

Definition at line 59 of file gpu_types.cuh.

6.21.2 Constructor & Destructor Documentation

6.21.2.1 GRay() [1/2]

```
__host__ __device__ GRay::GRay ()  [inline]
```

Definition at line 65 of file gpu_types.cuh.

6.21.2.2 GRay() [2/2]

```
__host__ __device__ GRay::GRay (
            const GVec3 & o,
            const GVec3 & d)  [inline]
```

Definition at line 66 of file gpu_types.cuh.

6.21.3 Member Function Documentation

6.21.3.1 at()

```
__host__ __device__ GVec3 GRay::at (
            float t) const  [inline]
```

Definition at line 71 of file gpu_types.cuh.

6.21.4 Member Data Documentation

6.21.4.1 dir

```
GVec3 GRay::dir
```

Definition at line 61 of file gpu_types.cuh.

6.21.4.2 inv_dir

```
GVec3 GRay::inv_dir
```

Definition at line 62 of file gpu_types.cuh.

6.21.4.3 orig

```
GVec3 GRay::orig
```

Definition at line 60 of file gpu_types.cuh.

6.21.4.4 orig_mul_inv

```
GVec3 GRay::orig_mul_inv
```

Definition at line 63 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.22 GSphere Struct Reference

Public Attributes

- GVec3 center
- float radius
- int material_id

6.22.1 Detailed Description

Definition at line 116 of file gpu_types.cuh.

6.22.2 Member Data Documentation

6.22.2.1 center

```
GVec3 GSphere::center
```

Definition at line 117 of file gpu_types.cuh.

6.22.2.2 material_id

```
int GSphere::material_id
```

Definition at line 119 of file gpu_types.cuh.

6.22.2.3 radius

```
float GSphere::radius
```

Definition at line 118 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.23 GSphereSoA Struct Reference

Public Attributes

- float * cx
- float * cy
- float * cz
- float * r
- int * mat
- int count

6.23.1 Detailed Description

Definition at line 123 of file gpu_types.cuh.

6.23.2 Member Data Documentation

6.23.2.1 count

```
int GSphereSoA::count
```

Definition at line 129 of file gpu_types.cuh.

6.23.2.2 cx

```
float* GSphereSoA::cx
```

Definition at line 124 of file gpu_types.cuh.

6.23.2.3 cy

```
float* GSphereSoA::cy
```

Definition at line 125 of file gpu_types.cuh.

6.23.2.4 cz

```
float* GSphereSoA::cz
```

Definition at line 126 of file gpu_types.cuh.

6.23.2.5 mat

```
int* GSphereSoA::mat
```

Definition at line 128 of file gpu_types.cuh.

6.23.2.6 r

```
float* GSphereSoA::r
```

Definition at line 127 of file gpu_types.cuh.

The documentation for this struct was generated from the following file:

- src/gpu/gpu_types.cuh

6.24 HitRecord Struct Reference

Public Member Functions

- void set_face_normal (const Ray &r, const Vec3 &outward_normal)

Public Attributes

- Point3 p
- Vec3 normal
- const Material ∗ material = nullptr
- double t = 0.0
- bool front_face = true

6.24.1 Detailed Description

Definition at line 15 of file primitives.cuh.

6.24.2 Member Function Documentation

6.24.2.1 set_face_normal()

```
void HitRecord::set_face_normal (
            const Ray & r,
            const Vec3 & outward_normal)  [inline]
```

Definition at line 23 of file primitives.cuh.

6.24.3 Member Data Documentation

6.24.3.1 front_face

```
bool HitRecord::front_face = true
```

Definition at line 20 of file primitives.cuh.

6.24.3.2 material

```
const Material* HitRecord::material = nullptr
```

Definition at line 18 of file primitives.cuh.

6.24.3.3 normal

```
Vec3 HitRecord::normal
```

Definition at line 17 of file primitives.cuh.

6.24.3.4 p

```
Point3 HitRecord::p
```

Definition at line 16 of file primitives.cuh.

6.24.3.5 t

```
double HitRecord::t = 0.0
```

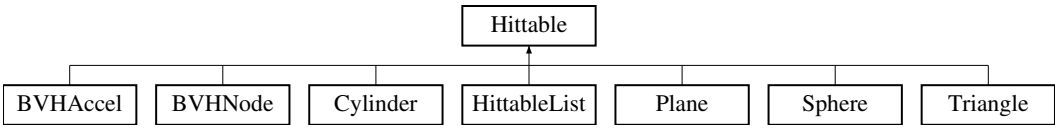
Definition at line 19 of file primitives.cuh.

The documentation for this struct was generated from the following file:

- src/scene/primitives.cuh

6.25 Hittable Class Reference

Inheritance diagram for Hittable:



Public Member Functions

- virtual ~Hittable ()=default
- virtual bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const =0
- virtual AABB bounding_box () const =0

6.25.1 Detailed Description

Definition at line 31 of file primitives.cuh.

6.25.2 Constructor & Destructor Documentation

6.25.2.1 ~Hittable()

```
virtual Hittable::~Hittable () [virtual], [default]
```

6.25.3 Member Function Documentation

6.25.3.1 bounding_box()

```
virtual AABB Hittable::bounding_box () const  [pure virtual]
```

Implemented in BVHAccel, BVHNode, Cylinder, HittableList, Plane, Sphere, and Triangle.

6.25.3.2 hit()

```
virtual bool Hittable::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [pure virtual]
```

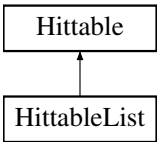
Implemented in BVHAccel, BVHNode, Cylinder, HittableList, Plane, Sphere, and Triangle.

The documentation for this class was generated from the following file:

- src/scene/primitives.cuh

6.26 HittableList Class Reference

Inheritance diagram for HittableList:



Public Member Functions

- HittableList ()=default
- void add (std::shared_ptr< Hittable > object)
- void clear ()
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override
- const std::vector< std::shared_ptr< Hittable > > & objects () const

Public Member Functions inherited from Hittable

- virtual ~Hittable ()=default

Private Attributes

- std::vector< std::shared_ptr< Hittable > > objects_
- AABB bbox_

6.26.1 Detailed Description

Definition at line 105 of file primitives.cuh.

6.26.2 Constructor & Destructor Documentation

6.26.2.1 HittableList()

```
HittableList::HittableList () [default]
```

6.26.3 Member Function Documentation

6.26.3.1 add()

```
void HittableList::add (
            std::shared_ptr< Hittable > object)
```

Definition at line 199 of file primitives.cpp.

6.26.3.2 bounding_box()

```
AABB HittableList::bounding_box () const [override], [virtual]
```

Implements Hittable.

Definition at line 224 of file primitives.cpp.

6.26.3.3 clear()

```
void HittableList::clear ()
```

Definition at line 204 of file primitives.cpp.

6.26.3.4 hit()

```
bool HittableList::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const [override], [virtual]
```

Implements Hittable.

Definition at line 208 of file primitives.cpp.

6.26.3.5 objects()

```
const std::vector< std::shared_ptr< Hittable > > & HittableList::objects () const [inline]
```

Definition at line 115 of file primitives.cuh.

6.26.4 Member Data Documentation

6.26.4.1 bbox_

```
AABB HittableList::bbox_ [private]
```

Definition at line 119 of file primitives.cuh.

6.26.4.2 objects_

```
std::vector<std::shared_ptr<Hittable> > HittableList::objects_ [private]
```

Definition at line 118 of file primitives.cuh.

The documentation for this class was generated from the following files:

- src/scene/primitives.cuh
- src/scene/primitives.cpp

6.27 ImageBuffer Class Reference

```
#include <image.h>
```

Public Member Functions

- ImageBuffer (int width, int height)
- int width () const
- int height () const
- void set_pixel (int x, int y, const Color &color)
- Color get_pixel (int x, int y) const
- void write_ppm (std::ostream &out) const
- void write_ppm (const std::string &filename) const
- void write_ppm_binary (const std::string &filename) const

Private Member Functions

- int index (int x, int y) const

Static Private Member Functions

- static int to_byte (double linear)

Private Attributes

- int width_
- int height_
- std::vector< Color > pixels_

6.27.1 Detailed Description

Definition at line 9 of file image.h.

6.27.2 Constructor & Destructor Documentation

6.27.2.1 ImageBuffer()

```
ImageBuffer::ImageBuffer (
            int width,
            int height)
```

Definition at line 7 of file image.cpp.

6.27.3 Member Function Documentation

6.27.3.1 get_pixel()

```
Color ImageBuffer::get_pixel (
            int x,
            int y) const
```

Definition at line 14 of file image.cpp.

6.27.3.2 height()

```
int ImageBuffer::height () const  [inline]
```

Definition at line 14 of file image.h.

6.27.3.3 index()

```
int ImageBuffer::index (
            int x,
            int y) const  [inline], [private]
```

Definition at line 31 of file image.h.

6.27.3.4 set_pixel()

```
void ImageBuffer::set_pixel (
            int x,
            int y,
            const Color & color)
```

Definition at line 10 of file image.cpp.

6.27.3.5 to_byte()

```
int ImageBuffer::to_byte (
            double linear)  [static], [private]
```

Definition at line 26 of file image.cpp.

6.27.3.6 width()

```
int ImageBuffer::width () const  [inline]
```

Definition at line 13 of file image.h.

6.27.3.7 write_ppm() [1/2]

```
void ImageBuffer::write_ppm (
            const std::string & filename) const
```

Definition at line 45 of file image.cpp.

6.27.3.8 write_ppm() [2/2]

```
void ImageBuffer::write_ppm (
            std::ostream & out) const
```

Definition at line 33 of file image.cpp.

6.27.3.9 write_ppm_binary()

```
void ImageBuffer::write_ppm_binary (
            const std::string & filename) const
```

Definition at line 52 of file image.cpp.

6.27.4 Member Data Documentation

6.27.4.1 height_

```
int ImageBuffer::height_  [private]
```

Definition at line 28 of file image.h.

6.27.4.2 pixels_

```
std::vector<Color> ImageBuffer::pixels_ [private]
```

Definition at line 29 of file image.h.

6.27.4.3 width_

```
int ImageBuffer::width_ [private]
```

Definition at line 27 of file image.h.

The documentation for this class was generated from the following files:

- src/utils/image.h
- src/utils/image.cpp

6.28 Interval Class Reference

Public Member Functions

- RT_HOSTDEV Interval ()
- RT_HOSTDEV Interval (double min, double max)
- RT_HOSTDEV double size () const
- RT_HOSTDEV bool contains (double x) const
- RT_HOSTDEV bool surrounds (double x) const
- RT_HOSTDEV double clamp (double x) const

Public Attributes

- double min
- double max

Static Public Attributes

- static const Interval empty
- static const Interval universe

6.28.1 Detailed Description

Definition at line 13 of file interval.cuh.

6.28.2 Constructor & Destructor Documentation

6.28.2.1 Interval() [1/2]

```
RT_HOSTDEV Interval::Interval () [inline]
```

Definition at line 18 of file interval.cuh.

6.28.2.2 Interval() [2/2]

```
RT_HOSTDEV Interval::Interval (
            double min,
            double max)  [inline]
```

Definition at line 22 of file interval.cuh.

6.28.3 Member Function Documentation

6.28.3.1 clamp()

```
RT_HOSTDEV double Interval::clamp (
            double x) const  [inline]
```

Definition at line 33 of file interval.cuh.

6.28.3.2 contains()

```
RT_HOSTDEV bool Interval::contains (
            double x) const  [inline]
```

Definition at line 27 of file interval.cuh.

6.28.3.3 size()

```
RT_HOSTDEV double Interval::size () const  [inline]
```

Definition at line 24 of file interval.cuh.

6.28.3.4 surrounds()

```
RT_HOSTDEV bool Interval::surrounds (
            double x) const  [inline]
```

Definition at line 31 of file interval.cuh.

6.28.4 Member Data Documentation

6.28.4.1 empty

```
const Interval Interval::empty  [inline], [static]
```

Initial value:

```
= Interval(+std::numeric_limits<double>::infinity(),
           -std::numeric_limits<double>::infinity())
```

Definition at line 39 of file interval.cuh.

6.28.4.2 max

```
double Interval::max
```

Definition at line 15 of file interval.cuh.

6.28.4.3 min

```
double Interval::min
```

Definition at line 15 of file interval.cuh.

6.28.4.4 universe

```
const Interval Interval::universe  [inline], [static]
```

Initial value:

```
= Interval(-std::numeric_limits<double>::infinity(),
           +std::numeric_limits<double>::infinity())
```

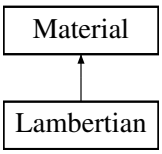
Definition at line 40 of file interval.cuh.

The documentation for this class was generated from the following file:

- src/math/interval.cuh

6.29 Lambertian Class Reference

Inheritance diagram for Lambertian:



Public Member Functions

- Lambertian (const Color &albedo)
- bool scatter (const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered) const override
- const Color & albedo () const

Public Member Functions inherited from Material

- virtual ~Material ()=default
- Type type () const

Private Attributes

- Color albedo_

Additional Inherited Members

Public Types inherited from Material

- enum class Type : uint8_t {
  Lambertian , Metal , Dielectric , Checker ,
  Emissive , Unknown }

Protected Attributes inherited from Material

- Type type_ = Type::Unknown

6.29.1 Detailed Description

Definition at line 30 of file material.cuh.

6.29.2 Constructor & Destructor Documentation

6.29.2.1 Lambertian()

```
Lambertian::Lambertian (
            const Color & albedo)  [explicit]
```

Definition at line 11 of file material.cpp.

6.29.3 Member Function Documentation

6.29.3.1 albedo()

```
const Color & Lambertian::albedo () const  [inline]
```

Definition at line 37 of file material.cuh.

6.29.3.2 scatter()

```
bool Lambertian::scatter (
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered) const  [override], [virtual]
```

Implements Material.

Definition at line 15 of file material.cpp.

6.29.4 Member Data Documentation

6.29.4.1 albedo_

```
Color Lambertian::albedo_ [private]
```

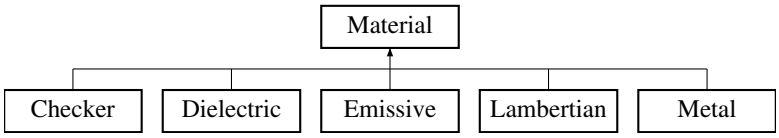
Definition at line 40 of file material.cuh.

The documentation for this class was generated from the following files:

- src/scene/material.cuh
- src/scene/material.cpp

6.30 Material Class Reference

Inheritance diagram for Material:



Public Types

- enum class Type : uint8_t {
  Lambertian , Metal , Dielectric , Checker ,
  Emissive , Unknown }

Public Member Functions

- virtual ~Material ()=default
- virtual bool scatter (const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered) const =0
- Type type () const

Protected Attributes

- Type type_ = Type::Unknown

6.30.1 Detailed Description

Definition at line 12 of file material.cuh.

6.30.2 Member Enumeration Documentation

6.30.2.1 Type

```
enum class Material::Type : uint8_t [strong]
```

Enumerator

Lambertian	
Metal	
Dielectric	
Checker	
Emissive	
Unknown	

Definition at line 15 of file material.cuh.

6.30.3 Constructor & Destructor Documentation

6.30.3.1 ~Material()

```
virtual Material::~Material () [virtual], [default]
```

6.30.4 Member Function Documentation

6.30.4.1 scatter()

```
virtual bool Material::scatter (
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered) const [pure virtual]
```

Implemented in Checker, Dielectric, Emissive, Lambertian, and Metal.

6.30.4.2 type()

```
Type Material::type () const [inline]
```

Definition at line 22 of file material.cuh.

6.30.5 Member Data Documentation

6.30.5.1 type_

```
Type Material::type_ = Type::Unknown [protected]
```

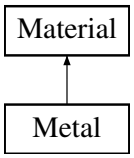
Definition at line 25 of file material.cuh.

The documentation for this class was generated from the following file:

- src/scene/material.cuh

6.31 Metal Class Reference

Inheritance diagram for Metal:



Public Member Functions

- Metal (const Color &albedo, double fuzz)
- bool scatter (const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered) const override
- const Color & albedo () const
- double fuzz () const

Public Member Functions inherited from Material

- virtual ~Material ()=default
- Type type () const

Private Attributes

- Color albedo_
- double fuzz_

Additional Inherited Members

Public Types inherited from Material

- enum class Type : uint8_t {
  Lambertian , Metal , Dielectric , Checker ,
  Emissive , Unknown }

Protected Attributes inherited from Material

- Type type_ = Type::Unknown

6.31.1 Detailed Description

Definition at line 45 of file material.cuh.

6.31.2 Constructor & Destructor Documentation

6.31.2.1 Metal()

```
Metal::Metal (
            const Color & albedo,
            double fuzz)
```

Definition at line 32 of file material.cpp.

6.31.3 Member Function Documentation

6.31.3.1 albedo()

```
const Color & Metal::albedo () const  [inline]
```

Definition at line 52 of file material.cuh.

6.31.3.2 fuzz()

```
double Metal::fuzz () const  [inline]
```

Definition at line 53 of file material.cuh.

6.31.3.3 scatter()

```
bool Metal::scatter (
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered) const  [override], [virtual]
```

Implements Material.

Definition at line 37 of file material.cpp.

6.31.4 Member Data Documentation

6.31.4.1 albedo_

```
Color Metal::albedo_  [private]
```

Definition at line 56 of file material.cuh.

6.31.4.2 fuzz_

```
double Metal::fuzz_  [private]
```

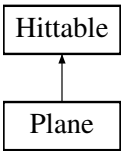
Definition at line 57 of file material.cuh.

The documentation for this class was generated from the following files:

- src/scene/material.cuh
- src/scene/material.cpp

6.32 Plane Class Reference

Inheritance diagram for Plane:



Public Member Functions

- Plane (const Point3 &point, const Vec3 &normal, const Material ∗material)
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override

Public Member Functions inherited from Hittable

- virtual ∼Hittable ()=default

Private Attributes

- Point3 point_
- Vec3 normal_
- const Material ∗ material_

6.32.1 Detailed Description

Definition at line 59 of file primitives.cuh.

6.32.2 Constructor & Destructor Documentation

6.32.2.1 Plane()

```
Plane::Plane (
            const Point3 & point,
            const Vec3 & normal,
            const Material * material)
```

Definition at line 50 of file primitives.cpp.

6.32.3 Member Function Documentation

6.32.3.1 bounding_box()

```
AABB Plane::bounding_box () const  [override], [virtual]
```

Implements Hittable.

Definition at line 73 of file primitives.cpp.

6.32.3.2 hit()

```
bool Plane::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [override], [virtual]
```

Implements Hittable.

Definition at line 53 of file primitives.cpp.

6.32.4 Member Data Documentation

6.32.4.1 material_

```
const Material* Plane::material_  [private]
```

Definition at line 69 of file primitives.cuh.

6.32.4.2 normal_

```
Vec3 Plane::normal_  [private]
```

Definition at line 68 of file primitives.cuh.

6.32.4.3 point_

```
Point3 Plane::point_  [private]
```

Definition at line 67 of file primitives.cuh.

The documentation for this class was generated from the following files:

- src/scene/primitives.cuh
- src/scene/primitives.cpp

6.33 Ray Class Reference

Public Member Functions

- RT_HOSTDEV Ray ()
- RT_HOSTDEV Ray (const Point3 &origin, const Vec3 &direction)
- RT_HOSTDEV const Point3 & origin () const
- RT_HOSTDEV const Vec3 & direction () const
- RT_HOSTDEV Point3 at (double t) const

Public Attributes

- Point3 orig
- Vec3 dir
- Vec3 inv_dir
- int dir_sign [3]

6.33.1 Detailed Description

Definition at line 11 of file ray.cuh.

6.33.2 Constructor & Destructor Documentation

6.33.2.1 Ray() [1/2]

```
RT_HOSTDEV Ray::Ray ()  [inline]
```

Definition at line 18 of file ray.cuh.

6.33.2.2 Ray() [2/2]

```
RT_HOSTDEV Ray::Ray (
            const Point3 & origin,
            const Vec3 & direction)  [inline]
```

Definition at line 19 of file ray.cuh.

6.33.3 Member Function Documentation

6.33.3.1 at()

```
RT_HOSTDEV Point3 Ray::at (
            double t) const  [inline]
```

Definition at line 33 of file ray.cuh.

6.33.3.2 direction()

```
RT_HOSTDEV const Vec3 & Ray::direction () const  [inline]
```

Definition at line 30 of file ray.cuh.

6.33.3.3 origin()

```
RT_HOSTDEV const Point3 & Ray::origin () const  [inline]
```

Definition at line 29 of file ray.cuh.

6.33.4 Member Data Documentation

6.33.4.1 dir

```
Vec3 Ray::dir
```

Definition at line 14 of file ray.cuh.

6.33.4.2 dir_sign

```
int Ray::dir_sign[3]
```

Definition at line 16 of file ray.cuh.

6.33.4.3 inv_dir

```
Vec3 Ray::inv_dir
```

Definition at line 15 of file ray.cuh.

6.33.4.4 orig

```
Point3 Ray::orig
```

Definition at line 13 of file ray.cuh.

The documentation for this class was generated from the following file:

- src/math/ray.cuh

6.34 Scene Class Reference

```
#include <scene.h>
```

Public Member Functions

- template<typename T, typename... Args>
  T * add_material (Args &&... args)

Static Public Member Functions

- static Scene build_three_spheres ()
- static Scene build_final_scene ()
- static Scene build_triangle_scene ()
- static Scene build_cornell_box ()
- static Scene build_showcase ()
- static Scene build_hall ()
- static Scene build_galaxy ()
- static Scene build_minimal ()

Public Attributes

- Camera camera
- HittableList world

Private Attributes

- std::vector< std::unique_ptr< Material > > materials_

6.34.1 Detailed Description

Definition at line 10 of file scene.h.

6.34.2 Member Function Documentation

6.34.2.1 add_material()

```
template<typename T, typename... Args>
T * Scene::add_material (
            Args &&... args) [inline]
```

Definition at line 17 of file scene.h.

6.34.2.2 build_cornell_box()

```
Scene Scene::build_cornell_box () [static]
```

Definition at line 153 of file scene.cpp.

6.34.2.3 build_final_scene()

```
Scene Scene::build_final_scene () [static]
```

Definition at line 45 of file scene.cpp.

6.34.2.4 build_galaxy()

```
Scene Scene::build_galaxy () [static]
```

Definition at line 298 of file scene.cpp.

6.34.2.5 build_hall()

```
Scene Scene::build_hall () [static]
```

Definition at line 259 of file scene.cpp.

6.34.2.6 build_minimal()

```
Scene Scene::build_minimal () [static]
```

Definition at line 335 of file scene.cpp.

6.34.2.7 build_showcase()

```
Scene Scene::build_showcase () [static]
```

Definition at line 208 of file scene.cpp.

6.34.2.8 build_three_spheres()

```
Scene Scene::build_three_spheres () [static]
```

Definition at line 9 of file scene.cpp.

6.34.2.9 build_triangle_scene()

```
Scene Scene::build_triangle_scene () [static]
```

Definition at line 108 of file scene.cpp.

6.34.3 Member Data Documentation

6.34.3.1 camera

```
Camera Scene::camera
```

Definition at line 12 of file scene.h.

6.34.3.2 materials_

```
std::vector<std::unique_ptr<Material> > Scene::materials_ [private]
```

Definition at line 49 of file scene.h.

6.34.3.3 world

```
HittableList Scene::world
```

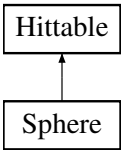
Definition at line 13 of file scene.h.

The documentation for this class was generated from the following files:

- src/scene/scene.h
- src/scene/scene.cpp

6.35 Sphere Class Reference

Inheritance diagram for Sphere:



Public Member Functions

- Sphere (const Point3 &center, double radius, const Material *material)
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override
- const Point3 & center () const
- double radius () const

Public Member Functions inherited from Hittable

- virtual ~Hittable ()=default

Private Attributes

- Point3 center_
- double radius_
- const Material * material_

6.35.1 Detailed Description

Definition at line 41 of file primitives.cuh.

6.35.2 Constructor & Destructor Documentation

6.35.2.1 Sphere()

```
Sphere::Sphere (
            const Point3 & center,
            double radius,
            const Material * material)
```

Definition at line 8 of file primitives.cpp.

6.35.3 Member Function Documentation

6.35.3.1 bounding_box()

```
AABB Sphere::bounding_box () const  [override], [virtual]
```

Implements Hittable.

Definition at line 41 of file primitives.cpp.

6.35.3.2 center()

```
const Point3 & Sphere::center () const  [inline]
```

Definition at line 48 of file primitives.cuh.

6.35.3.3 hit()

```
bool Sphere::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [override], [virtual]
```

Implements Hittable.

Definition at line 11 of file primitives.cpp.

6.35.3.4 radius()

```
double Sphere::radius () const  [inline]
```

Definition at line 49 of file primitives.cuh.

6.35.4 Member Data Documentation

6.35.4.1 center_

```
Point3 Sphere::center_  [private]
```

Definition at line 52 of file primitives.cuh.

6.35.4.2 material_

```
const Material* Sphere::material_  [private]
```

Definition at line 54 of file primitives.cuh.

6.35.4.3 radius_

```
double Sphere::radius_  [private]
```

Definition at line 53 of file primitives.cuh.

The documentation for this class was generated from the following files:

- src/scene/primitives.cuh
- src/scene/primitives.cpp

6.36 Timer Class Reference

```
#include <timer.h>
```

Public Member Functions

- Timer ()
- void reset ()
- double elapsed () const
- double elapsed_ms () const

Private Attributes

- std::chrono::high_resolution_clock::time_point start_

6.36.1 Detailed Description

Definition at line 6 of file timer.h.

6.36.2 Constructor & Destructor Documentation

6.36.2.1 Timer()

```
Timer::Timer () [inline]
```

Definition at line 8 of file timer.h.

6.36.3 Member Function Documentation

6.36.3.1 elapsed()

```
double Timer::elapsed () const [inline]
```

Definition at line 15 of file timer.h.

6.36.3.2 elapsed_ms()

```
double Timer::elapsed_ms () const [inline]
```

Definition at line 21 of file timer.h.

6.36.3.3 reset()

```
void Timer::reset () [inline]
```

Definition at line 10 of file timer.h.

6.36.4 Member Data Documentation

6.36.4.1 start_

```
std::chrono::high_resolution_clock::time_point Timer::start_ [private]
```

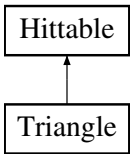
Definition at line 26 of file timer.h.

The documentation for this class was generated from the following file:

- src/utils/timer.h

6.37 Triangle Class Reference

Inheritance diagram for Triangle:



Public Member Functions

- Triangle (const Point3 &v0, const Point3 &v1, const Point3 &v2, const Material *material)
- bool hit (const Ray &r, Interval ray_t, HitRecord &rec) const override
- AABB bounding_box () const override

Public Member Functions inherited from Hittable

- virtual ~Hittable ()=default

Private Attributes

- Point3 v0_
- Point3 v1_
- Point3 v2_
- Vec3 normal_
- const Material * material_

6.37.1 Detailed Description

Definition at line 90 of file primitives.cuh.

6.37.2 Constructor & Destructor Documentation

6.37.2.1 Triangle()

```
Triangle::Triangle (
            const Point3 & v0,
            const Point3 & v1,
            const Point3 & v2,
            const Material * material)
```

Definition at line 145 of file primitives.cpp.

6.37.3 Member Function Documentation

6.37.3.1 bounding_box()

```
AABB Triangle::bounding_box () const  [override], [virtual]
```

Implements Hittable.

Definition at line 181 of file primitives.cpp.

6.37.3.2 hit()

```
bool Triangle::hit (
            const Ray & r,
            Interval ray_t,
            HitRecord & rec) const  [override], [virtual]
```

Implements Hittable.

Definition at line 150 of file primitives.cpp.

6.37.4 Member Data Documentation

6.37.4.1 material_

```
const Material* Triangle::material_  [private]
```

Definition at line 100 of file primitives.cuh.

6.37.4.2 normal_

```
Vec3 Triangle::normal_  [private]
```

Definition at line 99 of file primitives.cuh.

6.37.4.3 v0_

```
Point3 Triangle::v0_ [private]
```

Definition at line 98 of file primitives.cuh.

6.37.4.4 v1_

```
Point3 Triangle::v1_ [private]
```

Definition at line 98 of file primitives.cuh.

6.37.4.5 v2_

```
Point3 Triangle::v2_ [private]
```

Definition at line 98 of file primitives.cuh.

The documentation for this class was generated from the following files:

- src/scene/primitives.cuh
- src/scene/primitives.cpp

6.38 Vec3 Class Reference

Public Member Functions

- RT_HOSTDEV Vec3 ()
- RT_HOSTDEV Vec3 (double x, double y, double z)
- RT_HOSTDEV double x () const
- RT_HOSTDEV double y () const
- RT_HOSTDEV double z () const
- RT_HOSTDEV Vec3 operator- () const
- RT_HOSTDEV double operator[ ] (int i) const
- RT_HOSTDEV double & operator[ ] (int i)
- RT_HOSTDEV Vec3 & operator+= (const Vec3 &v)
- RT_HOSTDEV Vec3 & operator-= (const Vec3 &v)
- RT_HOSTDEV Vec3 & operator*= (double t)
- RT_HOSTDEV Vec3 & operator/= (double t)
- RT_HOSTDEV double length_squared () const
- RT_HOSTDEV double length () const
- RT_HOSTDEV bool near_zero () const

Public Attributes

- double e [3]

6.38.1 Detailed Description

Definition at line 19 of file vec3.cuh.

6.38.2 Constructor & Destructor Documentation

6.38.2.1 Vec3() [1/2]

```
RT_HOSTDEV Vec3::Vec3 ()  [inline]
```

Definition at line 23 of file vec3.cuh.

6.38.2.2 Vec3() [2/2]

```
RT_HOSTDEV Vec3::Vec3 (
            double x,
            double y,
            double z)  [inline]
```

Definition at line 24 of file vec3.cuh.

6.38.3 Member Function Documentation

6.38.3.1 length()

```
RT_HOSTDEV double Vec3::length () const  [inline]
```

Definition at line 58 of file vec3.cuh.

6.38.3.2 length_squared()

```
RT_HOSTDEV double Vec3::length_squared () const  [inline]
```

Definition at line 54 of file vec3.cuh.

6.38.3.3 near_zero()

```
RT_HOSTDEV bool Vec3::near_zero () const  [inline]
```

Definition at line 64 of file vec3.cuh.

6.38.3.4 operator*=()

```
RT_HOSTDEV Vec3 & Vec3::operator*= (
            double t)  [inline]
```

Definition at line 44 of file vec3.cuh.

6.38.3.5 operator+=()

```
RT_HOSTDEV Vec3 & Vec3::operator+= (
            const Vec3 & v)  [inline]
```

Definition at line 34 of file vec3.cuh.

6.38.3.6 operator-()

```
RT_HOSTDEV Vec3 Vec3::operator- () const  [inline]
```

Definition at line 30 of file vec3.cuh.

6.38.3.7 operator-=()

```
RT_HOSTDEV Vec3 & Vec3::operator-= (
            const Vec3 & v)  [inline]
```

Definition at line 39 of file vec3.cuh.

6.38.3.8 operator/=()

```
RT_HOSTDEV Vec3 & Vec3::operator/= (
            double t)  [inline]
```

Definition at line 50 of file vec3.cuh.

6.38.3.9 operator[]() [1/2]

```
RT_HOSTDEV double & Vec3::operator[] (
            int i)  [inline]
```

Definition at line 32 of file vec3.cuh.

6.38.3.10 operator[]() [2/2]

```
RT_HOSTDEV double Vec3::operator[] (
            int i) const  [inline]
```

Definition at line 31 of file vec3.cuh.

6.38.3.11 x()

```
RT_HOSTDEV double Vec3::x () const  [inline]
```

Definition at line 26 of file vec3.cuh.

6.38.3.12 y()

```
RT_HOSTDEV double Vec3::y () const  [inline]
```

Definition at line 27 of file vec3.cuh.

6.38.3.13 z()

```
RT_HOSTDEV double Vec3::z () const  [inline]
```

Definition at line 28 of file vec3.cuh.

6.38.4 Member Data Documentation

6.38.4.1 e

```
double Vec3::e[3]
```

Definition at line 21 of file vec3.cuh.

The documentation for this class was generated from the following file:

- src/math/vec3.cuh

Chapter 7

File Documentation

7.1 src/cpu/cpu_renderer.cpp File Reference

```
#include "cpu/cpu_renderer.h"
#include "math/random.cuh"
#include "math/interval.cuh"
#include "scene/material.cuh"
#include "scene/primitives.cuh"
#include "scene/bvh_builder.h"
#include <iostream>
#include <limits>
#include <thread>
#include <atomic>
#include <vector>
#include <cmath>
```

Functions

- static bool scatter_inline (const Material *mat, const Ray &r_in, const HitRecord &rec, Color &attenuation, Ray &scattered)

7.1.1 Function Documentation

7.1.1.1 scatter_inline()

```
bool scatter_inline (
            const Material * mat,
            const Ray & r_in,
            const HitRecord & rec,
            Color & attenuation,
            Ray & scattered)  [inline], [static]
```

Definition at line 29 of file cpu_renderer.cpp.

7.2 cpu_renderer.cpp

Go to the documentation of this file.

```
#include "cpu/cpu_renderer.h"
#include "math/random.cuh"
#include "math/interval.cuh"
#include "scene/material.cuh"
#include "scene/primitives.cuh"
#include "scene/bvh_builder.h"
#include <iostream>
#include <limits>
#include <thread>
#include <atomic>
#include <vector>
#include <cmath>

CpuRenderer::CpuRenderer(const Scene& scene, ImageBuffer& image)
    : scene_(scene), image_(image) {}

void CpuRenderer::enable_bvh() {
    bvh_root_ = std::make_shared<BVHAccel>(scene_.world);
    use_bvh_ = true;
}

const Hittable& CpuRenderer::get_world() const {
    if (use_bvh_ && bvh_root_)
        return *bvh_root_;
    return scene_.world;
}

// Inline scatter to avoid virtual dispatch in the hot path
static inline bool scatter_inline(const Material* mat, const Ray& r_in,
                                  const HitRecord& rec,
                                  Color& attenuation, Ray& scattered) {
    switch (mat->type()) {
    case Material::Type::Lambertian: {
        const auto* lam = static_cast<const Lambertian*>(mat);
        Vec3 scatter_dir = rec.normal + random_unit_vector();
        if (scatter_dir.near_zero()) scatter_dir = rec.normal;
        scattered = Ray(rec.p, scatter_dir);
        attenuation = lam->albedo();
        return true;
    }
    case Material::Type::Metal: {
        const auto* met = static_cast<const Metal*>(mat);
        Vec3 reflected = reflect(unit_vector(r_in.direction()), rec.normal);
        reflected = reflected + met->fuzz() * random_in_unit_sphere();
        scattered = Ray(rec.p, reflected);
        attenuation = met->albedo();
        return (dot(scattered.direction(), rec.normal) > 0.0);
    }
    case Material::Type::Dielectric: {
        const auto* die = static_cast<const Dielectric*>(mat);
        attenuation = Color(1.0, 1.0, 1.0);
        double ri_val = die->refraction_index();
        double ri = rec.front_face ? (1.0 / ri_val) : ri_val;
        Vec3 unit_dir = unit_vector(r_in.direction());
        double cos_theta = std::fmin(dot(-unit_dir, rec.normal), 1.0);
        double sin_theta = std::sqrt(1.0 - cos_theta * cos_theta);
        bool cannot_refract = ri * sin_theta > 1.0;
        Vec3 direction;
        if (cannot_refract || Dielectric::reflectance(cos_theta, ri) > random_double())
            direction = reflect(unit_dir, rec.normal);
        else
            direction = refract(unit_dir, rec.normal, ri);
        scattered = Ray(rec.p, direction);
        return true;
    }
    case Material::Type::Checker: {
        // Devirtualise comme Lambertian mais avec couleur alternee
        return mat->scatter(r_in, rec, attenuation, scattered);
    }
    default:
        return mat->scatter(r_in, rec, attenuation, scattered);
    }
}

void CpuRenderer::render() {
    const int width = image_.width();
    const int height = image_.height();
    const int spp = scene_.camera.samples_per_pixel;
    const int max_depth = scene_.camera.max_depth;
    const double inv_spp = 1.0 / spp;
    const Hittable& world = get_world();
    const Camera& cam = scene_.camera;
```

```cpp

    for (int j = 0; j < height; ++j) {
        std::cerr << "\rScanlines remaining: " << (height - j) << ' ' << std::flush;
        for (int i = 0; i < width; ++i) {
            Color pixel_color(0, 0, 0);
            for (int s = 0; s < spp; ++s) {
                Ray r = cam.get_ray(i, j);
                pixel_color += ray_color(r, world, max_depth);
            }
            image_.set_pixel(i, j, pixel_color * inv_spp);
        }
    }
    std::cerr << "\rDone.                          \n";
}

void CpuRenderer::render_tile(int x0, int y0, int x1, int y1) const {
    const int spp = scene_.camera.samples_per_pixel;
    const int max_depth = scene_.camera.max_depth;
    const double inv_spp = 1.0 / spp;
    const Hittable& world = get_world();
    const Camera& cam = scene_.camera;

    // Process samples in pairs for better ILP
    const int spp_pairs = spp / 2;
    const int spp_remainder = spp % 2;

    for (int j = y0; j < y1; ++j) {
        for (int i = x0; i < x1; ++i) {
            Color acc0(0, 0, 0);
            Color acc1(0, 0, 0);

            for (int s = 0; s < spp_pairs; ++s) {
                Ray r0 = cam.get_ray(i, j);
                acc0 += ray_color(r0, world, max_depth);
                Ray r1 = cam.get_ray(i, j);
                acc1 += ray_color(r1, world, max_depth);
            }
            if (spp_remainder) {
                Ray r = cam.get_ray(i, j);
                acc0 += ray_color(r, world, max_depth);
            }

            image_.set_pixel(i, j, (acc0 + acc1) * inv_spp);
        }
    }
}

void CpuRenderer::render_threaded(int num_threads) {
    if (num_threads <= 0)
        num_threads = static_cast<int>(std::thread::hardware_concurrency());
    if (num_threads <= 0)
        num_threads = 4;

    const int width = image_.width();
    const int height = image_.height();
    constexpr int TILE_SIZE = 16;

    const int tiles_x = (width + TILE_SIZE - 1) / TILE_SIZE;
    const int tiles_y = (height + TILE_SIZE - 1) / TILE_SIZE;
    const int total_tiles = tiles_x * tiles_y;

    std::atomic<int> next_tile(0);
    std::atomic<int> completed_tiles(0);

    auto worker = [&]() {
        while (true) {
            int tile_idx = next_tile.fetch_add(1);
            if (tile_idx >= total_tiles)
                break;

            int tx = tile_idx % tiles_x;
            int ty = tile_idx / tiles_x;
            int x0 = tx * TILE_SIZE;
            int y0 = ty * TILE_SIZE;
            int x1 = std::min(x0 + TILE_SIZE, width);
            int y1 = std::min(y0 + TILE_SIZE, height);

            render_tile(x0, y0, x1, y1);

            int done = completed_tiles.fetch_add(1) + 1;
            if (done % 10 == 0 || done == total_tiles) {
                std::cerr << "\rTiles: " << done << "/" << total_tiles
                          << " (" << (100 * done / total_tiles) << "%)" << std::flush;
            }
        }
    };

```

```
    std::cerr « "Rendering with " « num_threads « " threads, "
              « total_tiles « " tiles (" « TILE_SIZE « "x" « TILE_SIZE « ")...\n";

    std::vector<std::thread> threads;
    threads.reserve(num_threads);
    for (int i = 0; i < num_threads; ++i)
        threads.emplace_back(worker);

    for (auto& t : threads)
        t.join();

    std::cerr « "\rDone.                                        \n";
}

// CS:APP 5.5 + 5.6: Iterative ray_color with devirtualized scatter
// - No recursion (eliminates stack frame per bounce)
// - Inline scatter avoids vtable lookup per bounce
// - Local variables eliminate repeated memory dereferences
Color CpuRenderer::ray_color(const Ray& r, const Hittable& world, int depth) const {
    Color throughput(1, 1, 1);
    Ray current_ray = r;
    constexpr double T_MIN = 0.001;
    constexpr double T_MAX = 1e30; // CS:APP 5.6: avoid std::numeric_limits call

    for (int d = 0; d < depth; ++d) {
        HitRecord rec;
        if (!world.hit(current_ray, Interval(T_MIN, T_MAX), rec)) {
            // Sky: CS:APP 5.6 - compute with local values
            const Vec3& dir = current_ray.direction();
            // Inline unit_vector Y component to avoid full normalization
            // We only need the Y component for the sky gradient
            double len_sq = dir.length_squared();
            double inv_len = 1.0 / std::sqrt(len_sq);
            double unit_y = dir.y() * inv_len;
            double a = 0.5 * (unit_y + 1.0);
            // Sky color: lerp white->blue
            Color sky(1.0 - 0.5 * a, 1.0 - 0.3 * a, 1.0);
            return throughput * sky;
        }

        // CS:APP 5.5: Devirtualized scatter - switch on type tag, no vtable
        const Material* mat = rec.material;
        if (!mat) break;

        // Area light : si emissif, retourner la lumiere emise
        if (mat->type() == Material::Type::Emissive) {
            const auto* em = static_cast<const Emissive*>(mat);
            return throughput * em->emitted();
        }

        Ray scattered;
        Color attenuation;
        if (!scatter_inline(mat, current_ray, rec, attenuation, scattered))
            break; // absorbed

        // CS:APP 5.6: Multiply throughput components directly
        throughput = throughput * attenuation;

        // Early termination + Russian Roulette (RTIOW CUDA article Opt#4+5)
        double max_comp = std::fmax(throughput.x(), std::fmax(throughput.y(), throughput.z()));
        if (max_comp < 0.001)
            break;
        if (d > 3) {
            if (random_double() > max_comp)
                break;
            throughput = throughput / max_comp;
        }

        current_ray = scattered;
    }

    return Color(0, 0, 0);
}
```

7.3 src/cpu/cpu_renderer.h File Reference

```
#include "scene/scene.h"
#include "utils/image.h"
#include <memory>
```

Classes

- class CpuRenderer

7.4 cpu_renderer.h

Go to the documentation of this file.

```
#ifndef RT_CPU_RENDERER_H
#define RT_CPU_RENDERER_H

#include "scene/scene.h"
#include "utils/image.h"
#include <memory>

class Hittable;

class CpuRenderer {
public:
    CpuRenderer(const Scene& scene, ImageBuffer& image);

    // Single-threaded render
    void render();

    // Multi-threaded render with tile decomposition
    void render_threaded(int num_threads = 0);  // 0 = auto-detect

    // Enable BVH acceleration (builds BVH from scene)
    void enable_bvh();

    // Render a single tile (used by hybrid renderer)
    void render_tile(int x0, int y0, int x1, int y1) const;

private:
    const Scene& scene_;
    ImageBuffer& image_;
    std::shared_ptr<Hittable> bvh_root_;
    bool use_bvh_ = false;

    const Hittable& get_world() const;
    Color ray_color(const Ray& r, const Hittable& world, int depth) const;
};

#endif // RT_CPU_RENDERER_H
```

7.5 src/gpu/gpu_realtime.cpp File Reference

```
#include "gpu/gpu_realtime.h"
#include <cstdio>
```

Functions

- void gpu_realtime_render (const Scene &, int, int)

7.5.1 Function Documentation

7.5.1.1 gpu_realtime_render()

```
void gpu_realtime_render (
            const Scene & scene,
            int width,
            int height)
```

Definition at line 173 of file gpu_realtime.cpp.

7.6 gpu_realtime.cpp

Go to the documentation of this file.

```
// Real-time GPU raytracer with SDL3
// Pipeline GPU-side : render -> accumulate -> tonemap -> ARGB, tout sur GPU
// Seulement le buffer ARGB 8-bit est transfere au host (4 bytes/px vs 12 bytes/px)

#ifdef RT_REALTIME_ENABLED

#include "gpu/gpu_realtime.h"
#include "gpu/gpu_renderer.h"
#include <SDL3/SDL.h>
#include <cstdio>
#include <cmath>
#include <algorithm>
#include <chrono>
#include <cstring>

void gpu_realtime_render(const Scene& scene, int width, int height) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        fprintf(stderr, "SDL_Init error: %s\n", SDL_GetError());
        return;
    }

    Scene& rt_scene = const_cast<Scene&>(scene);
    int orig_w = rt_scene.camera.image_width;
    int orig_spp = rt_scene.camera.samples_per_pixel;
    int orig_depth = rt_scene.camera.max_depth;
    Point3 orig_from = rt_scene.camera.lookfrom;
    Point3 orig_at = rt_scene.camera.lookat;

    rt_scene.camera.image_width = width;
    // Le spp demande par l'utilisateur est respecte
    // Pas de changement dynamique : si --spp 500, c'est 500
    rt_scene.camera.max_depth = 50;
    rt_scene.camera.initialize();
    int h = rt_scene.camera.get_image_height();

    SDL_Window* window = SDL_CreateWindow("Raytracer [GPU Real-time]", width, h, 0);
    if (!window) { fprintf(stderr, "SDL error: %s\n", SDL_GetError()); SDL_Quit(); return; }

    SDL_Renderer* sdl_renderer = SDL_CreateRenderer(window, NULL);
    SDL_Texture* texture = SDL_CreateTexture(sdl_renderer,
        SDL_PIXELFORMAT_ARGB8888, SDL_TEXTUREACCESS_STREAMING, width, h);

    // Init GPU-side pipeline
    gpu_realtime_init(rt_scene, width, h);

    // Host ARGB buffer (pinned, alloue par gpu_realtime_init)
    unsigned char* display = new unsigned char[width * h * 4];

    Vec3 cam_pos = rt_scene.camera.lookfrom;
    Vec3 cam_dir = unit_vector(rt_scene.camera.lookat - cam_pos);
    double cam_yaw = std::atan2(cam_dir.x(), -cam_dir.z());
    double cam_pitch = std::asin(std::clamp(cam_dir.y(), -0.99, 0.99));
    double move_speed = 2.0, mouse_sens = 0.003;
    bool mouse_captured = false, camera_moved = false;

    fprintf(stderr, "Real-time GPU-side: %dx%d | WASD + Souris | S=screenshot | ESC=quit\n", width, h);

    bool running = true;
    int frame = 0;
    int frames_since_move = 0;
    auto t_last_fps = std::chrono::high_resolution_clock::now();
    int frames_since_fps = 0;
    float current_fps = 0, last_km = 0;

    while (running) {
        camera_moved = false;

        SDL_Event event;
        while (SDL_PollEvent(&event)) {
            switch (event.type) {
            case SDL_EVENT_QUIT: running = false; break;
            case SDL_EVENT_KEY_DOWN:
                if (event.key.key == SDLK_ESCAPE) running = false;
                if (event.key.key == SDLK_S) {
                    // Screenshot : sauve l'image actuelle en PPM
                    char fname[128];
                    snprintf(fname, sizeof(fname), "output/screenshot_%d.ppm", frame);
                    ImageBuffer screenshot(width, h);
                    // Copier depuis le display ARGB vers ImageBuffer
                    for (int y = 0; y < h; y++)
                        for (int x = 0; x < width; x++) {
```

```
                            int idx = (y * width + x) * 4;
                            double b = display[idx] / 255.0;
                            double g = display[idx+1] / 255.0;
                            double r = display[idx+2] / 255.0;
                            // Inverse gamma+ACES pour stocker lineaire dans ImageBuffer
                            screenshot.set_pixel(x, y, Color(r*r, g*g, b*b));
                        }
                    screenshot.write_ppm(fname);
                    fprintf(stderr, "Screenshot saved: %s\n", fname);
                }
                break;
            case SDL_EVENT_MOUSE_BUTTON_DOWN:
                if (!mouse_captured) { SDL_SetWindowRelativeMouseMode(window, true); mouse_captured = true; }
                break;
            case SDL_EVENT_MOUSE_MOTION:
                if (mouse_captured && (event.motion.xrel != 0 || event.motion.yrel != 0)) {
                    cam_yaw += event.motion.xrel * mouse_sens;
                    cam_pitch -= event.motion.yrel * mouse_sens;
                    cam_pitch = std::clamp(cam_pitch, -1.5, 1.5);
                    camera_moved = true;
                }
                break;
            }
        }

        const bool* keys = SDL_GetKeyboardState(NULL);
        Vec3 forward(std::sin(cam_yaw)*std::cos(cam_pitch), std::sin(cam_pitch), -std::cos(cam_yaw)*std::cos(cam_pitch));
        Vec3 right_vec = unit_vector(cross(forward, Vec3(0,1,0)));
        double speed = move_speed * (keys[SDL_SCANCODE_LSHIFT] ? 5.0 : 1.0);
        double dt = (current_fps > 0) ? 1.0 / current_fps : 0.016;

        Vec3 old_pos = cam_pos;
        if (keys[SDL_SCANCODE_W]) cam_pos = cam_pos + forward * (speed * dt);
        if (keys[SDL_SCANCODE_S] && !keys[SDL_SCANCODE_LCTRL]) cam_pos = cam_pos - forward * (speed * dt);
        if (keys[SDL_SCANCODE_A]) cam_pos = cam_pos - right_vec * (speed * dt);
        if (keys[SDL_SCANCODE_D]) cam_pos = cam_pos + right_vec * (speed * dt);
        if (keys[SDL_SCANCODE_SPACE]) cam_pos = cam_pos + Vec3(0,1,0) * (speed * dt);
        if (keys[SDL_SCANCODE_LCTRL]) cam_pos = cam_pos - Vec3(0,1,0) * (speed * dt);
        if ((cam_pos - old_pos).length_squared() > 1e-10) camera_moved = true;

        if (camera_moved)
            gpu_realtime_reset_accum();

        rt_scene.camera.lookfrom = cam_pos;
        rt_scene.camera.lookat = cam_pos + forward;
        rt_scene.camera.initialize();

        last_km = gpu_realtime_render_frame(rt_scene);
        frame++;
        frames_since_fps++;

        // Tonemap GPU -> ARGB host (seulement 4 bytes/px transferes)
        int total_spp = gpu_realtime_tonemap(display, width, h);

        SDL_UpdateTexture(texture, NULL, display, width * 4);
        SDL_RenderTexture(sdl_renderer, texture, NULL, NULL);
        SDL_RenderPresent(sdl_renderer);

        auto t_now = std::chrono::high_resolution_clock::now();
        double elapsed = std::chrono::duration<double>(t_now - t_last_fps).count();
        if (elapsed >= 0.5) {
            current_fps = (float)(frames_since_fps / elapsed);
            frames_since_fps = 0;
            t_last_fps = t_now;
            char title[256];
            snprintf(title, sizeof(title),
                "Raytracer [GPU] %dx%d | %.1f FPS | %.1fms | %d spp | pos(%.1f,%.1f,%.1f) | S=screenshot",
                width, h, current_fps, last_km,
                total_spp * rt_scene.camera.samples_per_pixel,
                cam_pos.x(), cam_pos.y(), cam_pos.z());
            SDL_SetWindowTitle(window, title);
        }
    }

    rt_scene.camera.image_width = orig_w;
    rt_scene.camera.samples_per_pixel = orig_spp;
    rt_scene.camera.max_depth = orig_depth;
    rt_scene.camera.lookfrom = orig_from;
    rt_scene.camera.lookat = orig_at;
    rt_scene.camera.initialize();

    delete[] display;
    SDL_DestroyTexture(texture);
```

```
    SDL_DestroyRenderer(sdl_renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
}

#else
#include "gpu/gpu_realtime.h"
#include <cstdio>
void gpu_realtime_render(const Scene&, int, int) {
    fprintf(stderr, "Real-time requires SDL3\n");
}
#endif
```

7.7 src/gpu/gpu_realtime.h File Reference

```
#include "scene/scene.h"
#include "utils/image.h"
```

Functions

- void gpu_realtime_render (const Scene &scene, int width, int height)

7.7.1 Function Documentation

7.7.1.1 gpu_realtime_render()

```
void gpu_realtime_render (
            const Scene & scene,
            int width,
            int height)
```

Definition at line 173 of file gpu_realtime.cpp.

7.8 gpu_realtime.h

Go to the documentation of this file.

```
#ifndef RT_GPU_REALTIME_H
#define RT_GPU_REALTIME_H

#include "scene/scene.h"
#include "utils/image.h"

// Lance le rendu GPU temps reel dans une fenetre Win32
// Boucle infinie : 1 spp par frame, affiche FPS
// Appuyer sur ESC pour quitter
void gpu_realtime_render(const Scene& scene, int width, int height);

#endif
```

7.9 src/gpu/gpu_renderer.cu File Reference

```
#include "gpu/gpu_types.cuh"
#include "gpu/gpu_renderer.h"
#include "scene/primitives.cuh"
#include "scene/material.cuh"
#include <cstdio>
#include <cmath>
#include <vector>
#include <curand_kernel.h>
```

Classes

- struct GBuildInfo

Macros

- #define BVH_SHARED_NODES 7
- #define d_camera (∗(const GCamera∗)d_camera_bytes)
- #define CUDA_CHECK(call)

Typedefs

- typedef curandState GRng

Functions

- __device__ void grng_init (GRng &state, uint32_t tid, uint32_t seed)
- __device__ float grng_float (GRng &state)
- __device__ GVec3 random_in_unit_sphere (GRng &rng)
- __device__ GVec3 random_unit_vector (GRng &rng)
- __device__ GVec3 random_in_unit_disk (GRng &rng)
- __device__ bool intersect_sphere (const GSphere &s, const GRay &r, float t_min, float t_max, GHitRecord &rec)
- __device__ bool intersect_sphere_soa (const GSphereSoA &soa, int idx, const GRay &r, float t_min, float t_max, GHitRecord &rec)
- __device__ bool intersect_plane (const GPlane &pl, const GRay &r, float t_min, float t_max, GHitRecord &rec)
- __device__ bool trace_bvh (const GBVHNode ∗__restrict__ nodes, int num_nodes, const GSphereSoA &soa, const GPlane ∗__restrict__ planes, const GRay &r, float t_min, float t_max, GHitRecord &rec, const GBVHNode ∗__restrict__ sh_bvh, int sh_count)
- __device__ bool trace_brute (const GSphere ∗spheres, int num_spheres, const GPlane ∗planes, int num_↩ planes, const GRay &r, float t_min, float t_max, GHitRecord &rec)
- __device__ bool scatter_mat (const GMaterial &mat, const GRay &r_in, const GHitRecord &rec, GVec3 &attenuation, GRay &scattered, GRng &rng)
- __device__ GVec3 ray_color (GRay r, const GBVHNode ∗bvh, int num_bvh, const GSphereSoA &soa, const GSphere ∗spheres, int num_spheres, const GPlane ∗planes, int num_planes, const GMaterial ∗materials, int max_depth, GRng &rng, const GBVHNode ∗sh_bvh, int sh_count)

- __global__ void render_kernel (float *__restrict__ framebuffer, int width, int height, const GBVHNode *__restrict__ bvh, int num_bvh, const GSphere *__restrict__ spheres, int num_spheres, const GPlane *__restrict__ planes, int num_planes, const GMaterial *__restrict__ materials, const float *__restrict__ soa_cx, const float *__restrict__ soa_cy, const float *__restrict__ soa_cz, const float *__restrict__ soa_r, const int *__restrict__ soa_mat)
- __device__ float aces_tonemap_gpu (float x)
- __global__ void tonemap_kernel (const float *__restrict__ linear_fb, unsigned char *__restrict__ argb_out, int width, int height, int total_samples)
- __global__ void accumulate_kernel (const float *__restrict__ frame_fb, float *__restrict__ accum_fb, int total_pixels)
- __global__ void denoise_bilateral_kernel (const float *__restrict__ input, float *__restrict__ output, int width, int height, float sigma_color)
- static void convert_scene (const Scene &scene, std::vector< GSphere > &out_spheres, std::vector< GPlane > &out_planes, std::vector< GMaterial > &out_materials, GCamera &out_camera)
- static void convert_camera_only (const Camera &cam, GCamera &out)
- static void build_gpu_bvh (std::vector< GBuildInfo > &info, int start, int end, std::vector< GBVHNode > &nodes, int num_planes)
- void gpu_render_scene (const Scene &scene, ImageBuffer &image, float &kernel_ms)
- void gpu_render_scene_denoised (const Scene &scene, ImageBuffer &image, float &kernel_ms, float sigma_color)
- void gpu_realtime_init (const Scene &scene, int width, int height)
- float gpu_realtime_render_frame (const Scene &scene)
- int gpu_realtime_tonemap (unsigned char *argb_host, int width, int height)
- void gpu_realtime_reset_accum ()

Variables

- __constant__ char d_camera_bytes [sizeof(GCamera)]
- GpuRenderer * s_renderer = nullptr
- static float * s_rt_frame_fb = nullptr
- static float * s_rt_accum_fb = nullptr
- static unsigned char * s_rt_argb = nullptr
- static unsigned char * s_rt_argb_host = nullptr
- static int s_rt_w = 0
- static int s_rt_h = 0
- static int s_rt_accum_count = 0
- static cudaStream_t s_rt_stream = nullptr
- static dim3 s_rt_block
- static dim3 s_rt_grid
- static int s_rt_acc_grid = 0
- static cudaEvent_t s_rt_start = nullptr
- static cudaEvent_t s_rt_stop = nullptr
- static bool s_rt_config_done = false

7.9.1 Macro Definition Documentation

7.9.1.1 BVH_SHARED_NODES

```
#define BVH_SHARED_NODES 7
```

Definition at line 110 of file gpu_renderer.cu.

7.9.1.2 CUDA_CHECK

```
#define CUDA_CHECK(
            call)
```

Value:

```
    do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        return; \
    } \
} while(0)
```

Definition at line 435 of file gpu_renderer.cu.

7.9.1.3 d_camera

```
#define d_camera (*(const GCamera*)d_camera_bytes)
```

Definition at line 286 of file gpu_renderer.cu.

7.9.2 Typedef Documentation

7.9.2.1 GRng

```
typedef curandState GRng
```

Definition at line 16 of file gpu_renderer.cu.

7.9.3 Function Documentation

7.9.3.1 accumulate_kernel()

```
__global__ void accumulate_kernel (
            const float *__restrict__ frame_fb,
            float *__restrict__ accum_fb,
            int total_pixels)
```

Definition at line 383 of file gpu_renderer.cu.

7.9.3.2 aces_tonemap_gpu()

```
__device__ float aces_tonemap_gpu (
            float x)
```

Definition at line 352 of file gpu_renderer.cu.

7.9.3.3 build_gpu_bvh()

```
void build_gpu_bvh (
            std::vector< GBuildInfo > & info,
            int start,
            int end,
            std::vector< GBVHNode > & nodes,
            int num_planes)  [static]
```

Definition at line 725 of file gpu_renderer.cu.

7.9.3.4 convert_camera_only()

```
void convert_camera_only (
            const Camera & cam,
            GCamera & out)  [static]
```

Definition at line 695 of file gpu_renderer.cu.

7.9.3.5 convert_scene()

```
void convert_scene (
            const Scene & scene,
            std::vector< GSphere > & out_spheres,
            std::vector< GPlane > & out_planes,
            std::vector< GMaterial > & out_materials,
            GCamera & out_camera)  [static]
```

Definition at line 593 of file gpu_renderer.cu.

7.9.3.6 denoise_bilateral_kernel()

```
__global__ void denoise_bilateral_kernel (
            const float *__restrict__ input,
            float *__restrict__ output,
            int width,
            int height,
            float sigma_color)
```

Definition at line 395 of file gpu_renderer.cu.

7.9.3.7 gpu_realtime_init()

```
void gpu_realtime_init (
            const Scene & scene,
            int width,
            int height)
```

Definition at line 917 of file gpu_renderer.cu.

7.9.3.8 gpu_realtime_render_frame()

```
float gpu_realtime_render_frame (
            const Scene & scene)
```

Definition at line 948 of file gpu_renderer.cu.

7.9.3.9 gpu_realtime_reset_accum()

```
void gpu_realtime_reset_accum ()
```

Definition at line 1012 of file gpu_renderer.cu.

7.9.3.10 gpu_realtime_tonemap()

```
int gpu_realtime_tonemap (
            unsigned char * argb_host,
            int width,
            int height)
```

Definition at line 997 of file gpu_renderer.cu.

7.9.3.11 gpu_render_scene()

```
void gpu_render_scene (
            const Scene & scene,
            ImageBuffer & image,
            float & kernel_ms)
```

Definition at line 788 of file gpu_renderer.cu.

7.9.3.12 gpu_render_scene_denoised()

```
void gpu_render_scene_denoised (
            const Scene & scene,
            ImageBuffer & image,
            float & kernel_ms,
            float sigma_color)
```

Definition at line 861 of file gpu_renderer.cu.

7.9.3.13 grng_float()

```
__device__ float grng_float (
            GRng & state)
```

Definition at line 22 of file gpu_renderer.cu.

7.9.3.14 grng_init()

```
__device__ void grng_init (
            GRng & state,
            uint32_t tid,
            uint32_t seed)
```

Definition at line 18 of file gpu_renderer.cu.

7.9.3.15 intersect_plane()

```
__device__ bool intersect_plane (
            const GPlane & pl,
            const GRay & r,
            float t_min,
            float t_max,
            GHitRecord & rec)
```

Definition at line 94 of file gpu_renderer.cu.

7.9.3.16 intersect_sphere()

```
__device__ bool intersect_sphere (
            const GSphere & s,
            const GRay & r,
            float t_min,
            float t_max,
            GHitRecord & rec)
```

Definition at line 47 of file gpu_renderer.cu.

7.9.3.17 intersect_sphere_soa()

```
__device__ bool intersect_sphere_soa (
            const GSphereSoA & soa,
            int idx,
            const GRay & r,
            float t_min,
            float t_max,
            GHitRecord & rec)
```

Definition at line 70 of file gpu_renderer.cu.

7.9.3.18 random_in_unit_disk()

```
__device__ GVec3 random_in_unit_disk (
            GRng & rng)
```

Definition at line 37 of file gpu_renderer.cu.

7.9.3.19 random_in_unit_sphere()

```
__device__ GVec3 random_in_unit_sphere (
            GRng & rng)
```

Definition at line 26 of file gpu_renderer.cu.

7.9.3.20 random_unit_vector()

```
__device__ GVec3 random_unit_vector (
            GRng & rng)
```

Definition at line 33 of file gpu_renderer.cu.

7.9.3.21 ray_color()

```
__device__ GVec3 ray_color (
            GRay r,
            const GBVHNode * bvh,
            int num_bvh,
            const GSphereSoA & soa,
            const GSphere * spheres,
            int num_spheres,
            const GPlane * planes,
            int num_planes,
            const GMaterial * materials,
            int max_depth,
            GRng & rng,
            const GBVHNode * sh_bvh,
            int sh_count)
```

Definition at line 220 of file gpu_renderer.cu.

7.9.3.22 render_kernel()

```
__global__ void render_kernel (
            float *__restrict__ framebuffer,
            int width,
            int height,
            const GBVHNode *__restrict__ bvh,
            int num_bvh,
            const GSphere *__restrict__ spheres,
            int num_spheres,
            const GPlane *__restrict__ planes,
            int num_planes,
            const GMaterial *__restrict__ materials,
            const float *__restrict__ soa_cx,
            const float *__restrict__ soa_cy,
            const float *__restrict__ soa_cz,
            const float *__restrict__ soa_r,
            const int *__restrict__ soa_mat)
```

Definition at line 288 of file gpu_renderer.cu.

7.9.3.23 scatter_mat()

```
__device__ bool scatter_mat (
            const GMaterial & mat,
            const GRay & r_in,
            const GHitRecord & rec,
            GVec3 & attenuation,
            GRay & scattered,
            GRng & rng)
```

Definition at line 177 of file gpu_renderer.cu.

7.9.3.24 tonemap_kernel()

```
__global__ void tonemap_kernel (
            const float *__restrict__ linear_fb,
            unsigned char *__restrict__ argb_out,
            int width,
            int height,
            int total_samples)
```

Definition at line 358 of file gpu_renderer.cu.

7.9.3.25 trace_brute()

```
__device__ bool trace_brute (
            const GSphere * spheres,
            int num_spheres,
            const GPlane * planes,
            int num_planes,
            const GRay & r,
            float t_min,
            float t_max,
            GHitRecord & rec)
```

Definition at line 154 of file gpu_renderer.cu.

7.9.3.26 trace_bvh()

```
__device__ bool trace_bvh (
            const GBVHNode *__restrict__ nodes,
            int num_nodes,
            const GSphereSoA & soa,
            const GPlane *__restrict__ planes,
            const GRay & r,
            float t_min,
            float t_max,
            GHitRecord & rec,
            const GBVHNode *__restrict__ sh_bvh,
            int sh_count)
```

Definition at line 112 of file gpu_renderer.cu.

7.9.4 Variable Documentation

7.9.4.1 d_camera_bytes

```
__constant__ char d_camera_bytes[sizeof(GCamera)]
```

Definition at line 285 of file gpu_renderer.cu.

7.9.4.2 s_renderer

```
GpuRenderer* s_renderer = nullptr
```

Definition at line 786 of file gpu_renderer.cu.

7.9.4.3 s_rt_acc_grid

```
int s_rt_acc_grid = 0  [static]
```

Definition at line 944 of file gpu_renderer.cu.

7.9.4.4 s_rt_accum_count

```
int s_rt_accum_count = 0  [static]
```

Definition at line 914 of file gpu_renderer.cu.

7.9.4.5 s_rt_accum_fb

```
float* s_rt_accum_fb = nullptr  [static]
```

Definition at line 910 of file gpu_renderer.cu.

7.9.4.6 s_rt_argb

```
unsigned char* s_rt_argb = nullptr  [static]
```

Definition at line 911 of file gpu_renderer.cu.

7.9.4.7 s_rt_argb_host

```
unsigned char* s_rt_argb_host = nullptr  [static]
```

Definition at line 912 of file gpu_renderer.cu.

7.9.4.8 s_rt_block

```
dim3 s_rt_block [static]
```

Definition at line 943 of file gpu_renderer.cu.

7.9.4.9 s_rt_config_done

```
bool s_rt_config_done = false [static]
```

Definition at line 946 of file gpu_renderer.cu.

7.9.4.10 s_rt_frame_fb

```
float* s_rt_frame_fb = nullptr [static]
```

Definition at line 909 of file gpu_renderer.cu.

7.9.4.11 s_rt_grid

```
dim3 s_rt_grid [static]
```

Definition at line 943 of file gpu_renderer.cu.

7.9.4.12 s_rt_h

```
int s_rt_h = 0 [static]
```

Definition at line 913 of file gpu_renderer.cu.

7.9.4.13 s_rt_start

```
cudaEvent_t s_rt_start = nullptr [static]
```

Definition at line 945 of file gpu_renderer.cu.

7.9.4.14 s_rt_stop

```
cudaEvent_t s_rt_stop = nullptr [static]
```

Definition at line 945 of file gpu_renderer.cu.

7.9.4.15 s_rt_stream

```
cudaStream_t s_rt_stream = nullptr [static]
```

Definition at line 915 of file gpu_renderer.cu.

7.9.4.16 s_rt_w

```
int s_rt_w = 0  [static]
```

Definition at line 913 of file gpu_renderer.cu.

7.10 gpu_renderer.cu

Go to the documentation of this file.

```
00001 // GPU Renderer : tout le code CUDA dans un seul fichier .cu
00002 // Evite la separable compilation (problemes de linkage RDC sur MSVC)
00003
00004 #include "gpu/gpu_types.cuh"
00005 #include "gpu/gpu_renderer.h"
00006 #include "scene/primitives.cuh"
00007 #include "scene/material.cuh"
00008 #include <cstdio>
00009 #include <cmath>
00010 #include <vector>
00011 #include <curand_kernel.h>
00012
00013 // ============================================================
00014 // GPU RNG : curand (standard NVIDIA)
00015 // ============================================================
00016 typedef curandState GRng;
00017
00018 __device__ void grng_init(GRng& state, uint32_t tid, uint32_t seed) {
00019     curand_init(seed, tid, 0, &state);
00020 }
00021
00022 __device__ float grng_float(GRng& state) {
00023     return curand_uniform(&state);
00024 }
00025
00026 __device__ GVec3 random_in_unit_sphere(GRng& rng) {
00027     while (true) {
00028         GVec3 p(grng_float(rng)*2.0f-1.0f, grng_float(rng)*2.0f-1.0f, grng_float(rng)*2.0f-1.0f);
00029         if (p.length_squared() < 1.0f) return p;
00030     }
00031 }
00032
00033 __device__ GVec3 random_unit_vector(GRng& rng) {
00034     return normalize(random_in_unit_sphere(rng));
00035 }
00036
00037 __device__ GVec3 random_in_unit_disk(GRng& rng) {
00038     while (true) {
00039         GVec3 p(grng_float(rng)*2.0f-1.0f, grng_float(rng)*2.0f-1.0f, 0.0f);
00040         if (p.length_squared() < 1.0f) return p;
00041     }
00042 }
00043
00044 // ============================================================
00045 // Intersections
00046 // ============================================================
00047 __device__ bool intersect_sphere(const GSphere& s, const GRay& r, float t_min, float t_max,
      GHitRecord& rec) {
00048     GVec3 oc = r.orig - s.center;
00049     float a = dot(r.dir, r.dir);
00050     float half_b = dot(oc, r.dir);
00051     float c = oc.length_squared() - s.radius * s.radius;
00052     float disc = half_b * half_b - a * c;
00053     if (disc < 0.0f) return false;
00054     float sqrtd = sqrtf(disc);
00055     float inv_a = __fdividef(1.0f, a); // fast HW reciprocal, avoids full division
00056     float root = (-half_b - sqrtd) * inv_a;
00057     if (root < t_min || root > t_max) {
00058         root = (-half_b + sqrtd) * inv_a;
00059         if (root < t_min || root > t_max) return false;
00060     }
00061     rec.t = root;
00062     rec.p = r.at(root);
00063     float inv_r = __fdividef(1.0f, s.radius);
00064     rec.set_face_normal(r, (rec.p - s.center) * inv_r);
00065     rec.material_id = s.material_id;
00066     return true;
00067 }
00068
```

```
// SoA layout: coalesced loads when warp threads test consecutive spheres
__device__ bool intersect_sphere_soa(const GSphereSoA& soa, int idx,
                                     const GRay& r, float t_min, float t_max, GHitRecord& rec) {
    float sx = soa.cx[idx], sy = soa.cy[idx], sz = soa.cz[idx], sr = soa.r[idx];
    GVec3 oc(r.orig.x - sx, r.orig.y - sy, r.orig.z - sz);
    float a = dot(r.dir, r.dir);
    float half_b = dot(oc, r.dir);
    float c = oc.length_squared() - sr * sr;
    float disc = half_b * half_b - a * c;
    if (disc < 0.0f) return false;
    float sqrtd = sqrtf(disc);
    float inv_a = __fdividef(1.0f, a);
    float root = (-half_b - sqrtd) * inv_a;
    if (root < t_min || root > t_max) {
        root = (-half_b + sqrtd) * inv_a;
        if (root < t_min || root > t_max) return false;
    }
    rec.t = root;
    rec.p = r.at(root);
    float inv_r = __fdividef(1.0f, sr);
    rec.set_face_normal(r, GVec3(rec.p.x - sx, rec.p.y - sy, rec.p.z - sz) * inv_r);
    rec.material_id = soa.mat[idx];
    return true;
}

__device__ bool intersect_plane(const GPlane& pl, const GRay& r, float t_min, float t_max, GHitRecord& rec) {
    float denom = dot(r.dir, pl.normal);
    if (fabsf(denom) < 1e-6f) return false;
    float t = dot(pl.point - r.orig, pl.normal) * __fdividef(1.0f, denom);
    if (t < t_min || t > t_max) return false;
    rec.t = t;
    rec.p = r.at(t);
    rec.set_face_normal(r, pl.normal);
    rec.material_id = pl.material_id;
    return true;
}

// ============================================================
// BVH traversal iteratif (stack fixe, front-to-back)
// ============================================================
// Top BVH nodes in shared mem: accessed by nearly all rays in the block
#define BVH_SHARED_NODES 7  // 3 premiers niveaux = 1+2+4 = 7 nodes

__device__ bool trace_bvh(const GBVHNode* __restrict__ nodes, int num_nodes,
                          const GSphereSoA& soa,
                          const GPlane* __restrict__ planes,
                          const GRay& r, float t_min, float t_max,
                          GHitRecord& rec,
                          const GBVHNode* __restrict__ sh_bvh, int sh_count) {
    bool hit_anything = false;
    float closest = t_max;
    int stack[16];
    int sp = 0;
    stack[sp++] = 0;

    while (sp > 0) {
        int idx = stack[--sp];
        const GBVHNode& node = (idx < sh_count) ? sh_bvh[idx] : nodes[idx];

        if (!node.bbox.hit(r, t_min, closest))
            continue;

        if (node.num_prims > 0) {
            #pragma unroll 4
            for (uint16_t i = 0; i < node.num_prims; i++) {
                GHitRecord tmp;
                // SoA: each float[] is contiguous -> coalesced reads
                if (intersect_sphere_soa(soa, node.data + i, r, t_min, closest, tmp)) {
                    hit_anything = true; closest = tmp.t; rec = tmp;
                }
            }
        } else {
            int near_child = idx + 1;
            int far_child = node.data;
            float dir_comp = (node.split_axis == 0) ? r.dir.x :
                             (node.split_axis == 1) ? r.dir.y : r.dir.z;
            if (dir_comp < 0.0f) { int tmp = near_child; near_child = far_child; far_child = tmp; }
            stack[sp++] = far_child;
            stack[sp++] = near_child;
        }
    }
    return hit_anything;
}

// Brute force fallback
__device__ bool trace_brute(const GSphere* spheres, int num_spheres,
```

```
                            const GPlane* planes, int num_planes,
                            const GRay& r, float t_min, float t_max,
                            GHitRecord& rec) {
    bool hit = false;
    float closest = t_max;
    GHitRecord tmp;
    for (int i = 0; i < num_spheres; i++) {
        if (intersect_sphere(spheres[i], r, t_min, closest, tmp)) {
            hit = true; closest = tmp.t; rec = tmp;
        }
    }
    for (int i = 0; i < num_planes; i++) {
        if (intersect_plane(planes[i], r, t_min, closest, tmp)) {
            hit = true; closest = tmp.t; rec = tmp;
        }
    }
    return hit;
}

// ============================================================
// Material scatter
// ============================================================
__device__ bool scatter_mat(const GMaterial& mat, const GRay& r_in, const GHitRecord& rec,
                            GVec3& attenuation, GRay& scattered, GRng& rng) {
    switch (mat.type) {
    case GMAT_LAMBERTIAN: {
        GVec3 dir = rec.normal + random_unit_vector(rng);
        if (dir.near_zero()) dir = rec.normal;
        scattered = GRay(rec.p, dir);
        attenuation = mat.albedo;
        return true;
    }
    case GMAT_METAL: {
        GVec3 reflected = reflect(normalize(r_in.dir), rec.normal);
        reflected = reflected + mat.param * random_in_unit_sphere(rng);
        scattered = GRay(rec.p, reflected);
        attenuation = mat.albedo;
        return dot(scattered.dir, rec.normal) > 0.0f;
    }
    case GMAT_DIELECTRIC: {
        attenuation = mat.albedo; // tinted glass support
        float ri = rec.front_face ? (1.0f / mat.param) : mat.param;
        GVec3 unit_dir = normalize(r_in.dir);
        float cos_theta = fminf(dot(-unit_dir, rec.normal), 1.0f);
        float sin_theta = sqrtf(1.0f - cos_theta * cos_theta);
        float r0 = (1.0f - ri) / (1.0f + ri); r0 = r0 * r0;
        // manual pow5: avoids expensive powf (exp+log chain)
        float omc = 1.0f - cos_theta;
        float omc2 = omc * omc;
        float schlick = r0 + (1.0f - r0) * omc2 * omc2 * omc;
        GVec3 direction;
        if (ri * sin_theta > 1.0f || schlick > grng_float(rng))
            direction = reflect(unit_dir, rec.normal);
        else
            direction = refract(unit_dir, rec.normal, ri);
        scattered = GRay(rec.p, direction);
        return true;
    }
    }
    return false;
}

// ============================================================
// Ray color iteratif
// ============================================================
__device__ GVec3 ray_color(GRay r,
                           const GBVHNode* bvh, int num_bvh,
                           const GSphereSoA& soa,
                           const GSphere* spheres, int num_spheres,
                           const GPlane* planes, int num_planes,
                           const GMaterial* materials,
                           int max_depth, GRng& rng,
                           const GBVHNode* sh_bvh, int sh_count) {
    GVec3 throughput(1.0f, 1.0f, 1.0f);

    for (int d = 0; d < max_depth; d++) {
        GHitRecord rec;
        bool hit;
        if (num_bvh > 0) {
            // BVH pour les spheres + brute force pour les plans (peu nombreux)
            float closest = 1e30f;
            hit = trace_bvh(bvh, num_bvh, soa, planes, r, 0.001f, closest, rec, sh_bvh, sh_count);
            if (hit) closest = rec.t;
            // Tester les plans en brute force
            GHitRecord tmp;
            for (int i = 0; i < num_planes; i++) {
                if (intersect_plane(planes[i], r, 0.001f, closest, tmp)) {
```

```
                    hit = true; closest = tmp.t; rec = tmp;
                }
            }
        } else {
            hit = trace_brute(spheres, num_spheres, planes, num_planes, r, 0.001f, 1e30f, rec);
        }
        if (!hit) {
            GVec3 unit_dir = normalize(r.dir);
            float a = 0.5f * (unit_dir.y + 1.0f);
            GVec3 sky(1.0f - 0.5f*a, 1.0f - 0.3f*a, 1.0f);
            return throughput * sky;
        }
        const GMaterial& mat = materials[rec.material_id];
        if (mat.type == GMAT_EMISSIVE)
            return throughput * mat.albedo * mat.param;

        GVec3 attenuation;
        GRay scattered;
        if (!scatter_mat(mat, r, rec, attenuation, scattered, rng))
            return GVec3(0, 0, 0);
        throughput = throughput * attenuation;

        // Russian roulette: unbiased path termination after depth 3
        float max_comp = fmaxf(throughput.x, fmaxf(throughput.y, throughput.z));
        if (max_comp < 0.001f)
            break;

        if (d > 3) {
            if (grng_float(rng) > max_comp)
                break;
            throughput = throughput * (1.0f / max_comp);
        }

        r = scattered;
    }
    return GVec3(0, 0, 0);
}

// ============================================================
// Kernel : 1 thread = 1 pixel, blocs 16x16
// ============================================================
// Camera in constant memory: broadcast to all threads, dedicated cache
// Stored as raw bytes to avoid dynamic initialization issues
__constant__ char d_camera_bytes[sizeof(GCamera)];
#define d_camera (*(const GCamera*)d_camera_bytes)

__global__ void render_kernel(float* __restrict__ framebuffer, int width, int height,
                              const GBVHNode* __restrict__ bvh, int num_bvh,
                              const GSphere* __restrict__ spheres, int num_spheres,
                              const GPlane* __restrict__ planes, int num_planes,
                              const GMaterial* __restrict__ materials,
                              const float* __restrict__ soa_cx, const float* __restrict__ soa_cy,
                              const float* __restrict__ soa_cz, const float* __restrict__ soa_r,
                              const int* __restrict__ soa_mat) {
    // Cooperative load of top BVH nodes into shared mem
    __shared__ GBVHNode sh_bvh[BVH_SHARED_NODES];
    int tid_flat = threadIdx.y * blockDim.x + threadIdx.x;
    if (tid_flat < BVH_SHARED_NODES && tid_flat < num_bvh) {
        sh_bvh[tid_flat] = bvh[tid_flat];
    }
    __syncthreads(); // Attendre que tous les top nodes soient en shared mem

    int sh_count = (num_bvh < BVH_SHARED_NODES) ? num_bvh : BVH_SHARED_NODES;

    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) return;

    GRng rng;
    grng_init(rng, y * width + x, 42);

    GVec3 pixel_color(0, 0, 0);
    int spp = d_camera.samples_per_pixel;
    float inv_spp = 1.0f / (float)spp;

    for (int s = 0; s < spp; s++) {
        float u_off = grng_float(rng) - 0.5f;
        float v_off = grng_float(rng) - 0.5f;
        GVec3 pixel_sample = d_camera.pixel00_loc
                           + ((float)x + u_off) * d_camera.pixel_delta_u
                           + ((float)y + v_off) * d_camera.pixel_delta_v;
        GVec3 ray_origin = d_camera.origin;
        if (d_camera.defocus_angle > 0.0f) {
            GVec3 pd = random_in_unit_disk(rng);
            ray_origin = d_camera.origin + pd.x * d_camera.defocus_disk_u + pd.y *
d_camera.defocus_disk_v;
        }
```

```
        GRay r(ray_origin, pixel_sample - ray_origin);
        GSphereSoA soa;
        soa.cx = const_cast<float*>(soa_cx); soa.cy = const_cast<float*>(soa_cy);
        soa.cz = const_cast<float*>(soa_cz); soa.r = const_cast<float*>(soa_r);
        soa.mat = const_cast<int*>(soa_mat); soa.count = num_spheres;
        pixel_color += ray_color(r, bvh, num_bvh, soa, spheres, num_spheres, planes, num_planes,
                                 materials, d_camera.max_depth, rng, sh_bvh, sh_count);
    }

    pixel_color *= inv_spp;

    // Planar layout (RRR...GGG...BBB): stride-1 writes for coalescing
    int pixel_idx = y * width + x;
    int total_pixels = width * height;
    // Linear HDR output, gamma/tonemap done later
    framebuffer[pixel_idx]                  = fmaxf(pixel_color.x, 0.0f);
    framebuffer[pixel_idx + total_pixels]   = fmaxf(pixel_color.y, 0.0f);
    framebuffer[pixel_idx + total_pixels*2] = fmaxf(pixel_color.z, 0.0f);
}

// ============================================================
// Tonemap GPU kernel : lineaire -> ACES -> gamma -> ARGB 8-bit
// Evite le transfert float D->H + tonemap CPU
// ============================================================
__device__ float aces_tonemap_gpu(float x) {
    float a = x * (2.51f * x + 0.03f);
    float b = x * (2.43f * x + 0.59f) + 0.14f;
    return fmaxf(0.0f, fminf(1.0f, a / b));
}

__global__ void tonemap_kernel(const float* __restrict__ linear_fb,
                               unsigned char* __restrict__ argb_out,
                               int width, int height, int total_samples) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) return;

    int px = y * width + x;
    int tp = width * height;
    float inv_n = 1.0f / (float)total_samples;

    float r = sqrtf(aces_tonemap_gpu(fmaxf(linear_fb[px] * inv_n, 0.0f)));
    float g = sqrtf(aces_tonemap_gpu(fmaxf(linear_fb[px + tp] * inv_n, 0.0f)));
    float b = sqrtf(aces_tonemap_gpu(fmaxf(linear_fb[px + tp*2] * inv_n, 0.0f)));

    int idx = px * 4;
    argb_out[idx + 0] = (unsigned char)(fminf(b, 1.0f) * 255.0f);
    argb_out[idx + 1] = (unsigned char)(fminf(g, 1.0f) * 255.0f);
    argb_out[idx + 2] = (unsigned char)(fminf(r, 1.0f) * 255.0f);
    argb_out[idx + 3] = 255;
}

// ============================================================
// Accumulation kernel : ajoute le rendu au buffer d'accumulation
// ============================================================
__global__ void accumulate_kernel(const float* __restrict__ frame_fb,
                                  float* __restrict__ accum_fb,
                                  int total_pixels) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= total_pixels * 3) return;
    accum_fb[idx] += frame_fb[idx];
}

// ============================================================
// GPU Denoise kernel : filtre bilateral 3x3 sur le framebuffer lineaire
// Lisse le bruit en preservant les contours (difference de couleur)
// ============================================================
__global__ void denoise_bilateral_kernel(
    const float* __restrict__ input,
    float* __restrict__ output,
    int width, int height, float sigma_color)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) return;

    int tp = width * height;
    int px = y * width + x;
    float cr = input[px], cg = input[px+tp], cb = input[px+tp*2];

    float sum_r = 0, sum_g = 0, sum_b = 0, wsum = 0;
    float inv_sigma2 = 1.0f / (2.0f * sigma_color * sigma_color);

    for (int dy = -1; dy <= 1; dy++) {
        int ny = y + dy;
        if (ny < 0 || ny >= height) continue;
        for (int dx = -1; dx <= 1; dx++) {
```

```
            int nx = x + dx;
            if (nx < 0 || nx >= width) continue;
            int npx = ny * width + nx;
            float nr = input[npx], ng = input[npx+tp], nb = input[npx+tp*2];
            float dr = cr-nr, dg = cg-ng, db = cb-nb;
            float color_dist = dr*dr + dg*dg + db*db;
            float w = expf(-color_dist * inv_sigma2);
            sum_r += nr * w; sum_g += ng * w; sum_b += nb * w;
            wsum += w;
        }
    }
    float inv_w = 1.0f / (wsum + 1e-8f);
    output[px] = sum_r * inv_w;
    output[px+tp] = sum_g * inv_w;
    output[px+tp*2] = sum_b * inv_w;
}

// ============================================================
// GpuRenderer host code
// ============================================================
#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        return; \
    } \
} while(0)

GpuRenderer::GpuRenderer() {}
GpuRenderer::~GpuRenderer() { free_scene(); free_framebuffer(); }

void GpuRenderer::free_scene() {
    if (d_spheres_) { cudaFree(d_spheres_); d_spheres_ = nullptr; }
    if (d_soa_cx_) { cudaFree(d_soa_cx_); d_soa_cx_ = nullptr; }
    if (d_soa_cy_) { cudaFree(d_soa_cy_); d_soa_cy_ = nullptr; }
    if (d_soa_cz_) { cudaFree(d_soa_cz_); d_soa_cz_ = nullptr; }
    if (d_soa_r_) { cudaFree(d_soa_r_); d_soa_r_ = nullptr; }
    if (d_soa_mat_) { cudaFree(d_soa_mat_); d_soa_mat_ = nullptr; }
    if (d_planes_) { cudaFree(d_planes_); d_planes_ = nullptr; }
    if (d_materials_) { cudaFree(d_materials_); d_materials_ = nullptr; }
    if (d_bvh_nodes_) { cudaFree(d_bvh_nodes_); d_bvh_nodes_ = nullptr; }
}

void GpuRenderer::free_framebuffer() {
    if (d_framebuffer_) { cudaFree(d_framebuffer_); d_framebuffer_ = nullptr; }
    fb_width_ = fb_height_ = 0;
}

void GpuRenderer::ensure_framebuffer(int width, int height) {
    if (fb_width_ == width && fb_height_ == height && d_framebuffer_) return;
    free_framebuffer();
    fb_width_ = width; fb_height_ = height;
    cudaMalloc(&d_framebuffer_, width * height * 3 * sizeof(float));
}

void GpuRenderer::upload_scene(const GSphere* spheres, int num_spheres,
                               const GPlane* planes, int num_planes,
                               const GMaterial* materials, int num_materials,
                               const GBVHNode* bvh_nodes, int num_bvh_nodes) {
    free_scene();
    num_spheres_ = num_spheres;
    num_planes_ = num_planes;
    num_materials_ = num_materials;
    num_bvh_nodes_ = num_bvh_nodes;

    if (num_spheres > 0) {
        CUDA_CHECK(cudaMalloc(&d_spheres_, num_spheres * sizeof(GSphere)));
        CUDA_CHECK(cudaMemcpy(d_spheres_, spheres, num_spheres * sizeof(GSphere),
cudaMemcpyHostToDevice));

        // SoA upload: split fields for coalesced access in kernel
        std::vector<float> cx(num_spheres), cy(num_spheres), cz(num_spheres), radii(num_spheres);
        std::vector<int> mats(num_spheres);
        for (int i = 0; i < num_spheres; i++) {
            cx[i] = spheres[i].center.x; cy[i] = spheres[i].center.y; cz[i] = spheres[i].center.z;
            radii[i] = spheres[i].radius; mats[i] = spheres[i].material_id;
        }
        CUDA_CHECK(cudaMalloc(&d_soa_cx_, num_spheres * sizeof(float)));
        CUDA_CHECK(cudaMalloc(&d_soa_cy_, num_spheres * sizeof(float)));
        CUDA_CHECK(cudaMalloc(&d_soa_cz_, num_spheres * sizeof(float)));
        CUDA_CHECK(cudaMalloc(&d_soa_r_, num_spheres * sizeof(float)));
        CUDA_CHECK(cudaMalloc(&d_soa_mat_, num_spheres * sizeof(int)));
        CUDA_CHECK(cudaMemcpy(d_soa_cx_, cx.data(), num_spheres * sizeof(float),
cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(d_soa_cy_, cy.data(), num_spheres * sizeof(float),
cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(d_soa_cz_, cz.data(), num_spheres * sizeof(float),
```

```
cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(d_soa_r_, radii.data(), num_spheres * sizeof(float), cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(d_soa_mat_, mats.data(), num_spheres * sizeof(int), cudaMemcpyHostToDevice));
    }
    if (num_planes > 0) {
        CUDA_CHECK(cudaMalloc(&d_planes_, num_planes * sizeof(GPlane)));
        CUDA_CHECK(cudaMemcpy(d_planes_, planes, num_planes * sizeof(GPlane), cudaMemcpyHostToDevice));
    }
    if (num_materials > 0) {
        CUDA_CHECK(cudaMalloc(&d_materials_, num_materials * sizeof(GMaterial)));
        CUDA_CHECK(cudaMemcpy(d_materials_, materials, num_materials * sizeof(GMaterial), cudaMemcpyHostToDevice));
    }
    if (num_bvh_nodes > 0) {
        CUDA_CHECK(cudaMalloc(&d_bvh_nodes_, num_bvh_nodes * sizeof(GBVHNode)));
        CUDA_CHECK(cudaMemcpy(d_bvh_nodes_, bvh_nodes, num_bvh_nodes * sizeof(GBVHNode), cudaMemcpyHostToDevice));

        // L2 cache persistence pour BVH (Ampere CC 8.6+)
        // Les top nodes sont accedes par quasi tous les rayons
        size_t bvh_bytes = num_bvh_nodes * sizeof(GBVHNode);
        cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, bvh_bytes);
    }
}

void GpuRenderer::render(const GCamera& camera, float* h_output, int width, int height) {
    ensure_framebuffer(width, height);

    // Persistent stream for kernel/transfer overlap
    static cudaStream_t s_stream = nullptr;
    if (!s_stream) {
        cudaStreamCreate(&s_stream);

        // L2 cache persistence : BVH nodes en persistent L2 (Ampere CC 8.6+)
        if (d_bvh_nodes_ && num_bvh_nodes_ > 0) {
            cudaStreamAttrValue attr = {};
            attr.accessPolicyWindow.base_ptr = d_bvh_nodes_;
            attr.accessPolicyWindow.num_bytes = num_bvh_nodes_ * sizeof(GBVHNode);
            attr.accessPolicyWindow.hitRatio = 1.0f;
            attr.accessPolicyWindow.hitProp = cudaAccessPropertyPersisting;
            attr.accessPolicyWindow.missProp = cudaAccessPropertyStreaming;
            cudaStreamSetAttribute(s_stream, cudaStreamAttributeAccessPolicyWindow, &attr);
        }
    }

    // Upload camera en constant memory
    cudaMemcpyToSymbol(d_camera_bytes, &camera, sizeof(GCamera));

    // Block size via occupancy API
    int flat_block_size = 0, min_grid = 0;
    cudaOccupancyMaxPotentialBlockSize(&min_grid, &flat_block_size, render_kernel, 0, 0);
    int bx = 16, by = flat_block_size / bx;
    if (by < 1) by = 1;
    if (by > 16) by = 16;
    dim3 block_size(bx, by);
    dim3 grid_size((width + block_size.x - 1) / block_size.x,
                   (height + block_size.y - 1) / block_size.y);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start, s_stream);

    render_kernel<<grid_size, block_size, 0, s_stream>>(
        d_framebuffer_, width, height,
        d_bvh_nodes_, num_bvh_nodes_,
        d_spheres_, num_spheres_,
        d_planes_, num_planes_,
        d_materials_,
        d_soa_cx_, d_soa_cy_, d_soa_cz_, d_soa_r_, d_soa_mat_
    );

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        fprintf(stderr, "Kernel error: %s\n", cudaGetErrorString(err));
        cudaEventDestroy(start); cudaEventDestroy(stop);
        return;
    }

    // Transfert async D->H sur le meme stream (sequentialise apres le kernel)
    // h_output DOIT etre en memoire pinned (cudaMallocHost) pour l'async
    cudaMemcpyAsync(h_output, d_framebuffer_, width * height * 3 * sizeof(float),
                    cudaMemcpyDeviceToHost, s_stream);
```

```

    cudaEventRecord(stop, s_stream);
    // Synchroniser le stream (attendre kernel + transfert)
    cudaStreamSynchronize(s_stream);

    cudaEventElapsedTime(&last_ms_, start, stop);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
}

// ============================================================
// Bridge: conversion CPU Scene -> GPU + rendu
// ============================================================
static void convert_scene(const Scene& scene,
                          std::vector<GSphere>& out_spheres,
                          std::vector<GPlane>& out_planes,
                          std::vector<GMaterial>& out_materials,
                          GCamera& out_camera) {
    std::vector<const Material*> mat_ptrs;
    auto get_mat = [&](const Material* m) -> int {
        for (int i = 0; i < (int)mat_ptrs.size(); i++)
            if (mat_ptrs[i] == m) return i;
        int idx = (int)mat_ptrs.size();
        mat_ptrs.push_back(m);
        GMaterial gm;
        switch (m->type()) {
        case Material::Type::Lambertian: {
            auto* lam = static_cast<const Lambertian*>(m);
            gm.type = GMAT_LAMBERTIAN;
            gm.albedo = GVec3((float)lam->albedo().x(), (float)lam->albedo().y(), (float)lam->albedo().z());
            break;
        }
        case Material::Type::Metal: {
            auto* met = static_cast<const Metal*>(m);
            gm.type = GMAT_METAL;
            gm.albedo = GVec3((float)met->albedo().x(), (float)met->albedo().y(), (float)met->albedo().z());
            gm.param = (float)met->fuzz();
            break;
        }
        case Material::Type::Dielectric: {
            auto* die = static_cast<const Dielectric*>(m);
            gm.type = GMAT_DIELECTRIC;
            gm.albedo = GVec3(1, 1, 1);
            gm.param = (float)die->refraction_index();
            break;
        }
        case Material::Type::Emissive: {
            auto* em = static_cast<const Emissive*>(m);
            gm.type = GMAT_EMISSIVE;
            gm.albedo = GVec3((float)em->emit_color().x(), (float)em->emit_color().y(), (float)em->emit_color().z());
            gm.param = (float)em->intensity();
            break;
        }
        default: gm.type = GMAT_LAMBERTIAN; gm.albedo = GVec3(0.5f, 0.5f, 0.5f); break;
        }
        out_materials.push_back(gm);
        return idx;
    };

    for (const auto& obj : scene.world.objects()) {
        if (auto* sp = dynamic_cast<const Sphere*>(obj.get())) {
            GSphere gs;
            gs.center = GVec3((float)sp->center().x(), (float)sp->center().y(), (float)sp->center().z());
            gs.radius = (float)sp->radius();
            HitRecord rec;
            Ray test_ray(sp->center() + Vec3(0, sp->radius() + 1, 0), Vec3(0, -1, 0));
            gs.material_id = (sp->hit(test_ray, Interval(0.001, 1000.0), rec) && rec.material)
                             ? get_mat(rec.material) : 0;
            out_spheres.push_back(gs);
        } else if (auto* pl = dynamic_cast<const Plane*>(obj.get())) {
            GPlane gp;
            HitRecord rec;
            Ray test_ray(Point3(0, 100, 0), Vec3(0, -1, 0));
            if (pl->hit(test_ray, Interval(0.001, 10000.0), rec) && rec.material) {
                gp.point = GVec3((float)rec.p.x(), (float)rec.p.y(), (float)rec.p.z());
                gp.normal = GVec3((float)rec.normal.x(), (float)rec.normal.y(), (float)rec.normal.z());
                gp.material_id = get_mat(rec.material);
            }
            out_planes.push_back(gp);
        } else {
            static bool warned = false;
            if (!warned) {
```

```
                fprintf(stderr, "Warning: GPU ne supporte que Sphere/Plane. Triangles/Cylindres ignores.\n");
                fprintf(stderr, "         Utilisez --scene showcase, final, ou three_spheres pour le GPU.\n");
                warned = true;
            }
        }
    }

    const Camera& cam = scene.camera;
    int w = cam.get_image_width(), h = cam.get_image_height();
    double theta = cam.vfov * 3.14159265358979323846 / 180.0;
    double vp_h = 2.0 * std::tan(theta / 2.0) * cam.focus_dist;
    double vp_w = vp_h * ((double)w / h);
    Vec3 wd = unit_vector(cam.lookfrom - cam.lookat);
    Vec3 ud = unit_vector(cross(cam.vup, wd));
    Vec3 vd = cross(wd, ud);
    Vec3 vu = vp_w * ud, vv = vp_h * (-vd);
    Vec3 pdu = vu / (double)w, pdv = vv / (double)h;
    Point3 p00 = cam.lookfrom - cam.focus_dist * wd - vu/2.0 - vv/2.0 + 0.5*(pdu+pdv);
    double dr = cam.focus_dist * std::tan((cam.defocus_angle/2.0)*3.14159265358979323846/180.0);

    out_camera.origin = GVec3((float)cam.lookfrom.x(), (float)cam.lookfrom.y(), (float)cam.lookfrom.z());
    out_camera.pixel00_loc = GVec3((float)p00.x(), (float)p00.y(), (float)p00.z());
    out_camera.pixel_delta_u = GVec3((float)pdu.x(), (float)pdu.y(), (float)pdu.z());
    out_camera.pixel_delta_v = GVec3((float)pdv.x(), (float)pdv.y(), (float)pdv.z());
    out_camera.defocus_angle = (float)cam.defocus_angle;
    out_camera.defocus_disk_u = GVec3((float)(ud.x()*dr), (float)(ud.y()*dr), (float)(ud.z()*dr));
    out_camera.defocus_disk_v = GVec3((float)(vd.x()*dr), (float)(vd.y()*dr), (float)(vd.z()*dr));
    out_camera.image_width = w; out_camera.image_height = h;
    out_camera.samples_per_pixel = cam.samples_per_pixel;
    out_camera.max_depth = cam.max_depth;
}

// Camera-only conversion (pas de re-conversion des objets/materiaux)
static void convert_camera_only(const Camera& cam, GCamera& out) {
    int w = cam.get_image_width(), h = cam.get_image_height();
    double theta = cam.vfov * 3.14159265358979323846 / 180.0;
    double vp_h = 2.0 * std::tan(theta / 2.0) * cam.focus_dist;
    double vp_w = vp_h * ((double)w / h);
    Vec3 wd = unit_vector(cam.lookfrom - cam.lookat);
    Vec3 ud = unit_vector(cross(cam.vup, wd));
    Vec3 vd = cross(wd, ud);
    Vec3 vu = vp_w * ud, vv = vp_h * (-vd);
    Vec3 pdu = vu / (double)w, pdv = vv / (double)h;
    Point3 p00 = cam.lookfrom - cam.focus_dist * wd - vu/2.0 - vv/2.0 + 0.5*(pdu+pdv);
    double dr = cam.focus_dist * std::tan((cam.defocus_angle/2.0)*3.14159265358979323846/180.0);

    out.origin = GVec3((float)cam.lookfrom.x(), (float)cam.lookfrom.y(), (float)cam.lookfrom.z());
    out.pixel00_loc = GVec3((float)p00.x(), (float)p00.y(), (float)p00.z());
    out.pixel_delta_u = GVec3((float)pdu.x(), (float)pdu.y(), (float)pdu.z());
    out.pixel_delta_v = GVec3((float)pdv.x(), (float)pdv.y(), (float)pdv.z());
    out.defocus_angle = (float)cam.defocus_angle;
    out.defocus_disk_u = GVec3((float)(ud.x()*dr), (float)(ud.y()*dr), (float)(ud.z()*dr));
    out.defocus_disk_v = GVec3((float)(vd.x()*dr), (float)(vd.y()*dr), (float)(vd.z()*dr));
    out.image_width = w; out.image_height = h;
    out.samples_per_pixel = cam.samples_per_pixel;
    out.max_depth = cam.max_depth;
}

// ============================================================
// Simple GPU BVH builder (on spheres only, planes are tested brute-force in leaves)
// ============================================================
struct GBuildInfo { int sphere_idx; GAABB bbox; GVec3 centroid; };

static void build_gpu_bvh(std::vector<GBuildInfo>& info, int start, int end,
                          std::vector<GBVHNode>& nodes, int num_planes) {
    int my_idx = (int)nodes.size();
    nodes.push_back(GBVHNode());

    GAABB bounds;
    for (int i = start; i < end; i++) {
        bounds.mn.x = fminf(bounds.mn.x, info[i].bbox.mn.x);
        bounds.mn.y = fminf(bounds.mn.y, info[i].bbox.mn.y);
        bounds.mn.z = fminf(bounds.mn.z, info[i].bbox.mn.z);
        bounds.mx.x = fmaxf(bounds.mx.x, info[i].bbox.mx.x);
        bounds.mx.y = fmaxf(bounds.mx.y, info[i].bbox.mx.y);
        bounds.mx.z = fmaxf(bounds.mx.z, info[i].bbox.mx.z);
    }
    nodes[my_idx].bbox = bounds;

    int n = end - start;
    if (n <= 4) {
        // Leaf : stores sphere indices [start, end)
        nodes[my_idx].data = (uint32_t)start;
        nodes[my_idx].num_prims = (uint16_t)n;
```

```
        nodes[my_idx].split_axis = 0;
        nodes[my_idx].prim_type = 0; // spheres
        return;
    }

    // Find longest axis of centroid bounds
    GVec3 cmin(1e30f,1e30f,1e30f), cmax(-1e30f,-1e30f,-1e30f);
    for (int i = start; i < end; i++) {
        cmin.x = fminf(cmin.x, info[i].centroid.x);
        cmin.y = fminf(cmin.y, info[i].centroid.y);
        cmin.z = fminf(cmin.z, info[i].centroid.z);
        cmax.x = fmaxf(cmax.x, info[i].centroid.x);
        cmax.y = fmaxf(cmax.y, info[i].centroid.y);
        cmax.z = fmaxf(cmax.z, info[i].centroid.z);
    }
    float dx = cmax.x-cmin.x, dy = cmax.y-cmin.y, dz = cmax.z-cmin.z;
    int axis = (dx > dy && dx > dz) ? 0 : (dy > dz) ? 1 : 2;

    // Sort by centroid on axis
    std::sort(info.begin()+start, info.begin()+end,
        [axis](const GBuildInfo& a, const GBuildInfo& b) {
            float va = (axis==0)?a.centroid.x:(axis==1)?a.centroid.y:a.centroid.z;
            float vb = (axis==0)?b.centroid.x:(axis==1)?b.centroid.y:b.centroid.z;
            return va < vb;
        });

    int mid = start + n / 2;

    nodes[my_idx].split_axis = (uint8_t)axis;
    nodes[my_idx].num_prims = 0; // interior
    nodes[my_idx].prim_type = 0;

    // Left child (immediately after)
    build_gpu_bvh(info, start, mid, nodes, num_planes);
    // Right child
    nodes[my_idx].data = (uint32_t)nodes.size();
    build_gpu_bvh(info, mid, end, nodes, num_planes);
}

// Renderer global persistant
GpuRenderer* s_renderer = nullptr;

void gpu_render_scene(const Scene& scene, ImageBuffer& image, float& kernel_ms) {
    // Tout est cache entre les appels : scene, BVH, renderer, framebuffer pinned
    // Renderer global pour acces depuis gpu_realtime_render_frame
    static const Scene* s_last_scene = nullptr;
    static float* s_fb = nullptr;
    static int s_fb_w = 0, s_fb_h = 0;

    GCamera gcam;

    // Re-upload scene seulement si elle a change
    if (!s_renderer || s_last_scene != &scene) {
        std::vector<GSphere> spheres;
        std::vector<GPlane> planes;
        std::vector<GMaterial> materials;
        convert_scene(scene, spheres, planes, materials, gcam);

        std::vector<GBVHNode> bvh_nodes;
        if (spheres.size() > 8) {
            std::vector<GBuildInfo> info(spheres.size());
            for (int i = 0; i < (int)spheres.size(); i++) {
                info[i].sphere_idx = i;
                float r = spheres[i].radius;
                GVec3 c = spheres[i].center;
                info[i].bbox = GAABB(GVec3(c.x-r, c.y-r, c.z-r), GVec3(c.x+r, c.y+r, c.z+r));
                info[i].centroid = c;
            }
            build_gpu_bvh(info, 0, (int)info.size(), bvh_nodes, (int)planes.size());
            std::vector<GSphere> ordered(spheres.size());
            for (int i = 0; i < (int)info.size(); i++)
                ordered[i] = spheres[info[i].sphere_idx];
            spheres = std::move(ordered);
            fprintf(stderr, "GPU BVH: %d nodes for %d spheres\n", (int)bvh_nodes.size(),
(int)spheres.size());
        }

        delete s_renderer;
        s_renderer = new GpuRenderer();
        s_renderer->upload_scene(spheres.data(), (int)spheres.size(),
                                 planes.data(), (int)planes.size(),
                                 materials.data(), (int)materials.size(),
                                 bvh_nodes.empty() ? nullptr : bvh_nodes.data(),
                                 (int)bvh_nodes.size());
        s_last_scene = &scene;
    } else {
        // Scene deja uploadee : recalculer seulement la camera
```

```
        convert_camera_only(scene.camera, gcam);
    }

    int w = gcam.image_width, h = gcam.image_height;

    // Framebuffer pinned persistant
    if (!s_fb || s_fb_w != w || s_fb_h != h) {
        if (s_fb) cudaFreeHost(s_fb);
        cudaMallocHost(&s_fb, w * h * 3 * sizeof(float));
        s_fb_w = w; s_fb_h = h;
    }

    s_renderer->render(gcam, s_fb, w, h);
    kernel_ms = s_renderer->last_render_ms();

    // Copie rapide vers ImageBuffer
    int total_px = w * h;
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++) {
            int px = y * w + x;
            image.set_pixel(x, y, Color(
                fminf(s_fb[px], 1.0f),
                fminf(s_fb[px + total_px], 1.0f),
                fminf(s_fb[px + total_px*2], 1.0f)));
        }
}

// ============================================================
// GPU render + bilateral denoise
void gpu_render_scene_denoised(const Scene& scene, ImageBuffer& image,
                               float& kernel_ms, float sigma_color) {
    // Render normalement (remplit s_renderer->d_framebuffer_)
    gpu_render_scene(scene, image, kernel_ms);

    // Le framebuffer device est dans s_renderer->d_framebuffer_
    if (!s_renderer) return;
    int w = image.width(), h = image.height();
    size_t fb_bytes = w * h * 3 * sizeof(float);

    // Buffer temporaire pour le resultat denoise
    static float* d_denoise_buf = nullptr;
    static int d_denoise_size = 0;
    if (!d_denoise_buf || d_denoise_size != w * h) {
        if (d_denoise_buf) cudaFree(d_denoise_buf);
        cudaMalloc(&d_denoise_buf, fb_bytes);
        d_denoise_size = w * h;
    }

    // Denoise sur le d_framebuffer_ du renderer (deja sur GPU)
    dim3 block(16, 16);
    dim3 grid((w+15)/16, (h+15)/16);
    denoise_bilateral_kernel<<grid, block>>(
        s_renderer->d_framebuffer_, d_denoise_buf, w, h, sigma_color);
    cudaDeviceSynchronize();

    // Copier le resultat denoise vers host (pinned)
    float* h_buf = nullptr;
    cudaMallocHost(&h_buf, fb_bytes);
    cudaMemcpy(h_buf, d_denoise_buf, fb_bytes, cudaMemcpyDeviceToHost);

    // Ecrire dans l'ImageBuffer
    int tp = w * h;
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++) {
            int px = y * w + x;
            image.set_pixel(x, y, Color(
                fminf(h_buf[px], 1.0f),
                fminf(h_buf[px+tp], 1.0f),
                fminf(h_buf[px+tp*2], 1.0f)));
        }
    cudaFreeHost(h_buf);
}

// ============================================================
// GPU-side realtime progressive rendering
// Tout reste sur le GPU : render -> accumulate -> tonemap -> transfer ARGB
// ============================================================
static float* s_rt_frame_fb = nullptr;    // framebuffer du frame courant (lineaire)
static float* s_rt_accum_fb = nullptr;    // accumulation (lineaire, somme)
static unsigned char* s_rt_argb = nullptr; // ARGB 8-bit pour affichage
static unsigned char* s_rt_argb_host = nullptr; // pinned host buffer
static int s_rt_w = 0, s_rt_h = 0;
static int s_rt_accum_count = 0;
static cudaStream_t s_rt_stream = nullptr;

void gpu_realtime_init(const Scene& scene, int width, int height) {
    s_rt_w = width; s_rt_h = height;
```

```
    size_t px = width * height;
    size_t fb_bytes = px * 3 * sizeof(float);

    if (s_rt_frame_fb) cudaFree(s_rt_frame_fb);
    if (s_rt_accum_fb) cudaFree(s_rt_accum_fb);
    if (s_rt_argb) cudaFree(s_rt_argb);
    if (s_rt_argb_host) cudaFreeHost(s_rt_argb_host);

    cudaMalloc(&s_rt_frame_fb, fb_bytes);
    cudaMalloc(&s_rt_accum_fb, fb_bytes);
    cudaMalloc(&s_rt_argb, px * 4);
    cudaMallocHost(&s_rt_argb_host, px * 4);
    cudaMemset(s_rt_accum_fb, 0, fb_bytes);
    s_rt_accum_count = 0;

    if (!s_rt_stream) cudaStreamCreate(&s_rt_stream);

    // Force scene upload
    ImageBuffer dummy(width, height);
    float km = 0;
    gpu_render_scene(scene, dummy, km);
}

// Cached per-session (pas recalcule a chaque frame)
static dim3 s_rt_block, s_rt_grid;
static int s_rt_acc_grid = 0;
static cudaEvent_t s_rt_start = nullptr, s_rt_stop = nullptr;
static bool s_rt_config_done = false;

float gpu_realtime_render_frame(const Scene& scene) {
    GCamera gcam;
    convert_camera_only(scene.camera, gcam);
    cudaMemcpyToSymbol(d_camera_bytes, &gcam, sizeof(GCamera));

    extern GpuRenderer* s_renderer;
    if (!s_renderer) return 0;

    // Config une seule fois (block size, grid, events)
    if (!s_rt_config_done) {
        s_renderer->ensure_framebuffer(s_rt_w, s_rt_h);
        int flat_block = 0, min_grid = 0;
        cudaOccupancyMaxPotentialBlockSize(&min_grid, &flat_block, render_kernel, 0, 0);
        int bx = 16, by = flat_block / bx;
        if (by < 1) by = 1; if (by > 16) by = 16;
        s_rt_block = dim3(bx, by);
        s_rt_grid = dim3((s_rt_w + bx - 1) / bx, (s_rt_h + by - 1) / by);
        int total_elems = s_rt_w * s_rt_h * 3;
        s_rt_acc_grid = (total_elems + 255) / 256;
        cudaEventCreate(&s_rt_start);
        cudaEventCreate(&s_rt_stop);
        s_rt_config_done = true;
    }

    cudaEventRecord(s_rt_start, s_rt_stream);

    // Render + accumulate, tout sur le stream (pas de sync entre les deux)
    render_kernel<<<s_rt_grid, s_rt_block, 0, s_rt_stream>>>(
        s_rt_frame_fb, s_rt_w, s_rt_h,
        s_renderer->d_bvh_nodes_, s_renderer->num_bvh_nodes_,
        s_renderer->d_spheres_, s_renderer->num_spheres_,
        s_renderer->d_planes_, s_renderer->num_planes_,
        s_renderer->d_materials_,
        s_renderer->d_soa_cx_, s_renderer->d_soa_cy_,
        s_renderer->d_soa_cz_, s_renderer->d_soa_r_, s_renderer->d_soa_mat_);

    accumulate_kernel<<<s_rt_acc_grid, 256, 0, s_rt_stream>>>(
        s_rt_frame_fb, s_rt_accum_fb, s_rt_w * s_rt_h);

    cudaEventRecord(s_rt_stop, s_rt_stream);
    // PAS de cudaStreamSynchronize ici - le tonemap synchronisera
    s_rt_accum_count++;

    float ms = 0;
    cudaEventSynchronize(s_rt_stop);
    cudaEventElapsedTime(&ms, s_rt_start, s_rt_stop);
    return ms;
}

int gpu_realtime_tonemap(unsigned char* argb_host, int width, int height) {
    dim3 block(16, 16);
    dim3 grid((width + 15) / 16, (height + 15) / 16);

    tonemap_kernel<<<grid, block, 0, s_rt_stream>>>(
        s_rt_accum_fb, s_rt_argb, width, height, s_rt_accum_count);

    // Transfer ARGB seulement (4 bytes/pixel au lieu de 12 bytes/pixel float)
    cudaMemcpyAsync(argb_host, s_rt_argb, width * height * 4,
```

```
                    cudaMemcpyDeviceToHost, s_rt_stream);
    cudaStreamSynchronize(s_rt_stream);

    return s_rt_accum_count;
}

void gpu_realtime_reset_accum() {
    if (s_rt_accum_fb)
        cudaMemset(s_rt_accum_fb, 0, s_rt_w * s_rt_h * 3 * sizeof(float));
    s_rt_accum_count = 0;
}
```

7.11 src/gpu/gpu_renderer.h File Reference

```
#include "scene/scene.h"
#include "utils/image.h"
```

Classes

- class GpuRenderer

Functions

- void gpu_render_scene (const Scene &scene, ImageBuffer &image, float &kernel_ms)
- void gpu_render_scene_denoised (const Scene &scene, ImageBuffer &image, float &kernel_ms, float sigma_color=0.1f)
- void gpu_realtime_init (const Scene &scene, int width, int height)
- float gpu_realtime_render_frame (const Scene &scene)
- int gpu_realtime_tonemap (unsigned char ∗argb_host, int width, int height)
- void gpu_realtime_reset_accum ()

7.11.1 Function Documentation

7.11.1.1 gpu_realtime_init()

```
void gpu_realtime_init (
            const Scene & scene,
            int width,
            int height)
```

Definition at line 917 of file gpu_renderer.cu.

7.11.1.2 gpu_realtime_render_frame()

```
float gpu_realtime_render_frame (
            const Scene & scene)
```

Definition at line 36 of file gpu_renderer.cu.

7.11.1.3 gpu_realtime_reset_accum()

```
void gpu_realtime_reset_accum ()
```

Definition at line 1012 of file gpu_renderer.cu.

7.11.1.4 gpu_realtime_tonemap()

```
int gpu_realtime_tonemap (
            unsigned char * argb_host,
            int width,
            int height)
```

Definition at line 997 of file gpu_renderer.cu.

7.11.1.5 gpu_render_scene()

```
void gpu_render_scene (
            const Scene & scene,
            ImageBuffer & image,
            float & kernel_ms)
```

Definition at line 788 of file gpu_renderer.cu.

7.11.1.6 gpu_render_scene_denoised()

```
void gpu_render_scene_denoised (
            const Scene & scene,
            ImageBuffer & image,
            float & kernel_ms,
            float sigma_color = 0.1f)
```

Definition at line 37 of file gpu_renderer.cu.

7.12 gpu_renderer.h

Go to the documentation of this file.

```
#ifndef RT_GPU_RENDERER_H
#define RT_GPU_RENDERER_H

// GPU renderer interface - includable by MSVC (no CUDA qualifiers in this header)
// The actual GpuRenderer class internals are in gpu_renderer.cu

#include "scene/scene.h"
#include "utils/image.h"

// Forward declarations for GPU types (defined in gpu_types.cuh, only visible to nvcc)
struct GSphere;
struct GPlane;
struct GMaterial;
struct GBVHNode;
struct GCamera;

// Host-side GPU renderer interface
class GpuRenderer {
public:
```

```
    GpuRenderer();
    ~GpuRenderer();

    // Upload scene data to GPU (call once)
    void upload_scene(const GSphere* spheres, int num_spheres,
                      const GPlane* planes, int num_planes,
                      const GMaterial* materials, int num_materials,
                      const GBVHNode* bvh_nodes, int num_bvh_nodes);

    // Render to host buffer (RGB float, 3 floats per pixel)
    void render(const GCamera& camera, float* h_output, int width, int height);

    // Get last render time in ms
    float last_render_ms() const { return last_ms_; }

    // Public accessors for realtime GPU rendering
    friend float gpu_realtime_render_frame(const Scene&);
    friend void gpu_render_scene_denoised(const Scene&, ImageBuffer&, float&, float);

private:
    GSphere* d_spheres_ = nullptr;
    // SoA spheres
    float* d_soa_cx_ = nullptr;
    float* d_soa_cy_ = nullptr;
    float* d_soa_cz_ = nullptr;
    float* d_soa_r_ = nullptr;
    int* d_soa_mat_ = nullptr;
    GPlane* d_planes_ = nullptr;
    GMaterial* d_materials_ = nullptr;
    GBVHNode* d_bvh_nodes_ = nullptr;
    float* d_framebuffer_ = nullptr;

    int num_spheres_ = 0;
    int num_planes_ = 0;
    int num_materials_ = 0;
    int num_bvh_nodes_ = 0;
    int fb_width_ = 0;
    int fb_height_ = 0;

    float last_ms_ = 0.0f;

public:
    void ensure_framebuffer(int width, int height);
private:
    void free_scene();
    void free_framebuffer();
};

// Bridge function: converts CPU Scene -> GPU data + renders
// Defined in gpu_renderer.cu (compiled by nvcc)
void gpu_render_scene(const Scene& scene, ImageBuffer& image,
                      float& kernel_ms);

// GPU render + bilateral denoise sur GPU
void gpu_render_scene_denoised(const Scene& scene, ImageBuffer& image,
                               float& kernel_ms, float sigma_color = 0.1f);

// GPU-side progressive rendering (no host transfer per frame)
// Initialise les buffers GPU pour l'accumulation
void gpu_realtime_init(const Scene& scene, int width, int height);
// Rend 1 frame et accumule sur GPU, retourne le kernel time
float gpu_realtime_render_frame(const Scene& scene);
// Tonemap GPU -> host ARGB buffer, retourne total samples
int gpu_realtime_tonemap(unsigned char* argb_host, int width, int height);
// Reset l'accumulation (quand camera bouge)
void gpu_realtime_reset_accum();

#endif // RT_GPU_RENDERER_H
```

7.13 src/gpu/gpu_types.cuh File Reference

```
#include <math.h>
#include <cstdint>
```

Classes

- struct GRay

- struct GAABB
- struct GMaterial
- struct GSphere
- struct GSphereSoA
- struct GPlane
- struct GHitRecord
- struct GBVHNode
- struct GCamera

Macros

- #define RT_GPU_TYPES_CUH

Enumerations

- enum GMaterialType : uint8_t { GMAT_LAMBERTIAN = 0 , GMAT_METAL = 1 , GMAT_DIELECTRIC = 2 , GMAT_EMISSIVE = 3 }

Functions

- struct __align__ (16) GVec3
- __host__ __device__ GVec3 operator∗ (float t, const GVec3 &v)
- __host__ __device__ float dot (const GVec3 &a, const GVec3 &b)
- __host__ __device__ GVec3 cross (const GVec3 &a, const GVec3 &b)
- __host__ __device__ GVec3 normalize (const GVec3 &v)
- __host__ __device__ GVec3 reflect (const GVec3 &v, const GVec3 &n)
- __host__ __device__ GVec3 refract (const GVec3 &uv, const GVec3 &n, float etai_over_etat)

7.13.1 Macro Definition Documentation

7.13.1.1 RT_GPU_TYPES_CUH

```
#define RT_GPU_TYPES_CUH
```

Definition at line 2 of file gpu_types.cuh.

7.13.2 Enumeration Type Documentation

7.13.2.1 GMaterialType

```
enum GMaterialType : uint8_t
```

Enumerator

GMAT_LAMBERTIAN	
GMAT_METAL	

GMAT_DIELECTRIC	
GMAT_EMISSIVE	

Definition at line 102 of file gpu_types.cuh.

7.13.3 Function Documentation

7.13.3.1 __align__()

```
struct __align__ (
            16 )
```

Definition at line 1 of file gpu_types.cuh.

7.13.3.2 cross()

```
__host__ __device__ GVec3 cross (
            const GVec3 & a,
            const GVec3 & b)  [inline]
```

Definition at line 35 of file gpu_types.cuh.

7.13.3.3 dot()

```
__host__ __device__ float dot (
            const GVec3 & a,
            const GVec3 & b)  [inline]
```

Definition at line 34 of file gpu_types.cuh.

7.13.3.4 normalize()

```
__host__ __device__ GVec3 normalize (
            const GVec3 & v)  [inline]
```

Definition at line 38 of file gpu_types.cuh.

7.13.3.5 operator∗()

```
__host__ __device__ GVec3 operator* (
            float t,
            const GVec3 & v)  [inline]
```

Definition at line 33 of file gpu_types.cuh.

7.13.3.6 reflect()

```
__host__ __device__ GVec3 reflect (
            const GVec3 & v,
            const GVec3 & n)  [inline]
```

Definition at line 46 of file gpu_types.cuh.

7.13.3.7 refract()

```
__host__ __device__ GVec3 refract (
            const GVec3 & uv,
            const GVec3 & n,
            float etai_over_etat)  [inline]
```

Definition at line 49 of file gpu_types.cuh.

7.14 gpu_types.cuh

Go to the documentation of this file.

```
00001 #ifndef RT_GPU_TYPES_CUH
00002 #define RT_GPU_TYPES_CUH
00003
00004 #include <math.h>
00005 #include <cstdint>
00006
00007 // ============================================================
00008 // GPU Vec3 (float, POD, no vtable)
00009 // ============================================================
00010 struct __align__(16) GVec3 {
00011     float x, y, z;
00012     float _pad; // padding to 16 bytes for coalesced 128-bit loads
00013
00014     __host__ __device__ GVec3() : x(0), y(0), z(0), _pad(0) {}
00015     __host__ __device__ GVec3(float x_, float y_, float z_) : x(x_), y(y_), z(z_), _pad(0) {}
00016
00017     __host__ __device__ GVec3 operator+(const GVec3& v) const { return {x+v.x, y+v.y, z+v.z}; }
00018     __host__ __device__ GVec3 operator-(const GVec3& v) const { return {x-v.x, y-v.y, z-v.z}; }
00019     __host__ __device__ GVec3 operator*(float t) const { return {x*t, y*t, z*t}; }
00020     __host__ __device__ GVec3 operator*(const GVec3& v) const { return {x*v.x, y*v.y, z*v.z}; }
00021     __host__ __device__ GVec3 operator/(float t) const { float inv = 1.0f/t; return {x*inv, y*inv,
      z*inv}; }
00022     __host__ __device__ GVec3 operator-() const { return {-x, -y, -z}; }
00023     __host__ __device__ GVec3& operator+=(const GVec3& v) { x+=v.x; y+=v.y; z+=v.z; return *this; }
00024     __host__ __device__ GVec3& operator*=(float t) { x*=t; y*=t; z*=t; return *this; }
00025
00026     __host__ __device__ float length_squared() const { return x*x + y*y + z*z; }
00027     __host__ __device__ float length() const { return sqrtf(length_squared()); }
00028     __host__ __device__ bool near_zero() const {
00029         return fabsf(x) < 1e-6f && fabsf(y) < 1e-6f && fabsf(z) < 1e-6f;
00030     }
00031 };
00032
00033 __host__ __device__ inline GVec3 operator*(float t, const GVec3& v) { return {t*v.x, t*v.y, t*v.z}; }
00034 __host__ __device__ inline float dot(const GVec3& a, const GVec3& b) { return a.x*b.x + a.y*b.y +
      a.z*b.z; }
00035 __host__ __device__ inline GVec3 cross(const GVec3& a, const GVec3& b) {
00036     return {a.y*b.z - a.z*b.y, a.z*b.x - a.x*b.z, a.x*b.y - a.y*b.x};
00037 }
00038 __host__ __device__ inline GVec3 normalize(const GVec3& v) {
00039     float len_sq = v.length_squared();
00040     if (len_sq > 0.0f) {
00041         float inv_len = rsqrtf(len_sq); // rsqrtf = hardware 1/sqrt, plus rapide que sqrtf+div
00042         return v * inv_len;
00043     }
00044     return v;
00045 }
00046 __host__ __device__ inline GVec3 reflect(const GVec3& v, const GVec3& n) {
00047     return v - 2.0f * dot(v, n) * n;
00048 }
```

```
__host__ __device__ inline GVec3 refract(const GVec3& uv, const GVec3& n, float etai_over_etat) {
    float cos_theta = fminf(dot(-uv, n), 1.0f);
    GVec3 r_perp = etai_over_etat * (uv + cos_theta * n);
    GVec3 r_parallel = -sqrtf(fabsf(1.0f - r_perp.length_squared())) * n;
    return r_perp + r_parallel;
}

// ============================================================
// GPU Ray (with precomputed inverse direction)
// ============================================================
struct GRay {
    GVec3 orig;
    GVec3 dir;
    GVec3 inv_dir;
    GVec3 orig_mul_inv; // precomputed -orig * inv_dir for FMA AABB test

    __host__ __device__ GRay() {}
    __host__ __device__ GRay(const GVec3& o, const GVec3& d)
        : orig(o), dir(d),
          inv_dir(1.0f/d.x, 1.0f/d.y, 1.0f/d.z),
          orig_mul_inv(-o.x/d.x, -o.y/d.y, -o.z/d.z) {}

    __host__ __device__ GVec3 at(float t) const { return orig + dir * t; }
};

// ============================================================
// GPU AABB
// ============================================================
struct GAABB {
    GVec3 mn, mx; // min, max

    __host__ __device__ GAABB() : mn(1e30f, 1e30f, 1e30f), mx(-1e30f, -1e30f, -1e30f) {}
    __host__ __device__ GAABB(const GVec3& a, const GVec3& b) : mn(a), mx(b) {}

    __host__ __device__ bool hit(const GRay& r, float t_min, float t_max) const {
        // FMA-based slab test (Opt#7 level 4 from RTIOW CUDA article)
        // fmaf(invDir, bound, origMulInv) = invDir*bound + (-orig*invDir) = (bound-orig)*invDir
        float tx0 = fmaf(r.inv_dir.x, mn.x, r.orig_mul_inv.x);
        float tx1 = fmaf(r.inv_dir.x, mx.x, r.orig_mul_inv.x);
        float ty0 = fmaf(r.inv_dir.y, mn.y, r.orig_mul_inv.y);
        float ty1 = fmaf(r.inv_dir.y, mx.y, r.orig_mul_inv.y);
        float tz0 = fmaf(r.inv_dir.z, mn.z, r.orig_mul_inv.z);
        float tz1 = fmaf(r.inv_dir.z, mx.z, r.orig_mul_inv.z);

        float tEnter = fmaxf(fmaxf(fminf(tx0,tx1), fminf(ty0,ty1)), fmaxf(fminf(tz0,tz1), t_min));
        float tExit  = fminf(fminf(fmaxf(tx0,tx1), fmaxf(ty0,ty1)), fminf(fmaxf(tz0,tz1), t_max));
        return tEnter <= tExit;
    }
};

// ============================================================
// GPU Material (tagged union, pas de vtable)
// ============================================================
enum GMaterialType : uint8_t { GMAT_LAMBERTIAN = 0, GMAT_METAL = 1, GMAT_DIELECTRIC = 2, GMAT_EMISSIVE = 3 };

struct GMaterial {
    GVec3 albedo;
    float param;           // fuzz pour metal, refraction_index pour dielectric
    GMaterialType type;

    __host__ __device__ GMaterial()
        : albedo(0.5f, 0.5f, 0.5f), param(0.0f), type(GMAT_LAMBERTIAN) {}
};

// ============================================================
// GPU Sphere
// ============================================================
struct GSphere {
    GVec3 center;
    float radius;
    int material_id;
};

// SoA layout pour spheres (acces coalescents sur GPU)
struct GSphereSoA {
    float* cx;  // center.x[]
    float* cy;  // center.y[]
    float* cz;  // center.z[]
    float* r;   // radius[]
    int* mat;   // material_id[]
    int count;
};

// ============================================================
// GPU Plane
// ============================================================
```

```
struct GPlane {
    GVec3 point;
    GVec3 normal;
    int material_id;
};

// ============================================================
// GPU HitRecord
// ============================================================
struct GHitRecord {
    GVec3 p;
    GVec3 normal;
    float t;
    int material_id;
    bool front_face;

    __host__ __device__ void set_face_normal(const GRay& r, const GVec3& outward_normal) {
        front_face = dot(r.dir, outward_normal) < 0.0f;
        normal = front_face ? outward_normal : -outward_normal;
    }
};

// ============================================================
// GPU Flat BVH Node
// ============================================================
struct GBVHNode {
    GAABB bbox;
    uint32_t data;        // leaf: primitives_offset | interior: second_child_offset
    uint16_t num_prims;   // 0 = interior
    uint8_t split_axis;
    uint8_t prim_type;    // 0 = sphere, 1 = plane (for leaves)
};

// ============================================================
// GPU Camera
// ============================================================
struct GCamera {
    GVec3 origin;
    GVec3 pixel00_loc;
    GVec3 pixel_delta_u;
    GVec3 pixel_delta_v;
    GVec3 defocus_disk_u;
    GVec3 defocus_disk_v;
    float defocus_angle;
    int image_width;
    int image_height;
    int samples_per_pixel;
    int max_depth;
};

#endif // RT_GPU_TYPES_CUH
```

7.15 src/hybrid/hybrid_renderer.cu File Reference

```
#include "hybrid/hybrid_renderer.h"
#include "gpu/gpu_renderer.h"
#include "gpu/gpu_types.cuh"
#include "scene/primitives.cuh"
#include "scene/material.cuh"
#include "scene/bvh_builder.h"
#include "cpu/cpu_renderer.h"
#include "math/random.cuh"
#include "math/interval.cuh"
#include <thread>
#include <atomic>
#include <chrono>
#include <cstdio>
#include <vector>
#include <cmath>
```

Functions

- void hybrid_render_scene (const Scene &scene, ImageBuffer &image, float gpu_fraction, float &cpu_time↩ _ms, float &gpu_time_ms)

7.15.1 Function Documentation

7.15.1.1 hybrid_render_scene()

```
void hybrid_render_scene (
            const Scene & scene,
            ImageBuffer & image,
            float gpu_fraction,
            float & cpu_time_ms,
            float & gpu_time_ms)
```

Definition at line 22 of file hybrid_renderer.cu.

7.16 hybrid_renderer.cu

Go to the documentation of this file.

```
00001 // Hybrid CPU+GPU renderer avec work stealing par tuiles
00002 // Le GPU rend un batch de tuiles (la fraction haute de l'image)
00003 // Le CPU rend les tuiles restantes en multi-thread
00004 // Les deux travaillent en parallele
00005
00006 #include "hybrid/hybrid_renderer.h"
00007 #include "gpu/gpu_renderer.h"
00008 #include "gpu/gpu_types.cuh"
00009 #include "scene/primitives.cuh"
00010 #include "scene/material.cuh"
00011 #include "scene/bvh_builder.h"
00012 #include "cpu/cpu_renderer.h"
00013 #include "math/random.cuh"
00014 #include "math/interval.cuh"
00015 #include <thread>
00016 #include <atomic>
00017 #include <chrono>
00018 #include <cstdio>
00019 #include <vector>
00020 #include <cmath>
00021
00022 void hybrid_render_scene(const Scene& scene, ImageBuffer& image,
00023                          float gpu_fraction,
00024                          float& cpu_time_ms, float& gpu_time_ms) {
00025     int w = image.width();
00026     int h = image.height();
00027     constexpr int TILE_SIZE = 32;
00028
00029     int tiles_x = (w + TILE_SIZE - 1) / TILE_SIZE;
00030     int tiles_y = (h + TILE_SIZE - 1) / TILE_SIZE;
00031     int total_tiles = tiles_x * tiles_y;
00032
00033     // Repartition : GPU prend les premieres tuiles, CPU le reste
00034     int gpu_tiles = (int)(total_tiles * gpu_fraction);
00035     if (gpu_tiles < 0) gpu_tiles = 0;
00036     if (gpu_tiles > total_tiles) gpu_tiles = total_tiles;
00037     int cpu_start_tile = gpu_tiles;
00038
00039     // Calcul des lignes GPU (tuiles completes depuis le haut)
00040     int gpu_rows = 0;
00041     if (gpu_tiles > 0) {
00042         int gpu_tile_rows = (gpu_tiles + tiles_x - 1) / tiles_x;
00043         gpu_rows = gpu_tile_rows * TILE_SIZE;
00044         if (gpu_rows > h) gpu_rows = h;
00045         gpu_tiles = gpu_tile_rows * tiles_x; // ajuster au nombre reel
00046         cpu_start_tile = gpu_tiles;
00047     }
```

```

    fprintf(stderr, "Hybrid: GPU %d tuiles (lignes 0-%d), CPU %d tuiles (lignes %d-%d), total %d\n",
            gpu_tiles, gpu_rows, total_tiles - cpu_start_tile, gpu_rows, h, total_tiles);

    // --- GPU thread : rend la partie haute ---
    std::thread gpu_thread;
    float gpu_kernel_ms = 0;

    if (gpu_rows > 0) {
        gpu_thread = std::thread([&]() {
            auto t0 = std::chrono::high_resolution_clock::now();

            // GPU rend toute l'image (plus simple que de modifier le kernel)
            // On ne copie que les lignes GPU dans l'ImageBuffer
            float km = 0;
            gpu_render_scene(scene, image, km);
            gpu_kernel_ms = km;

            auto t1 = std::chrono::high_resolution_clock::now();
            gpu_time_ms = (float)std::chrono::duration<double, std::milli>(t1 - t0).count();
        });
    }

    // --- CPU : rend les tuiles restantes avec work stealing ---
    if (cpu_start_tile < total_tiles) {
        auto t0 = std::chrono::high_resolution_clock::now();

        CpuRenderer cpu_renderer(scene, image);
        cpu_renderer.enable_bvh();

        int num_threads = (int)std::thread::hardware_concurrency();
        if (num_threads <= 0) num_threads = 4;

        int cpu_total = total_tiles - cpu_start_tile;
        std::atomic<int> next_tile(0);

        auto worker = [&]() {
            while (true) {
                int local_idx = next_tile.fetch_add(1);
                if (local_idx >= cpu_total) break;

                int tile_idx = cpu_start_tile + local_idx;
                int tx = tile_idx % tiles_x;
                int ty = tile_idx / tiles_x;
                int x0 = tx * TILE_SIZE;
                int y0 = ty * TILE_SIZE;
                int x1 = std::min(x0 + TILE_SIZE, w);
                int y1 = std::min(y0 + TILE_SIZE, h);

                cpu_renderer.render_tile(x0, y0, x1, y1);
            }
        };

        std::vector<std::thread> threads;
        for (int i = 0; i < num_threads; i++)
            threads.emplace_back(worker);
        for (auto& t : threads)
            t.join();

        auto t1 = std::chrono::high_resolution_clock::now();
        cpu_time_ms = (float)std::chrono::duration<double, std::milli>(t1 - t0).count();
    }

    if (gpu_thread.joinable())
        gpu_thread.join();

    fprintf(stderr, "Hybrid: CPU %.1fms, GPU kernel %.1fms (total GPU %.1fms)\n",
            cpu_time_ms, gpu_kernel_ms, gpu_time_ms);
}
```

7.17 src/hybrid/hybrid_renderer.h File Reference

```
#include "scene/scene.h"
#include "utils/image.h"
```

Functions

- void hybrid_render_scene (const Scene &scene, ImageBuffer &image, float gpu_fraction, float &cpu_time↩ _ms, float &gpu_time_ms)

7.17.1 Function Documentation

7.17.1.1 hybrid_render_scene()

```
void hybrid_render_scene (
            const Scene & scene,
            ImageBuffer & image,
            float gpu_fraction,
            float & cpu_time_ms,
            float & gpu_time_ms)
```

Definition at line 22 of file hybrid_renderer.cu.

7.18 hybrid_renderer.h

Go to the documentation of this file.

```
00001 #ifndef RT_HYBRID_RENDERER_H
00002 #define RT_HYBRID_RENDERER_H
00003
00004 #include "scene/scene.h"
00005 #include "utils/image.h"
00006
00007 // Rendu hybride CPU+GPU avec work stealing par tuiles
00008 // GPU et CPU consomment des tuiles depuis une queue partagee
00009 void hybrid_render_scene(const Scene& scene, ImageBuffer& image,
00010                          float gpu_fraction,  // fraction initiale pour le GPU
00011                          float& cpu_time_ms, float& gpu_time_ms);
00012
00013 #endif // RT_HYBRID_RENDERER_H
```

7.19 src/main.cpp File Reference

```
#include "scene/scene.h"
#include "utils/image.h"
#include "utils/denoise.h"
#include "cpu/cpu_renderer.h"
#include <iostream>
#include <string>
#include <chrono>
#include <cstdlib>
```

Functions

- int main (int argc, char *argv[])

7.19.1 Function Documentation

7.19.1.1 main()

```
int main (
            int argc,
            char * argv[])
```

Definition at line 16 of file main.cpp.

7.20 main.cpp

Go to the documentation of this file.

```
#include "scene/scene.h"
#include "utils/image.h"
#include "utils/denoise.h"
#include "cpu/cpu_renderer.h"
#include <iostream>
#include <string>
#include <chrono>
#include <cstdlib>

#ifdef RT_CUDA_ENABLED
#include "gpu/gpu_renderer.h"
#include "gpu/gpu_realtime.h"
#include "hybrid/hybrid_renderer.h"
#endif

int main(int argc, char* argv[]) {
    std::string scene_name = "three_spheres";
    std::string output_file = "output/render.ppm";
    int width = 0;
    int spp = 0;
    int threads = 0;
    bool use_bvh = true;
    bool single_thread = false;
    bool use_gpu = false;
    bool use_hybrid = false;
    bool use_realtime = false;
    bool use_denoise = false;
    bool use_denoise_gpu = false;
    float gpu_frac = 0.7f;

    for (int i = 1; i < argc; ++i) {
        std::string arg = argv[i];
        if (arg == "--scene" && i + 1 < argc) scene_name = argv[++i];
        else if (arg == "--output" && i + 1 < argc) output_file = argv[++i];
        else if (arg == "--width" && i + 1 < argc) width = std::atoi(argv[++i]);
        else if (arg == "--spp" && i + 1 < argc) spp = std::atoi(argv[++i]);
        else if (arg == "--threads" && i + 1 < argc) threads = std::atoi(argv[++i]);
        else if (arg == "--no-bvh") use_bvh = false;
        else if (arg == "--single-thread") single_thread = true;
        else if (arg == "--gpu") use_gpu = true;
        else if (arg == "--hybrid") use_hybrid = true;
        else if (arg == "--realtime") use_realtime = true;
        else if (arg == "--denoise") use_denoise = true;
        else if (arg == "--denoise-gpu") use_denoise_gpu = true;
        else if (arg == "--gpu-frac" && i + 1 < argc) gpu_frac = (float)std::atof(argv[++i]);
        else if (arg == "--help") {
            std::cout « "Usage: raytracer [options]\n"
                      « "  --scene <name>       Scene: three_spheres, final\n"
                      « "  --output <file>      Output PPM file\n"
                      « "  --width <pixels>     Image width\n"
                      « "  --spp <samples>      Samples per pixel\n"
                      « "  --threads <n>        Number of CPU threads (0=auto)\n"
                      « "  --no-bvh             Disable BVH\n"
                      « "  --single-thread      Single-threaded CPU\n"
                      « "  --gpu                Use GPU (CUDA) renderer\n"
                      « "  --hybrid             Use hybrid CPU+GPU renderer\n"
                      « "  --gpu-frac <0-1>     GPU fraction in hybrid mode (default: 0.7)\n";
            return 0;
        }
    }

    Scene scene;
    if (scene_name == "final")
        scene = Scene::build_final_scene();
    else if (scene_name == "triangle")
        scene = Scene::build_triangle_scene();
    else if (scene_name == "cornell")
        scene = Scene::build_cornell_box();
    else if (scene_name == "showcase")
        scene = Scene::build_showcase();
    else if (scene_name == "hall")
        scene = Scene::build_hall();
    else if (scene_name == "galaxy")
        scene = Scene::build_galaxy();
    else if (scene_name == "minimal")
        scene = Scene::build_minimal();
    else
        scene = Scene::build_three_spheres();

    if (width > 0) {
        scene.camera.image_width = width;
        scene.camera.initialize();
```

```
    }
    if (spp > 0)
        scene.camera.samples_per_pixel = spp;

    int img_w = scene.camera.get_image_width();
    int img_h = scene.camera.get_image_height();
    ImageBuffer image(img_w, img_h);

    std::cerr « "Rendering " « img_w « "x" « img_h
              « " (" « scene.camera.samples_per_pixel « " spp, depth "
              « scene.camera.max_depth « ")";

#ifdef RT_CUDA_ENABLED
    if (use_realtime) {
        std::cerr « " [REALTIME GPU]\n";
        int rt_w = (width > 0) ? width : 640;
        int rt_h = (int)(rt_w / scene.camera.aspect_ratio);
        scene.camera.image_width = rt_w;
        scene.camera.initialize();
        gpu_realtime_render(scene, rt_w, rt_h);
        return 0;
    }

    if (use_hybrid) {
        std::cerr « " [HYBRID gpu=" « (int)(gpu_frac*100) « "%]\n";
        float cpu_ms = 0, gpu_ms = 0;
        auto start = std::chrono::high_resolution_clock::now();
        hybrid_render_scene(scene, image, gpu_frac, cpu_ms, gpu_ms);
        auto end = std::chrono::high_resolution_clock::now();
        double elapsed = std::chrono::duration<double>(end - start).count();
        std::cerr « "Total: " « elapsed « "s\n";
        image.write_ppm(output_file);
        std::cerr « "Saved to: " « output_file « "\n";
        return 0;
    }

    if (use_gpu) {
        std::cerr « (use_denoise_gpu ? " [GPU+DENOISE]\n" : " [GPU]\n");
        float kernel_ms = 0;
        auto start = std::chrono::high_resolution_clock::now();
        if (use_denoise_gpu)
            gpu_render_scene_denoised(scene, image, kernel_ms);
        else
            gpu_render_scene(scene, image, kernel_ms);
        auto end = std::chrono::high_resolution_clock::now();
        double elapsed = std::chrono::duration<double>(end - start).count();
        std::cerr « "GPU kernel: " « kernel_ms « " ms\n";
        std::cerr « "Total (incl. transfer): " « elapsed « "s\n";
        image.write_ppm(output_file);
        std::cerr « "Saved to: " « output_file « "\n";
        return 0;
    }
#else
    if (use_gpu || use_hybrid) {
        std::cerr « "\nError: GPU not compiled. Rebuild with -DENABLE_CUDA=ON\n";
        return 1;
    }
#endif

    std::cerr « (use_bvh ? " [BVH]" : "") « "\n";
    CpuRenderer renderer(scene, image);
    if (use_bvh) {
        std::cerr « "Building BVH...\n";
        renderer.enable_bvh();
    }

    auto start = std::chrono::high_resolution_clock::now();
    if (single_thread)
        renderer.render();
    else
        renderer.render_threaded(threads);
    auto end = std::chrono::high_resolution_clock::now();

    double elapsed = std::chrono::duration<double>(end - start).count();
    std::cerr « "Render time: " « elapsed « "s\n";
    if (use_denoise) {
        std::cerr « "Denoising...\n";
        denoise_bilateral(image, 3, 0.1);
    }
    image.write_ppm(output_file);
    std::cerr « "Saved to: " « output_file « "\n";

    return 0;
}
```

7.21 src/math/aabb.cuh File Reference

```
#include "math/vec3.cuh"
#include "math/ray.cuh"
#include "math/interval.cuh"
#include <algorithm>
#include <cmath>
```

Classes

- class AABB

Macros

- #define RT_MATH_AABB_CUH

Functions

- RT_HOSTDEV AABB surrounding_box (const AABB &a, const AABB &b)

7.21.1 Macro Definition Documentation

7.21.1.1 RT_MATH_AABB_CUH

```
#define RT_MATH_AABB_CUH
```

Definition at line 8 of file aabb.cuh.

7.21.2 Function Documentation

7.21.2.1 surrounding_box()

```
RT_HOSTDEV AABB surrounding_box (
            const AABB & a,
            const AABB & b)  [inline]
```

Definition at line 107 of file aabb.cuh.

7.22 aabb.cuh

Go to the documentation of this file.

```
// aabb.cuh - Axis-Aligned Bounding Box pour l'acceleration BVH.
// Chaque noeud BVH stocke une AABB ; la traversee teste le rayon
// contre la boite avant de descendre dans les enfants.
// Le test slab (Williams et al. 2005) utilise inv_dir precalcule
// dans Ray pour transformer 6 divisions en 6 multiplications.

#ifndef RT_MATH_AABB_CUH
#define RT_MATH_AABB_CUH

#include "math/vec3.cuh"
#include "math/ray.cuh"
#include "math/interval.cuh"
#include <algorithm>
#include <cmath>

class AABB {
public:
    Interval x, y, z; // bornes sur chaque axe

    RT_HOSTDEV AABB() {} // intervalles vides par defaut

    RT_HOSTDEV AABB(const Interval& x, const Interval& y, const Interval& z)
        : x(x), y(y), z(z) {}

    // Construction depuis deux coins opposes (ordre quelconque)
    RT_HOSTDEV AABB(const Point3& a, const Point3& b) {
        x = (a[0] <= b[0]) ? Interval(a[0], b[0]) : Interval(b[0], a[0]);
        y = (a[1] <= b[1]) ? Interval(a[1], b[1]) : Interval(b[1], a[1]);
        z = (a[2] <= b[2]) ? Interval(a[2], b[2]) : Interval(b[2], a[2]);
    }

    // Acces generique par indice d'axe (0=x, 1=y, 2=z)
    // Utilise par le BVH pour choisir l'axe de split
    RT_HOSTDEV const Interval& axis_interval(int n) const {
        if (n == 1) return y;
        if (n == 2) return z;
        return x;
    }

    RT_HOSTDEV Point3 min_point() const { return Point3(x.min, y.min, z.min); }
    RT_HOSTDEV Point3 max_point() const { return Point3(x.max, y.max, z.max); }

    // Test slab de Williams et al. : intersection rayon-boite en 6 mul + 6 cmp.
    // inv_dir precalcule dans Ray transforme (bound - origin) / dir en multiplication.
    // On resserre progressivement [t_min, t_max] par axe ; si l'intervalle
    // devient vide (t_max < t_min), le rayon rate la boite -> early exit.
    RT_HOSTDEV bool hit(const Ray& r, double t_min, double t_max) const {
        const Point3& ro = r.orig;
        const Vec3& inv = r.inv_dir;

        // Axe X
        double tx1 = (x.min - ro[0]) * inv[0];
        double tx2 = (x.max - ro[0]) * inv[0];
        if (tx1 > tx2) { double tmp = tx1; tx1 = tx2; tx2 = tmp; }
        if (tx1 > t_min) t_min = tx1;
        if (tx2 < t_max) t_max = tx2;
        if (t_max < t_min) return false;

        // Axe Y
        double ty1 = (y.min - ro[1]) * inv[1];
        double ty2 = (y.max - ro[1]) * inv[1];
        if (ty1 > ty2) { double tmp = ty1; ty1 = ty2; ty2 = tmp; }
        if (ty1 > t_min) t_min = ty1;
        if (ty2 < t_max) t_max = ty2;
        if (t_max < t_min) return false;

        // Axe Z
        double tz1 = (z.min - ro[2]) * inv[2];
        double tz2 = (z.max - ro[2]) * inv[2];
        if (tz1 > tz2) { double tmp = tz1; tz1 = tz2; tz2 = tmp; }
        if (tz1 > t_min) t_min = tz1;
        if (tz2 < t_max) t_max = tz2;
        if (t_max < t_min) return false;

        return true;
    }

    // Surcharge pour les tests qui passent un Interval
    RT_HOSTDEV bool hit(const Ray& r, Interval ray_t) const {
        return hit(r, ray_t.min, ray_t.max);
    }

```

```
    // Surface totale de la boite : utilisee par la SAH (Surface Area Heuristic)
    // pour estimer le cout de split lors de la construction du BVH.
    // Cout ~ surface_enfant / surface_parent * nb_primitives_enfant
    RT_HOSTDEV double surface_area() const {
        double dx = x.size(), dy = y.size(), dz = z.size();
        return 2.0 * (dx*dy + dy*dz + dz*dx);
    }

    // Axe le plus long : heuristique de split quand SAH n'est pas utilisee
    RT_HOSTDEV int longest_axis() const {
        double sx = x.size(), sy = y.size(), sz = z.size();
        if (sx > sy) return (sx > sz) ? 0 : 2;
        return (sy > sz) ? 1 : 2;
    }

    static const AABB empty;
    static const AABB universe;
};

inline const AABB AABB::empty = AABB(Interval::empty, Interval::empty, Interval::empty);
inline const AABB AABB::universe = AABB(Interval::universe, Interval::universe, Interval::universe);

// Union de deux AABB : construit la plus petite boite englobant les deux.
// Utilise recursivement lors de la construction bottom-up du BVH.
inline RT_HOSTDEV AABB surrounding_box(const AABB& a, const AABB& b) {
    return AABB(
        Interval(std::fmin(a.x.min, b.x.min), std::fmax(a.x.max, b.x.max)),
        Interval(std::fmin(a.y.min, b.y.min), std::fmax(a.y.max, b.y.max)),
        Interval(std::fmin(a.z.min, b.z.min), std::fmax(a.z.max, b.z.max))
    );
}

#endif // RT_MATH_AABB_CUH
```

7.23 src/math/interval.cuh File Reference

```
#include <cmath>
#include <limits>
#include "math/vec3.cuh"
```

Classes

- class Interval

Macros

- #define RT_MATH_INTERVAL_CUH

7.23.1 Macro Definition Documentation

7.23.1.1 RT_MATH_INTERVAL_CUH

```
#define RT_MATH_INTERVAL_CUH
```

Definition at line 6 of file interval.cuh.

7.24 interval.cuh

Go to the documentation of this file.

```
// interval.cuh - Intervalle ferme [min, max] sur les reels.
// Utilise partout : parametrage t valide des rayons, bornes AABB,
// clamping des couleurs avant tone mapping.

#ifndef RT_MATH_INTERVAL_CUH
#define RT_MATH_INTERVAL_CUH

#include <cmath>
#include <limits>

#include "math/vec3.cuh" // pour RT_HOSTDEV

class Interval {
public:
    double min, max;

    // Par defaut : intervalle vide (min > max, aucun t valide)
    RT_HOSTDEV Interval()
        : min(+std::numeric_limits<double>::infinity()),
          max(-std::numeric_limits<double>::infinity()) {}

    RT_HOSTDEV Interval(double min, double max) : min(min), max(max) {}

    RT_HOSTDEV double size() const { return max - min; }

    // Inclusion large (bornes incluses)
    RT_HOSTDEV bool contains(double x) const { return min <= x && x <= max; }

    // Inclusion stricte (bornes exclues) : utilise pour ignorer les self-intersections
    // a t~0 (shadow acne) en testant t dans ]epsilon, +inf[
    RT_HOSTDEV bool surrounds(double x) const { return min < x && x < max; }

    RT_HOSTDEV double clamp(double x) const {
        if (x < min) return min;
        if (x > max) return max;
        return x;
    }

    static const Interval empty;
    static const Interval universe;
};

// Singletons inline (C++17) : evitent les problemes d'ordre d'initialisation
inline const Interval Interval::empty    = Interval(+std::numeric_limits<double>::infinity(),
                                                    -std::numeric_limits<double>::infinity());
inline const Interval Interval::universe = Interval(-std::numeric_limits<double>::infinity(),
                                                    +std::numeric_limits<double>::infinity());

#endif // RT_MATH_INTERVAL_CUH
```

7.25 src/math/random.cuh File Reference

```
#include "math/vec3.cuh"
#include <random>
```

Macros

- #define RT_MATH_RANDOM_CUH

Functions

- double random_double ()
- double random_double (double min, double max)
- Vec3 random_vec3 ()

- Vec3 random_vec3 (double min, double max)
- Vec3 random_in_unit_sphere ()
- Vec3 random_unit_vector ()
- Vec3 random_on_hemisphere (const Vec3 &normal)
- Vec3 random_in_unit_disk ()

7.25.1 Macro Definition Documentation

7.25.1.1 RT_MATH_RANDOM_CUH

```
#define RT_MATH_RANDOM_CUH
```

Definition at line 6 of file random.cuh.

7.25.2 Function Documentation

7.25.2.1 random_double() [1/2]

```
double random_double ()  [inline]
```

Definition at line 13 of file random.cuh.

7.25.2.2 random_double() [2/2]

```
double random_double (
            double min,
            double max)  [inline]
```

Definition at line 19 of file random.cuh.

7.25.2.3 random_in_unit_disk()

```
Vec3 random_in_unit_disk ()  [inline]
```

Definition at line 59 of file random.cuh.

7.25.2.4 random_in_unit_sphere()

```
Vec3 random_in_unit_sphere ()  [inline]
```

Definition at line 33 of file random.cuh.

7.25.2.5 random_on_hemisphere()

```
Vec3 random_on_hemisphere (
            const Vec3 & normal)  [inline]
```

Definition at line 50 of file random.cuh.

7.25.2.6 random_unit_vector()

```
Vec3 random_unit_vector ()  [inline]
```

Definition at line 44 of file random.cuh.

7.25.2.7 random_vec3() [1/2]

```
Vec3 random_vec3 ()  [inline]
```

Definition at line 23 of file random.cuh.

7.25.2.8 random_vec3() [2/2]

```
Vec3 random_vec3 (
            double min,
            double max)  [inline]
```

Definition at line 27 of file random.cuh.

7.26 random.cuh

Go to the documentation of this file.

```
00001 // random.cuh - Fonctions aleatoires CPU pour le Monte Carlo.
00002 // thread_local : chaque thread du pool CPU a son propre generateur,
00003 // pas de mutex ni contention. Le GPU utilise curand a la place (gpu_renderer.cu).
00004
00005 #ifndef RT_MATH_RANDOM_CUH
00006 #define RT_MATH_RANDOM_CUH
00007
00008 #include "math/vec3.cuh"
00009 #include <random>
00010
00011 // Mersenne Twister 19937 : periode 2^19937-1, qualite statistique suffisante
00012 // pour le path tracing. Seed via random_device (entropie hardware si disponible).
00013 inline double random_double() {
00014     thread_local std::mt19937 gen(std::random_device{}());
00015     thread_local std::uniform_real_distribution<double> dist(0.0, 1.0);
00016     return dist(gen);
00017 }
00018
00019 inline double random_double(double min, double max) {
00020     return min + (max - min) * random_double();
00021 }
00022
00023 inline Vec3 random_vec3() {
00024     return Vec3(random_double(), random_double(), random_double());
00025 }
00026
00027 inline Vec3 random_vec3(double min, double max) {
00028     return Vec3(random_double(min, max), random_double(min, max), random_double(min, max));
00029 }
00030
```

```
// Rejection sampling dans le cube [-1,1]^3 : ~52% d'acceptance.
// Plus simple et aussi rapide que Box-Muller pour 3 composantes.
inline Vec3 random_in_unit_sphere() {
    while (true) {
        Vec3 p = random_vec3(-1.0, 1.0);
        if (p.length_squared() < 1.0)
            return p;
    }
}

// Direction aleatoire uniforme sur la sphere unite :
// normaliser un point uniforme dans la boule donne une distribution
// uniforme sur la surface (distribution de Lambert pour le diffus).
inline Vec3 random_unit_vector() {
    return unit_vector(random_in_unit_sphere());
}

// Hemisphere cosinus-pondere implicite : si le vecteur pointe
// du mauvais cote de la normale, on le retourne.
inline Vec3 random_on_hemisphere(const Vec3& normal) {
    Vec3 on_sphere = random_unit_vector();
    if (dot(on_sphere, normal) > 0.0)
        return on_sphere;
    return -on_sphere;
}

// Disque unite : utilise pour le depth of field (DOF).
// L'ouverture de la lentille est echantillonnee uniformement sur un disque.
inline Vec3 random_in_unit_disk() {
    while (true) {
        Vec3 p(random_double(-1.0, 1.0), random_double(-1.0, 1.0), 0.0);
        if (p.length_squared() < 1.0)
            return p;
    }
}

#endif // RT_MATH_RANDOM_CUH
```

7.27 src/math/ray.cuh File Reference

```
#include "math/vec3.cuh"
```

Classes

- class Ray

Macros

- #define RT_MATH_RAY_CUH

7.27.1 Macro Definition Documentation

7.27.1.1 RT_MATH_RAY_CUH

```
#define RT_MATH_RAY_CUH
```

Definition at line 7 of file ray.cuh.

7.28 ray.cuh

Go to the documentation of this file.

```
// ray.cuh - Rayon parametrique P(t) = origin + t * direction.
// Precalcule inv_dir et dir_sign a la construction pour accelerer
// le test slab AABB (Williams et al. 2005) : on evite 6 divisions par
// test d'intersection boite, critique dans la traversee BVH.

#ifndef RT_MATH_RAY_CUH
#define RT_MATH_RAY_CUH

#include "math/vec3.cuh"

class Ray {
public:
    Point3 orig;
    Vec3 dir;
    Vec3 inv_dir;       // 1/direction, precalcule (Williams et al.)
    int dir_sign[3];    // 0 si positif, 1 si negatif : indexation directe min/max AABB

    RT_HOSTDEV Ray() {}
    RT_HOSTDEV Ray(const Point3& origin, const Vec3& direction)
        : orig(origin), dir(direction)
    {
        // Precalcul une seule fois par rayon, reutilise a chaque noeud BVH
        inv_dir = Vec3(1.0 / direction.x(), 1.0 / direction.y(), 1.0 / direction.z());
        dir_sign[0] = (inv_dir.x() < 0) ? 1 : 0;
        dir_sign[1] = (inv_dir.y() < 0) ? 1 : 0;
        dir_sign[2] = (inv_dir.z() < 0) ? 1 : 0;
    }

    RT_HOSTDEV const Point3& origin() const { return orig; }
    RT_HOSTDEV const Vec3& direction() const { return dir; }

    // Point le long du rayon a la distance t
    RT_HOSTDEV Point3 at(double t) const {
        return orig + t * dir;
    }
};

#endif // RT_MATH_RAY_CUH
```

7.29 src/math/vec3.cuh File Reference

```
#include <cmath>
#include <iostream>
```

Classes

- class Vec3

Macros

- #define RT_MATH_VEC3_CUH
- #define RT_HOSTDEV

Typedefs

- using Point3 = Vec3
- using Color = Vec3

Functions

- RT_HOSTDEV Vec3 operator+ (const Vec3 &u, const Vec3 &v)
- RT_HOSTDEV Vec3 operator- (const Vec3 &u, const Vec3 &v)
- RT_HOSTDEV Vec3 operator∗ (const Vec3 &u, const Vec3 &v)
- RT_HOSTDEV Vec3 operator∗ (double t, const Vec3 &v)
- RT_HOSTDEV Vec3 operator∗ (const Vec3 &v, double t)
- RT_HOSTDEV Vec3 operator/ (const Vec3 &v, double t)
- RT_HOSTDEV double dot (const Vec3 &u, const Vec3 &v)
- RT_HOSTDEV Vec3 cross (const Vec3 &u, const Vec3 &v)
- RT_HOSTDEV Vec3 unit_vector (const Vec3 &v)
- RT_HOSTDEV Vec3 reflect (const Vec3 &v, const Vec3 &n)
- RT_HOSTDEV Vec3 refract (const Vec3 &uv, const Vec3 &n, double etai_over_etat)
- std::ostream & operator<< (std::ostream &out, const Vec3 &v)

7.29.1 Macro Definition Documentation

7.29.1.1 RT_HOSTDEV

```
#define RT_HOSTDEV
```

Definition at line 16 of file vec3.cuh.

7.29.1.2 RT_MATH_VEC3_CUH

```
#define RT_MATH_VEC3_CUH
```

Definition at line 6 of file vec3.cuh.

7.29.2 Typedef Documentation

7.29.2.1 Color

```
using Color = Vec3
```

Definition at line 72 of file vec3.cuh.

7.29.2.2 Point3

```
using Point3 = Vec3
```

Definition at line 71 of file vec3.cuh.

7.29.3 Function Documentation

7.29.3.1 cross()

```
RT_HOSTDEV Vec3 cross (
            const Vec3 & u,
            const Vec3 & v)  [inline]
```

Definition at line 108 of file vec3.cuh.

7.29.3.2 dot()

```
RT_HOSTDEV double dot (
            const Vec3 & u,
            const Vec3 & v)  [inline]
```

Definition at line 104 of file vec3.cuh.

7.29.3.3 operator∗() [1/3]

```
RT_HOSTDEV Vec3 operator* (
            const Vec3 & u,
            const Vec3 & v)  [inline]
```

Definition at line 86 of file vec3.cuh.

7.29.3.4 operator∗() [2/3]

```
RT_HOSTDEV Vec3 operator* (
            const Vec3 & v,
            double t)  [inline]
```

Definition at line 94 of file vec3.cuh.

7.29.3.5 operator∗() [3/3]

```
RT_HOSTDEV Vec3 operator* (
            double t,
            const Vec3 & v)  [inline]
```

Definition at line 90 of file vec3.cuh.

7.29.3.6 operator+()

```
RT_HOSTDEV Vec3 operator+ (
            const Vec3 & u,
            const Vec3 & v)  [inline]
```

Definition at line 76 of file vec3.cuh.

7.29.3.7 operator-()

```
RT_HOSTDEV Vec3 operator- (
            const Vec3 & u,
            const Vec3 & v)  [inline]
```

Definition at line 80 of file vec3.cuh.

7.29.3.8 operator/()

```
RT_HOSTDEV Vec3 operator/ (
            const Vec3 & v,
            double t)  [inline]
```

Definition at line 98 of file vec3.cuh.

7.29.3.9 operator<<()

```
std::ostream & operator<< (
            std::ostream & out,
            const Vec3 & v)  [inline]
```

Definition at line 135 of file vec3.cuh.

7.29.3.10 reflect()

```
RT_HOSTDEV Vec3 reflect (
            const Vec3 & v,
            const Vec3 & n)  [inline]
```

Definition at line 121 of file vec3.cuh.

7.29.3.11 refract()

```
RT_HOSTDEV Vec3 refract (
            const Vec3 & uv,
            const Vec3 & n,
            double etai_over_etat)  [inline]
```

Definition at line 128 of file vec3.cuh.

7.29.3.12 unit_vector()

```
RT_HOSTDEV Vec3 unit_vector (
            const Vec3 & v)  [inline]
```

Definition at line 116 of file vec3.cuh.

7.30 vec3.cuh

Go to the documentation of this file.

```
// vec3.cuh - Vecteur 3D double precision, brique de base du path tracer.
// Header-only : force l'inlining partout (hot path intersections + shading).
// RT_HOSTDEV permet la compilation croisee CPU (MSVC) / GPU (nvcc).

#ifndef RT_MATH_VEC3_CUH
#define RT_MATH_VEC3_CUH

#include <cmath>
#include <iostream>

// Macro de portabilite CPU/GPU : declenche __host__ __device__ uniquement
// quand nvcc compile le fichier, transparent pour MSVC.
#ifdef __CUDACC__
#define RT_HOSTDEV __host__ __device__
#else
#define RT_HOSTDEV
#endif

class Vec3 {
public:
    double e[3]; // stockage brut, pas de padding

    RT_HOSTDEV Vec3() : e{0.0, 0.0, 0.0} {}
    RT_HOSTDEV Vec3(double x, double y, double z) : e{x, y, z} {}

    RT_HOSTDEV double x() const { return e[0]; }
    RT_HOSTDEV double y() const { return e[1]; }
    RT_HOSTDEV double z() const { return e[2]; }

    RT_HOSTDEV Vec3 operator-() const { return Vec3(-e[0], -e[1], -e[2]); }
    RT_HOSTDEV double operator[](int i) const { return e[i]; }
    RT_HOSTDEV double& operator[](int i) { return e[i]; }

    RT_HOSTDEV Vec3& operator+=(const Vec3& v) {
        e[0] += v.e[0]; e[1] += v.e[1]; e[2] += v.e[2];
        return *this;
    }

    RT_HOSTDEV Vec3& operator-=(const Vec3& v) {
        e[0] -= v.e[0]; e[1] -= v.e[1]; e[2] -= v.e[2];
        return *this;
    }

    RT_HOSTDEV Vec3& operator*=(double t) {
        e[0] *= t; e[1] *= t; e[2] *= t;
        return *this;
    }

    // Multiplication par l'inverse : evite 3 divisions
    RT_HOSTDEV Vec3& operator/=(double t) {
        return *this *= (1.0 / t);
    }

    RT_HOSTDEV double length_squared() const {
        return e[0]*e[0] + e[1]*e[1] + e[2]*e[2];
    }

    RT_HOSTDEV double length() const {
        return std::sqrt(length_squared());
    }

    // Detection de vecteur quasi-nul apres scatter :
    // evite les NaN en normalisant un vecteur degenere
    RT_HOSTDEV bool near_zero() const {
        constexpr double s = 1e-8;
        return (std::fabs(e[0]) < s) && (std::fabs(e[1]) < s) && (std::fabs(e[2]) < s);
    }
};

// Alias semantiques : meme type sous-jacent, intention differente
using Point3 = Vec3;
using Color = Vec3;

// --- Operateurs arithmetiques ---

inline RT_HOSTDEV Vec3 operator+(const Vec3& u, const Vec3& v) {
    return Vec3(u.e[0]+v.e[0], u.e[1]+v.e[1], u.e[2]+v.e[2]);
}

inline RT_HOSTDEV Vec3 operator-(const Vec3& u, const Vec3& v) {
    return Vec3(u.e[0]-v.e[0], u.e[1]-v.e[1], u.e[2]-v.e[2]);
}
```

```

// Produit composante par composante (Hadamard) :
// sert a moduler la couleur par l'albedo du materiau
inline RT_HOSTDEV Vec3 operator*(const Vec3& u, const Vec3& v) {
    return Vec3(u.e[0]*v.e[0], u.e[1]*v.e[1], u.e[2]*v.e[2]);
}

inline RT_HOSTDEV Vec3 operator*(double t, const Vec3& v) {
    return Vec3(t*v.e[0], t*v.e[1], t*v.e[2]);
}

inline RT_HOSTDEV Vec3 operator*(const Vec3& v, double t) {
    return t * v;
}

inline RT_HOSTDEV Vec3 operator/(const Vec3& v, double t) {
    return (1.0/t) * v;
}

// --- Fonctions geometriques ---

inline RT_HOSTDEV double dot(const Vec3& u, const Vec3& v) {
    return u.e[0]*v.e[0] + u.e[1]*v.e[1] + u.e[2]*v.e[2];
}

inline RT_HOSTDEV Vec3 cross(const Vec3& u, const Vec3& v) {
    return Vec3(
        u.e[1]*v.e[2] - u.e[2]*v.e[1],
        u.e[2]*v.e[0] - u.e[0]*v.e[2],
        u.e[0]*v.e[1] - u.e[1]*v.e[0]
    );
}

inline RT_HOSTDEV Vec3 unit_vector(const Vec3& v) {
    return v / v.length();
}

// Reflexion speculaire : v incident, n normale unitaire
inline RT_HOSTDEV Vec3 reflect(const Vec3& v, const Vec3& n) {
    return v - 2.0 * dot(v, n) * n;
}

// Refraction via loi de Snell-Descartes.
// uv doit etre unitaire. etai_over_etat = n1/n2 (indices de refraction).
// Decomposition en composante perpendiculaire + parallele a la normale.
inline RT_HOSTDEV Vec3 refract(const Vec3& uv, const Vec3& n, double etai_over_etat) {
    double cos_theta = std::fmin(dot(-uv, n), 1.0);
    Vec3 r_out_perp = etai_over_etat * (uv + cos_theta * n);
    Vec3 r_out_parallel = -std::sqrt(std::fabs(1.0 - r_out_perp.length_squared())) * n;
    return r_out_perp + r_out_parallel;
}

inline std::ostream& operator«(std::ostream& out, const Vec3& v) {
    return out « v.e[0] « ' ' « v.e[1] « ' ' « v.e[2];
}

#endif // RT_MATH_VEC3_CUH
```

7.31 src/scene/bvh_builder.cpp File Reference

```
#include "scene/bvh_builder.h"
#include "scene/primitives.cuh"
#include <algorithm>
#include <limits>
```

7.32 bvh_builder.cpp

Go to the documentation of this file.

```
#include "scene/bvh_builder.h"
#include "scene/primitives.cuh"
#include <algorithm>
#include <limits>
```

```

// ============================================================
// BVHAccel - Optimized flat BVH
// SAH construction, iterative traversal, front-to-back order
// ============================================================

BVHAccel::BVHAccel(const HittableList& list) {
    auto prims = list.objects();
    if (prims.empty()) return;

    // Build primitive info
    std::vector<BuildPrimInfo> prim_info(prims.size());
    for (size_t i = 0; i < prims.size(); ++i) {
        prim_info[i].prim_index = i;
        prim_info[i].bbox = prims[i]->bounding_box();
        prim_info[i].centroid = (prim_info[i].bbox.min_point() + prim_info[i].bbox.max_point()) * 0.5;
    }

    // Build tree (using new for stable pointers)
    int total_nodes = 0;
    BuildNode* root = recursive_build(prim_info, 0, static_cast<int>(prims.size()),
                                      prims, total_nodes);

    // Flatten to contiguous array
    nodes_.resize(total_nodes);
    int offset = 0;
    flatten_tree(root, offset);

    // Free build tree
    delete_tree(root);
}

BVHAccel::BuildNode* BVHAccel::recursive_build(
    std::vector<BuildPrimInfo>& prim_info,
    int start, int end,
    std::vector<std::shared_ptr<Hittable»& src_prims,
    int& total_nodes)
{
    BuildNode* node = new BuildNode();
    total_nodes++;

    // Compute bounds of all primitives in range
    AABB bounds;
    for (int i = start; i < end; ++i)
        bounds = surrounding_box(bounds, prim_info[i].bbox);

    int n_prims = end - start;

    // Small leaf
    if (n_prims <= 2) {
        int first_offset = static_cast<int>(ordered_prims_.size());
        for (int i = start; i < end; ++i)
            ordered_prims_.push_back(src_prims[prim_info[i].prim_index]);
        node->bbox = bounds;
        node->first_prim_offset = first_offset;
        node->num_primitives = n_prims;
        return node;
    }

    // Compute centroid bounds
    AABB centroid_bounds;
    for (int i = start; i < end; ++i) {
        Point3 c = prim_info[i].centroid;
        centroid_bounds = surrounding_box(centroid_bounds, AABB(c, c));
    }

    int axis = centroid_bounds.longest_axis();
    double axis_extent = centroid_bounds.axis_interval(axis).size();

    // Degenerate: all centroids at same position
    if (axis_extent < 1e-8) {
        int first_offset = static_cast<int>(ordered_prims_.size());
        for (int i = start; i < end; ++i)
            ordered_prims_.push_back(src_prims[prim_info[i].prim_index]);
        node->bbox = bounds;
        node->first_prim_offset = first_offset;
        node->num_primitives = n_prims;
        return node;
    }

    // SAH bucket evaluation
    constexpr int N_BUCKETS = 12;
    struct Bucket { int count = 0; AABB bounds; };
    Bucket buckets[N_BUCKETS];

    double cmin = centroid_bounds.axis_interval(axis).min;
    double cmax = centroid_bounds.axis_interval(axis).max;
```

```

    for (int i = start; i < end; ++i) {
        int b = static_cast<int>(N_BUCKETS *
            ((prim_info[i].centroid[axis] - cmin) / (cmax - cmin)));
        if (b >= N_BUCKETS) b = N_BUCKETS - 1;
        buckets[b].count++;
        buckets[b].bounds = surrounding_box(buckets[b].bounds, prim_info[i].bbox);
    }

    // Evaluate cost for each split position
    constexpr double T_TRAV = 0.125;
    constexpr double T_ISECT = 1.0;
    double parent_area = bounds.surface_area();
    double best_cost = std::numeric_limits<double>::infinity();
    int best_split = -1;

    // Forward sweep: accumulate left side
    double cost_left_area[N_BUCKETS - 1];
    int cost_left_count[N_BUCKETS - 1];
    {
        AABB accum;
        int count = 0;
        for (int i = 0; i < N_BUCKETS - 1; ++i) {
            accum = surrounding_box(accum, buckets[i].bounds);
            count += buckets[i].count;
            cost_left_area[i] = accum.surface_area();
            cost_left_count[i] = count;
        }
    }
    // Backward sweep: accumulate right side and find best
    {
        AABB accum;
        int count = 0;
        for (int i = N_BUCKETS - 1; i >= 1; --i) {
            accum = surrounding_box(accum, buckets[i].bounds);
            count += buckets[i].count;
            double cost = T_TRAV + T_ISECT *
                (cost_left_count[i-1] * cost_left_area[i-1] +
                 count * accum.surface_area()) / parent_area;
            if (cost < best_cost) {
                best_cost = cost;
                best_split = i;
            }
        }
    }

    // Compare with leaf cost
    double leaf_cost = T_ISECT * n_prims;
    if (best_cost >= leaf_cost && n_prims <= 16) {
        // Leaf is cheaper
        int first_offset = static_cast<int>(ordered_prims_.size());
        for (int i = start; i < end; ++i)
            ordered_prims_.push_back(src_prims[prim_info[i].prim_index]);
        node->bbox = bounds;
        node->first_prim_offset = first_offset;
        node->num_primitives = n_prims;
        return node;
    }

    // Partition primitives at best split
    auto mid_it = std::partition(
        prim_info.begin() + start, prim_info.begin() + end,
        [&](const BuildPrimInfo& pi) {
            int b = static_cast<int>(N_BUCKETS *
                ((pi.centroid[axis] - cmin) / (cmax - cmin)));
            if (b >= N_BUCKETS) b = N_BUCKETS - 1;
            return b < best_split;
        });
    int mid = static_cast<int>(std::distance(prim_info.begin(), mid_it));

    if (mid == start || mid == end)
        mid = start + n_prims / 2;

    // Recurse
    node->children[0] = recursive_build(prim_info, start, mid, src_prims, total_nodes);
    node->children[1] = recursive_build(prim_info, mid, end, src_prims, total_nodes);
    node->bbox = surrounding_box(node->children[0]->bbox, node->children[1]->bbox);
    node->split_axis = axis;
    node->num_primitives = 0;
    return node;
}

int BVHAccel::flatten_tree(BuildNode* node, int& offset) {
    FlatBVHNode& flat = nodes_[offset];
    flat.bbox = node->bbox;
    int my_offset = offset++;

```

```
    if (node->num_primitives > 0) {
        // Leaf
        flat.primitives_offset = static_cast<uint32_t>(node->first_prim_offset);
        flat.num_primitives = static_cast<uint16_t>(node->num_primitives);
    } else {
        // Interior
        flat.split_axis = static_cast<uint8_t>(node->split_axis);
        flat.num_primitives = 0;
        // Left child is immediately next (offset)
        flatten_tree(node->children[0], offset);
        // Right child offset stored in node
        flat.second_child_offset = static_cast<uint32_t>(
            flatten_tree(node->children[1], offset));
    }
    return my_offset;
}

void BVHAccel::delete_tree(BuildNode* node) {
    if (!node) return;
    delete_tree(node->children[0]);
    delete_tree(node->children[1]);
    delete node;
}

bool BVHAccel::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    if (nodes_.empty()) return false;

    bool hit_anything = false;
    double closest = ray_t.max;

    // Iterative traversal with explicit stack
    constexpr int MAX_STACK = 64;
    int stack[MAX_STACK];
    int stack_ptr = 0;
    stack[stack_ptr++] = 0;

    while (stack_ptr > 0) {
        int node_idx = stack[--stack_ptr];
        const FlatBVHNode& node = nodes_[node_idx];

        if (!node.bbox.hit(r, ray_t.min, closest))
            continue;

        if (node.num_primitives > 0) {
            // Leaf: test all primitives
            for (uint16_t i = 0; i < node.num_primitives; ++i) {
                if (ordered_prims_[node.primitives_offset + i]->hit(
                        r, Interval(ray_t.min, closest), rec)) {
                    hit_anything = true;
                    closest = rec.t;
                }
            }
        } else {
            // Interior: push children in front-to-back order
            // Left child = node_idx + 1, right child = second_child_offset
            int near_child = node_idx + 1;
            int far_child = node.second_child_offset;

            // If ray direction is negative along split axis, swap
            if (r.dir_sign[node.split_axis]) {
                int tmp = near_child; near_child = far_child; far_child = tmp;
            }

            // Push far first (popped last = traversed last)
            stack[stack_ptr++] = far_child;
            stack[stack_ptr++] = near_child;
        }
    }
    return hit_anything;
}

AABB BVHAccel::bounding_box() const {
    if (nodes_.empty()) return AABB::empty;
    return nodes_[0].bbox;
}

// ============================================================
// Legacy BVHNode (for test backward compatibility)
// ============================================================

BVHNode::BVHNode(const HittableList& list) {
    auto objects = list.objects();
    *this = BVHNode(objects, 0, objects.size());
}

BVHNode::BVHNode(std::vector<std::shared_ptr<Hittable»& objects, size_t start, size_t end) {
    AABB span_bbox;
```

```
    for (size_t i = start; i < end; ++i)
        span_bbox = surrounding_box(span_bbox, objects[i]->bounding_box());
    bbox_ = span_bbox;

    size_t object_span = end - start;

    if (object_span == 1) {
        left_ = right_ = objects[start];
    } else if (object_span == 2) {
        left_ = objects[start];
        right_ = objects[start + 1];
    } else {
        int axis = bbox_.longest_axis();
        split_axis_ = axis;

        std::sort(objects.begin() + start, objects.begin() + end,
            [axis](const std::shared_ptr<Hittable>& a, const std::shared_ptr<Hittable>& b) {
                return a->bounding_box().axis_interval(axis).min
                     < b->bounding_box().axis_interval(axis).min;
            });

        size_t mid = start + object_span / 2;
        left_ = std::make_shared<BVHNode>(objects, start, mid);
        right_ = std::make_shared<BVHNode>(objects, mid, end);
    }
}

bool BVHNode::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    if (!bbox_.hit(r, ray_t))
        return false;

    // Front-to-back based on ray direction
    Hittable* first = (r.dir_sign[split_axis_] == 0) ? left_.get() : right_.get();
    Hittable* second = (r.dir_sign[split_axis_] == 0) ? right_.get() : left_.get();

    bool hit_near = first->hit(r, ray_t, rec);
    bool hit_far = second->hit(r,
        Interval(ray_t.min, hit_near ? rec.t : ray_t.max), rec);

    return hit_near || hit_far;
}

AABB BVHNode::bounding_box() const {
    return bbox_;
}
```

7.33 src/scene/bvh_builder.h File Reference

```
#include "scene/bvh_node.cuh"
#include <vector>
#include <memory>
```

Classes

- class BVHAccel
- struct BVHAccel::BuildPrimInfo
- struct BVHAccel::BuildNode

7.34 bvh_builder.h

Go to the documentation of this file.

```
#ifndef RT_SCENE_BVH_BUILDER_H
#define RT_SCENE_BVH_BUILDER_H

#include "scene/bvh_node.cuh"
#include <vector>
#include <memory>
```

```

class HittableList;

// Optimized flat BVH: SAH construction, iterative traversal, front-to-back order
class BVHAccel : public Hittable {
public:
    BVHAccel(const HittableList& list);

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

private:
    std::vector<FlatBVHNode> nodes_;
    std::vector<std::shared_ptr<Hittable>> ordered_prims_;

    struct BuildPrimInfo {
        size_t prim_index;
        AABB bbox;
        Point3 centroid;
    };

    struct BuildNode {
        AABB bbox;
        BuildNode* children[2] = {nullptr, nullptr};
        int split_axis = 0;
        int first_prim_offset = 0;
        int num_primitives = 0;
    };

    BuildNode* recursive_build(
        std::vector<BuildPrimInfo>& prim_info,
        int start, int end,
        std::vector<std::shared_ptr<Hittable>>& src_prims,
        int& total_nodes);

    int flatten_tree(BuildNode* node, int& offset);
    void delete_tree(BuildNode* node);
};

#endif // RT_SCENE_BVH_BUILDER_H
```

7.35 src/scene/bvh_node.cuh File Reference

```
#include "math/aabb.cuh"
#include "scene/primitives.cuh"
#include <cstdint>
#include <memory>
#include <vector>
```

Classes

- struct FlatBVHNode
- class BVHNode

Macros

- #define RT_SCENE_BVH_NODE_CUH

7.35.1 Macro Definition Documentation

7.35.1.1 RT_SCENE_BVH_NODE_CUH

```
#define RT_SCENE_BVH_NODE_CUH
```

Definition at line 2 of file bvh_node.cuh.

7.36 bvh_node.cuh

Go to the documentation of this file.

```
#ifndef RT_SCENE_BVH_NODE_CUH
#define RT_SCENE_BVH_NODE_CUH

#include "math/aabb.cuh"
#include "scene/primitives.cuh"
#include <cstdint>
#include <memory>
#include <vector>

// Compact flat BVH node for cache-friendly iterative traversal
struct FlatBVHNode {
    AABB bbox;
    union {
        uint32_t primitives_offset;   // leaf: index into ordered primitives
        uint32_t second_child_offset; // interior: index of right child (left = this+1)
    };
    uint16_t num_primitives; // 0 = interior node
    uint8_t split_axis;      // for front-to-back traversal
    uint8_t pad;
};

// Legacy recursive BVH (kept for test compatibility)
class BVHNode : public Hittable {
public:
    BVHNode(std::vector<std::shared_ptr<Hittable»& objects, size_t start, size_t end);
    BVHNode(const class HittableList& list);

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

private:
    std::shared_ptr<Hittable> left_;
    std::shared_ptr<Hittable> right_;
    AABB bbox_;
    int split_axis_ = 0;
};

#endif // RT_SCENE_BVH_NODE_CUH
```

7.37 src/scene/camera.cpp File Reference

```
#include "scene/camera.cuh"
#include "math/random.cuh"
#include <cmath>
#include <algorithm>
```

7.38 camera.cpp

Go to the documentation of this file.

```
#include "scene/camera.cuh"
#include "math/random.cuh"
#include <cmath>
#include <algorithm>

void Camera::initialize() {
    image_height_ = std::max(1, static_cast<int>(image_width / aspect_ratio));

    center_ = lookfrom;

    // Viewport dimensions
    double theta = vfov * 3.14159265358979323846 / 180.0;
    double h = std::tan(theta / 2.0);
    double viewport_height = 2.0 * h * focus_dist;
    double viewport_width = viewport_height * (static_cast<double>(image_width) / image_height_);

    // Camera coordinate frame
```

```
    w_ = unit_vector(lookfrom - lookat);
    u_ = unit_vector(cross(vup, w_));
    v_ = cross(w_, u_);

    // Vectors across the horizontal and down the vertical viewport edges
    Vec3 viewport_u = viewport_width * u_;
    Vec3 viewport_v = viewport_height * (-v_);

    // Pixel deltas
    pixel_delta_u_ = viewport_u / static_cast<double>(image_width);
    pixel_delta_v_ = viewport_v / static_cast<double>(image_height_);

    // Upper left pixel location
    Point3 viewport_upper_left = center_ - focus_dist * w_
                                 - viewport_u / 2.0
                                 - viewport_v / 2.0;
    pixel100_loc_ = viewport_upper_left + 0.5 * (pixel_delta_u_ + pixel_delta_v_);

    // Defocus disk basis vectors
    double defocus_radius = focus_dist * std::tan((defocus_angle / 2.0) * 3.14159265358979323846 /
180.0);
    defocus_disk_u_ = u_ * defocus_radius;
    defocus_disk_v_ = v_ * defocus_radius;
}

Ray Camera::get_ray(int i, int j) const {
    Vec3 offset = sample_square();
    Point3 pixel_sample = pixel100_loc_
                        + ((i + offset.x()) * pixel_delta_u_)
                        + ((j + offset.y()) * pixel_delta_v_);

    Point3 ray_origin = (defocus_angle <= 0.0) ? center_ : defocus_disk_sample();
    Vec3 ray_direction = pixel_sample - ray_origin;

    return Ray(ray_origin, ray_direction);
}

Vec3 Camera::sample_square() const {
    return Vec3(random_double() - 0.5, random_double() - 0.5, 0.0);
}

Point3 Camera::defocus_disk_sample() const {
    Vec3 p = random_in_unit_disk();
    return center_ + (p[0] * defocus_disk_u_) + (p[1] * defocus_disk_v_);
}
```

7.39 src/scene/camera.cuh File Reference

```
#include "math/vec3.cuh"
#include "math/ray.cuh"
```

Classes

- class Camera

Macros

- #define RT_SCENE_CAMERA_CUH

7.39.1 Macro Definition Documentation

7.39.1.1 RT_SCENE_CAMERA_CUH

```
#define RT_SCENE_CAMERA_CUH
```

Definition at line 2 of file camera.cuh.

7.40 camera.cuh

Go to the documentation of this file.

```
#ifndef RT_SCENE_CAMERA_CUH
#define RT_SCENE_CAMERA_CUH

#include "math/vec3.cuh"
#include "math/ray.cuh"

class Camera {
public:
    // Configuration (set before calling initialize())
    double aspect_ratio = 16.0 / 9.0;
    int image_width = 400;
    int samples_per_pixel = 100;
    int max_depth = 50;

    double vfov = 90.0;        // vertical field of view in degrees
    Point3 lookfrom = Point3(0, 0, 0);
    Point3 lookat = Point3(0, 0, -1);
    Vec3 vup = Vec3(0, 1, 0);

    double defocus_angle = 0.0;  // variation angle of rays through each pixel
    double focus_dist = 10.0;    // distance from lookfrom to the plane of perfect focus

    // Call once after setting configuration
    void initialize();

    // Get image dimensions
    int get_image_width() const { return image_width; }
    int get_image_height() const { return image_height_; }

    // Generate a ray for pixel (i, j) with random sub-pixel offset
    Ray get_ray(int i, int j) const;

private:
    int image_height_;
    Point3 center_;
    Point3 pixel00_loc_;    // location of pixel (0,0)
    Vec3 pixel_delta_u_;    // offset to pixel to the right
    Vec3 pixel_delta_v_;    // offset to pixel below
    Vec3 u_, v_, w_;        // camera basis vectors
    Vec3 defocus_disk_u_;
    Vec3 defocus_disk_v_;

    Vec3 sample_square() const;
    Point3 defocus_disk_sample() const;
};

#endif // RT_SCENE_CAMERA_CUH
```

7.41 src/scene/material.cpp File Reference

```
#include "scene/material.cuh"
#include "scene/primitives.cuh"
#include "math/random.cuh"
#include <cmath>
#include <algorithm>
```

7.42 material.cpp

Go to the documentation of this file.

```
#include "scene/material.cuh"
#include "scene/primitives.cuh"
#include "math/random.cuh"
#include <cmath>
#include <algorithm>

```

```
// ============================================================
// Lambertian
// ============================================================

Lambertian::Lambertian(const Color& albedo) : albedo_(albedo) {
    type_ = Type::Lambertian;
}

bool Lambertian::scatter(const Ray& /*r_in*/, const HitRecord& rec,
                         Color& attenuation, Ray& scattered) const {
    Vec3 scatter_direction = rec.normal + random_unit_vector();

    // Catch degenerate scatter direction
    if (scatter_direction.near_zero())
        scatter_direction = rec.normal;

    scattered = Ray(rec.p, scatter_direction);
    attenuation = albedo_;
    return true;
}

// ============================================================
// Metal
// ============================================================

Metal::Metal(const Color& albedo, double fuzz)
    : albedo_(albedo), fuzz_(std::min(fuzz, 1.0)) {
    type_ = Type::Metal;
}

bool Metal::scatter(const Ray& r_in, const HitRecord& rec,
                    Color& attenuation, Ray& scattered) const {
    Vec3 reflected = reflect(unit_vector(r_in.direction()), rec.normal);
    reflected = reflected + fuzz_ * random_in_unit_sphere();
    scattered = Ray(rec.p, reflected);
    attenuation = albedo_;
    return (dot(scattered.direction(), rec.normal) > 0.0);
}

// ============================================================
// Dielectric
// ============================================================

Dielectric::Dielectric(double refraction_index)
    : refraction_index_(refraction_index) {
    type_ = Type::Dielectric;
}

bool Dielectric::scatter(const Ray& r_in, const HitRecord& rec,
                         Color& attenuation, Ray& scattered) const {
    attenuation = Color(1.0, 1.0, 1.0);
    double ri = rec.front_face ? (1.0 / refraction_index_) : refraction_index_;

    Vec3 unit_dir = unit_vector(r_in.direction());
    double cos_theta = std::fmin(dot(-unit_dir, rec.normal), 1.0);
    double sin_theta = std::sqrt(1.0 - cos_theta * cos_theta);

    bool cannot_refract = ri * sin_theta > 1.0;
    Vec3 direction;

    if (cannot_refract || reflectance(cos_theta, ri) > random_double())
        direction = reflect(unit_dir, rec.normal);
    else
        direction = refract(unit_dir, rec.normal, ri);

    scattered = Ray(rec.p, direction);
    return true;
}

double Dielectric::reflectance(double cosine, double refraction_index) {
    // Schlick's approximation
    double r0 = (1.0 - refraction_index) / (1.0 + refraction_index);
    r0 = r0 * r0;
    double omc = 1.0 - cosine;
    double omc2 = omc * omc;
    return r0 + (1.0 - r0) * omc2 * omc2 * omc;
}

// ============================================================
// Checker (damier procedural)
// ============================================================

Checker::Checker(const Color& color1, const Color& color2, double scale)
    : color1_(color1), color2_(color2), scale_(scale) {
    type_ = Type::Checker;
}

```

```
bool Checker::scatter(const Ray& /*r_in*/, const HitRecord& rec,
                      Color& attenuation, Ray& scattered) const {
    // Pattern damier base sur la position du hit
    double inv_scale = 1.0 / scale_;
    int ix = (int)std::floor(rec.p.x() * inv_scale);
    int iy = (int)std::floor(rec.p.y() * inv_scale);
    int iz = (int)std::floor(rec.p.z() * inv_scale);
    bool is_even = ((ix + iy + iz) % 2 == 0);

    attenuation = is_even ? color1_ : color2_;

    Vec3 scatter_direction = rec.normal + random_unit_vector();
    if (scatter_direction.near_zero()) scatter_direction = rec.normal;
    scattered = Ray(rec.p, scatter_direction);
    return true;
}

// ============================================================
// Emissive (area light)
// ============================================================

Emissive::Emissive(const Color& emit_color, double intensity)
    : emit_(emit_color), intensity_(intensity) {
    type_ = Type::Emissive;
}

bool Emissive::scatter(const Ray& /*r_in*/, const HitRecord& /*rec*/,
                       Color& /*attenuation*/, Ray& /*scattered*/) const {
    return false; // emissive ne scatter pas, juste emet
}
```

7.43 src/scene/material.cuh File Reference

```
#include "math/vec3.cuh"
#include "math/ray.cuh"
#include <cstdint>
```

Classes

- class Material
- class Lambertian
- class Metal
- class Dielectric
- class Checker
- class Emissive

Macros

- #define RT_SCENE_MATERIAL_CUH

7.43.1 Macro Definition Documentation

7.43.1.1 RT_SCENE_MATERIAL_CUH

```
#define RT_SCENE_MATERIAL_CUH
```

Definition at line 2 of file material.cuh.

7.44 material.cuh

Go to the documentation of this file.

```
#ifndef RT_SCENE_MATERIAL_CUH
#define RT_SCENE_MATERIAL_CUH

#include "math/vec3.cuh"
#include "math/ray.cuh"
#include <cstdint>

struct HitRecord; // forward declaration

// --- Base Material class ---

class Material {
public:
    // Type tag pour dispatch rapide sans vtable
    enum class Type : uint8_t { Lambertian, Metal, Dielectric, Checker, Emissive, Unknown };

    virtual ~Material() = default;

    virtual bool scatter(const Ray& r_in, const HitRecord& rec,
                         Color& attenuation, Ray& scattered) const = 0;

    Type type() const { return type_; }

protected:
    Type type_ = Type::Unknown;
};

// --- Lambertian ---

class Lambertian : public Material {
public:
    explicit Lambertian(const Color& albedo);

    bool scatter(const Ray& r_in, const HitRecord& rec,
                 Color& attenuation, Ray& scattered) const override;

    const Color& albedo() const { return albedo_; }

private:
    Color albedo_;
};

// --- Metal ---

class Metal : public Material {
public:
    Metal(const Color& albedo, double fuzz);

    bool scatter(const Ray& r_in, const HitRecord& rec,
                 Color& attenuation, Ray& scattered) const override;

    const Color& albedo() const { return albedo_; }
    double fuzz() const { return fuzz_; }

private:
    Color albedo_;
    double fuzz_;
};

// --- Dielectric ---

class Dielectric : public Material {
public:
    explicit Dielectric(double refraction_index);

    bool scatter(const Ray& r_in, const HitRecord& rec,
                 Color& attenuation, Ray& scattered) const override;

    double refraction_index() const { return refraction_index_; }

    // Schlick's approximation for reflectance (public for devirtualized scatter)
    static double reflectance(double cosine, double refraction_index);

private:
    double refraction_index_;
};

// --- Checker (damier procedural) ---

class Checker : public Material {
public:
    Checker(const Color& color1, const Color& color2, double scale = 1.0);
```

```

    bool scatter(const Ray& r_in, const HitRecord& rec,
                 Color& attenuation, Ray& scattered) const override;

    const Color& color1() const { return color1_; }
    const Color& color2() const { return color2_; }
    double scale() const { return scale_; }

private:
    Color color1_, color2_;
    double scale_;
};

// --- Emissive (area light) ---

class Emissive : public Material {
public:
    Emissive(const Color& emit_color, double intensity = 1.0);

    bool scatter(const Ray& r_in, const HitRecord& rec,
                 Color& attenuation, Ray& scattered) const override;

    Color emitted() const { return emit_ * intensity_; }
    const Color& emit_color() const { return emit_; }
    double intensity() const { return intensity_; }

private:
    Color emit_;
    double intensity_;
};

#endif // RT_SCENE_MATERIAL_CUH
```

7.45 src/scene/primitives.cpp File Reference

```
#include "scene/primitives.cuh"
#include <cmath>
```

7.46 primitives.cpp

Go to the documentation of this file.

```
#include "scene/primitives.cuh"
#include <cmath>

// ============================================================
// Sphere
// ============================================================

Sphere::Sphere(const Point3& center, double radius, const Material* material)
    : center_(center), radius_(radius), material_(material) {}

bool Sphere::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    Vec3 oc = r.origin() - center_;
    double a = r.direction().length_squared();
    double half_b = dot(oc, r.direction());
    double c = oc.length_squared() - radius_ * radius_;

    double discriminant = half_b * half_b - a * c;
    if (discriminant < 0.0)
        return false;

    double sqrtd = std::sqrt(discriminant);

    // Find nearest root in the acceptable range
    double inv_a = 1.0 / a;
    double root = (-half_b - sqrtd) * inv_a;
    if (!ray_t.surrounds(root)) {
        root = (-half_b + sqrtd) * inv_a;
        if (!ray_t.surrounds(root))
            return false;
    }

    rec.t = root;
```

```
    rec.p = r.at(rec.t);
    Vec3 outward_normal = (rec.p - center_) * (1.0 / radius_);
    rec.set_face_normal(r, outward_normal);
    rec.material = material_;

    return true;
}

AABB Sphere::bounding_box() const {
    Vec3 rvec(radius_, radius_, radius_);
    return AABB(center_ - rvec, center_ + rvec);
}

// ============================================================
// Plane
// ============================================================

Plane::Plane(const Point3& point, const Vec3& normal, const Material* material)
    : point_(point), normal_(unit_vector(normal)), material_(material) {}

bool Plane::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    double denom = dot(r.direction(), normal_);

    // Ray parallel to plane
    if (std::fabs(denom) < 1e-8)
        return false;

    double t = dot(point_ - r.origin(), normal_) / denom;

    if (!ray_t.surrounds(t))
        return false;

    rec.t = t;
    rec.p = r.at(t);
    rec.set_face_normal(r, normal_);
    rec.material = material_;

    return true;
}

AABB Plane::bounding_box() const {
    // Infinite plane: use a very large bounding box
    // Slightly thicken along normal to avoid zero-volume AABB
    constexpr double big = 1e4;
    constexpr double eps = 0.0001;
    Point3 lo = point_ - Vec3(big, big, big);
    Point3 hi = point_ + Vec3(big, big, big);
    // Ensure non-zero thickness along normal direction
    for (int i = 0; i < 3; ++i) {
        if (std::fabs(normal_[i]) > 0.99) {
            lo.e[i] = point_[i] - eps;
            hi.e[i] = point_[i] + eps;
        }
    }
    return AABB(lo, hi);
}

// ============================================================
// Cylinder (surface quadrique, axe Y)
// Equation : (px - cx)^2 + (pz - cz)^2 = r^2, base_.y <= py <= base_.y + height_
// ============================================================

Cylinder::Cylinder(const Point3& base, double radius, double height, const Material* material)
    : base_(base), radius_(radius), height_(height), material_(material) {}

bool Cylinder::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    // Intersection rayon-cylindre infini (axe Y)
    double ox = r.origin().x() - base_.x();
    double oz = r.origin().z() - base_.z();
    double dx = r.direction().x();
    double dz = r.direction().z();

    double a = dx*dx + dz*dz;
    double half_b = ox*dx + oz*dz;
    double c = ox*ox + oz*oz - radius_*radius_;

    double disc = half_b*half_b - a*c;
    if (disc < 0.0) return false;

    double sqrtd = std::sqrt(disc);
    double root = (-half_b - sqrtd) / a;

    // Tester les deux racines
    for (int i = 0; i < 2; i++) {
        if (i == 1) root = (-half_b + sqrtd) / a;
        if (!ray_t.surrounds(root)) continue;

```

```
        double y = r.origin().y() + root * r.direction().y();
        if (y < base_.y() || y > base_.y() + height_) continue;

        rec.t = root;
        rec.p = r.at(root);
        // Normale du cylindre : direction radiale (pas de composante Y)
        Vec3 outward_normal = Vec3(rec.p.x() - base_.x(), 0, rec.p.z() - base_.z()) / radius_;
        rec.set_face_normal(r, outward_normal);
        rec.material = material_;
        return true;
    }
    return false;
}

AABB Cylinder::bounding_box() const {
    return AABB(
        Point3(base_.x() - radius_, base_.y(), base_.z() - radius_),
        Point3(base_.x() + radius_, base_.y() + height_, base_.z() + radius_)
    );
}

// ============================================================
// Triangle (Moller-Trumbore)
// ============================================================

Triangle::Triangle(const Point3& v0, const Point3& v1, const Point3& v2, const Material* material)
    : v0_(v0), v1_(v1), v2_(v2), material_(material) {
    normal_ = unit_vector(cross(v1 - v0, v2 - v0));
}

bool Triangle::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    Vec3 edge1 = v1_ - v0_;
    Vec3 edge2 = v2_ - v0_;
    Vec3 h = cross(r.direction(), edge2);
    double a = dot(edge1, h);

    if (std::fabs(a) < 1e-8)
        return false; // rayon parallele au triangle

    double f = 1.0 / a;
    Vec3 s = r.origin() - v0_;
    double u = f * dot(s, h);
    if (u < 0.0 || u > 1.0)
        return false;

    Vec3 q = cross(s, edge1);
    double v = f * dot(r.direction(), q);
    if (v < 0.0 || u + v > 1.0)
        return false;

    double t = f * dot(edge2, q);
    if (!ray_t.surrounds(t))
        return false;

    rec.t = t;
    rec.p = r.at(t);
    rec.set_face_normal(r, normal_);
    rec.material = material_;
    return true;
}

AABB Triangle::bounding_box() const {
    Point3 lo(
        std::fmin(v0_.x(), std::fmin(v1_.x(), v2_.x())) - 0.0001,
        std::fmin(v0_.y(), std::fmin(v1_.y(), v2_.y())) - 0.0001,
        std::fmin(v0_.z(), std::fmin(v1_.z(), v2_.z())) - 0.0001
    );
    Point3 hi(
        std::fmax(v0_.x(), std::fmax(v1_.x(), v2_.x())) + 0.0001,
        std::fmax(v0_.y(), std::fmax(v1_.y(), v2_.y())) + 0.0001,
        std::fmax(v0_.z(), std::fmax(v1_.z(), v2_.z())) + 0.0001
    );
    return AABB(lo, hi);
}

// ============================================================
// HittableList
// ============================================================

void HittableList::add(std::shared_ptr<Hittable> object) {
    bbox_ = surrounding_box(bbox_, object->bounding_box());
    objects_.push_back(std::move(object));
}

void HittableList::clear() {
    objects_.clear();
}
```

```

bool HittableList::hit(const Ray& r, Interval ray_t, HitRecord& rec) const {
    HitRecord temp_rec;
    bool hit_anything = false;
    double closest_so_far = ray_t.max;

    for (const auto& object : objects_) {
        if (object->hit(r, Interval(ray_t.min, closest_so_far), temp_rec)) {
            hit_anything = true;
            closest_so_far = temp_rec.t;
            rec = temp_rec;
        }
    }

    return hit_anything;
}

AABB HittableList::bounding_box() const {
    return bbox_;
}
```

7.47 src/scene/primitives.cuh File Reference

```
#include "math/vec3.cuh"
#include "math/ray.cuh"
#include "math/interval.cuh"
#include "math/aabb.cuh"
#include <memory>
#include <vector>
```

Classes

- struct HitRecord
- class Hittable
- class Sphere
- class Plane
- class Cylinder
- class Triangle
- class HittableList

Macros

- #define RT_SCENE_PRIMITIVES_CUH

7.47.1 Macro Definition Documentation

7.47.1.1 RT_SCENE_PRIMITIVES_CUH

```
#define RT_SCENE_PRIMITIVES_CUH
```

Definition at line 2 of file primitives.cuh.

7.48 primitives.cuh

Go to the documentation of this file.

```
#ifndef RT_SCENE_PRIMITIVES_CUH
#define RT_SCENE_PRIMITIVES_CUH

#include "math/vec3.cuh"
#include "math/ray.cuh"
#include "math/interval.cuh"
#include "math/aabb.cuh"
#include <memory>
#include <vector>

class Material;

// --- HitRecord ---

struct HitRecord {
    Point3 p;
    Vec3 normal;
    const Material* material = nullptr;
    double t = 0.0;
    bool front_face = true;

    // Sets normal to always point against the incident ray
    void set_face_normal(const Ray& r, const Vec3& outward_normal) {
        front_face = dot(r.direction(), outward_normal) < 0.0;
        normal = front_face ? outward_normal : -outward_normal;
    }
};

// --- Hittable (abstract base) ---

class Hittable {
public:
    virtual ~Hittable() = default;

    virtual bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const = 0;
    virtual AABB bounding_box() const = 0;
};

// --- Sphere ---

class Sphere : public Hittable {
public:
    Sphere(const Point3& center, double radius, const Material* material);

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

    const Point3& center() const { return center_; }
    double radius() const { return radius_; }

private:
    Point3 center_;
    double radius_;
    const Material* material_;
};

// --- Plane ---

class Plane : public Hittable {
public:
    Plane(const Point3& point, const Vec3& normal, const Material* material);

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

private:
    Point3 point_;
    Vec3 normal_;   // stored as unit vector
    const Material* material_;
};

// --- Cylinder (surface quadrique, axe Y, hauteur finie) ---

class Cylinder : public Hittable {
public:
    Cylinder(const Point3& base, double radius, double height, const Material* material);

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

private:
    Point3 base_;
```

```
    double radius_;
    double height_;
    const Material* material_;
};

// --- Triangle (Moller-Trumbore intersection) ---

class Triangle : public Hittable {
public:
    Triangle(const Point3& v0, const Point3& v1, const Point3& v2, const Material* material);

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

private:
    Point3 v0_, v1_, v2_;
    Vec3 normal_;
    const Material* material_;
};

// --- HittableList ---

class HittableList : public Hittable {
public:
    HittableList() = default;

    void add(std::shared_ptr<Hittable> object);
    void clear();

    bool hit(const Ray& r, Interval ray_t, HitRecord& rec) const override;
    AABB bounding_box() const override;

    const std::vector<std::shared_ptr<Hittable»& objects() const { return objects_; }

private:
    std::vector<std::shared_ptr<Hittable» objects_;
    AABB bbox_;
};

#endif // RT_SCENE_PRIMITIVES_CUH
```

7.49 src/scene/scene.cpp File Reference

```
#include "scene/scene.h"
#include "math/random.cuh"
#include <cmath>
```

7.50 scene.cpp

Go to the documentation of this file.

```
#include "scene/scene.h"
#include "math/random.cuh"
#include <cmath>

// ============================================================
// Scene "three_spheres" : 3 materiaux sur sol gris
// Plan + 3 spheres + bulle interne = 5 objets
// ============================================================
Scene Scene::build_three_spheres() {
    Scene scene;

    auto* mat_ground = scene.add_material<Lambertian>(Color(0.45, 0.45, 0.42));
    auto* mat_terre = scene.add_material<Lambertian>(Color(0.6, 0.25, 0.1));
    auto* mat_chrome = scene.add_material<Metal>(Color(0.9, 0.9, 0.92), 0.0);
    auto* mat_eau = scene.add_material<Dielectric>(1.33);

    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), mat_ground));

    // Disposition en triangle : avant-gauche, arriere-centre, avant-droite
    scene.world.add(std::make_shared<Sphere>(Point3(-1.5, 0.6, 1.0), 0.6, mat_terre));
    scene.world.add(std::make_shared<Sphere>(Point3(0, 1.1, -1.2), 1.1, mat_eau));
    scene.world.add(std::make_shared<Sphere>(Point3(1.8, 0.5, 0.5), 0.5, mat_chrome));
```

```
    // Bulle dans la sphere d'eau
    scene.world.add(std::make_shared<Sphere>(Point3(0, 1.1, -1.2), -1.0, mat_eau));

    scene.camera.aspect_ratio = 16.0 / 9.0;
    scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 100;
    scene.camera.max_depth = 50;
    scene.camera.vfov = 40.0;
    scene.camera.lookfrom = Point3(4.0, 2.5, 4.5);
    scene.camera.lookat = Point3(0, 0.4, -0.2);
    scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.3;
    scene.camera.focus_dist = 6.0;
    scene.camera.initialize();

    return scene;
}

// ============================================================
// Scene "final" : grille concentrique de spheres (stress test)
// Anneaux concentriques autour d'une sphere centrale, hauteurs variees
// ============================================================
Scene Scene::build_final_scene() {
    Scene scene;

    auto* ground = scene.add_material<Lambertian>(Color(0.5, 0.5, 0.48));
    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), ground));

    // Sphere centrale : verre
    auto* m_center = scene.add_material<Dielectric>(1.5);
    scene.world.add(std::make_shared<Sphere>(Point3(0, 1.5, 0), 1.5, m_center));

    // Anneaux concentriques
    int n_rings = 6;
    for (int ring = 1; ring <= n_rings; ++ring) {
        double radius = ring * 2.0;
        int count = ring * 8;
        for (int i = 0; i < count; ++i) {
            double angle = 2.0 * 3.14159265 * i / count + ring * 0.3;
            double x = radius * std::cos(angle);
            double z = radius * std::sin(angle);
            double r = 0.15 + random_double() * 0.15;
            double y = r + random_double() * 0.1 * ring;

            double choose = random_double();
            if (choose < 0.5) {
                Color c(random_double(0.15, 0.85), random_double(0.15, 0.85), random_double(0.15, 0.85));
                auto* mat = scene.add_material<Lambertian>(c);
                scene.world.add(std::make_shared<Sphere>(Point3(x, y, z), r, mat));
            } else if (choose < 0.82) {
                Color c(random_double(0.5, 1.0), random_double(0.5, 1.0), random_double(0.5, 1.0));
                auto* mat = scene.add_material<Metal>(c, random_double(0.0, 0.35));
                scene.world.add(std::make_shared<Sphere>(Point3(x, y, z), r, mat));
            } else {
                auto* mat = scene.add_material<Dielectric>(1.4 + random_double() * 0.3);
                scene.world.add(std::make_shared<Sphere>(Point3(x, y, z), r, mat));
            }
        }
    }

    // 2 grosses spheres decoratives
    auto* m_gold = scene.add_material<Metal>(Color(0.85, 0.65, 0.2), 0.1);
    scene.world.add(std::make_shared<Sphere>(Point3(-4.0, 1.0, 3.0), 1.0, m_gold));

    auto* m_rouge = scene.add_material<Lambertian>(Color(0.7, 0.1, 0.1));
    scene.world.add(std::make_shared<Sphere>(Point3(3.5, 0.8, -4.0), 0.8, m_rouge));

    scene.camera.aspect_ratio = 16.0 / 9.0;
    scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 50;
    scene.camera.max_depth = 50;
    scene.camera.vfov = 30.0;
    scene.camera.lookfrom = Point3(12, 5, 10);
    scene.camera.lookat = Point3(0, 0.8, 0);
    scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.3;
    scene.camera.focus_dist = 15.0;
    scene.camera.initialize();

    return scene;
}

// ============================================================
// Scene "triangle" : pyramide + formes geometriques
// ============================================================
Scene Scene::build_triangle_scene() {
```

```
    Scene scene;

    auto* mat_ground = scene.add_material<Checker>(Color(0.8, 0.75, 0.7), Color(0.35, 0.3, 0.3), 1.0);
    auto* mat_red    = scene.add_material<Lambertian>(Color(0.75, 0.12, 0.12));
    auto* mat_teal   = scene.add_material<Lambertian>(Color(0.1, 0.6, 0.55));
    auto* mat_amber  = scene.add_material<Metal>(Color(0.85, 0.7, 0.25), 0.1);
    auto* mat_silver = scene.add_material<Metal>(Color(0.8, 0.82, 0.85), 0.02);
    auto* mat_glass  = scene.add_material<Dielectric>(1.5);

    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), mat_ground));

    // Pyramide
    Point3 a(-0.8, 0, -0.8), b(0.8, 0, -0.8), c(0.8, 0, 0.8), d(-0.8, 0, 0.8);
    Point3 apex(0, 1.6, 0);
    scene.world.add(std::make_shared<Triangle>(a, b, apex, mat_red));
    scene.world.add(std::make_shared<Triangle>(b, c, apex, mat_teal));
    scene.world.add(std::make_shared<Triangle>(c, d, apex, mat_amber));
    scene.world.add(std::make_shared<Triangle>(d, a, apex, mat_silver));
    scene.world.add(std::make_shared<Triangle>(a, c, b, mat_ground));
    scene.world.add(std::make_shared<Triangle>(a, d, c, mat_ground));

    // Sphere verre a cote
    scene.world.add(std::make_shared<Sphere>(Point3(2.2, 0.5, 0.5), 0.5, mat_glass));
    // Cylindre
    scene.world.add(std::make_shared<Cylinder>(Point3(-2.0, 0, 0.5), 0.3, 1.5, mat_silver));

    scene.camera.aspect_ratio = 16.0 / 9.0;
    scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 100;
    scene.camera.max_depth = 50;
    scene.camera.vfov = 40.0;
    scene.camera.lookfrom = Point3(3.5, 2.5, 4.5);
    scene.camera.lookat = Point3(0, 0.6, 0);
    scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.0;
    scene.camera.focus_dist = 5.0;
    scene.camera.initialize();

    return scene;
}

// ============================================================
// Scene "cornell" : boite avec 2 spheres
// ============================================================
Scene Scene::build_cornell_box() {
    Scene scene;

    auto* mat_white = scene.add_material<Lambertian>(Color(0.73, 0.73, 0.73));
    auto* mat_red   = scene.add_material<Lambertian>(Color(0.65, 0.05, 0.05));
    auto* mat_green = scene.add_material<Lambertian>(Color(0.12, 0.45, 0.15));
    auto* mat_metal = scene.add_material<Metal>(Color(0.8, 0.85, 0.88), 0.0);
    auto* mat_glass = scene.add_material<Dielectric>(1.5);
    auto* mat_light = scene.add_material<Emissive>(Color(1, 1, 1), 15.0);

    double S = 5.55; // taille de la boite
    // Sol
    scene.world.add(std::make_shared<Triangle>(Point3(0,0,0), Point3(S,0,0), Point3(S,0,S), mat_white));
    scene.world.add(std::make_shared<Triangle>(Point3(0,0,0), Point3(S,0,S), Point3(0,0,S), mat_white));
    // Plafond
    scene.world.add(std::make_shared<Triangle>(Point3(0,S,0), Point3(S,S,S), Point3(S,S,0), mat_white));
    scene.world.add(std::make_shared<Triangle>(Point3(0,S,0), Point3(0,S,S), Point3(S,S,S), mat_white));
    // Fond
    scene.world.add(std::make_shared<Triangle>(Point3(0,0,S), Point3(S,0,S), Point3(S,S,S), mat_white));
    scene.world.add(std::make_shared<Triangle>(Point3(0,0,S), Point3(S,S,S), Point3(0,S,S), mat_white));
    // Gauche rouge
    scene.world.add(std::make_shared<Triangle>(Point3(0,0,0), Point3(0,0,S), Point3(0,S,S), mat_red));
    scene.world.add(std::make_shared<Triangle>(Point3(0,0,0), Point3(0,S,S), Point3(0,S,0), mat_red));
    // Droit vert
    scene.world.add(std::make_shared<Triangle>(Point3(S,0,0), Point3(S,S,0), Point3(S,S,S), mat_green));
    scene.world.add(std::make_shared<Triangle>(Point3(S,0,0), Point3(S,S,S), Point3(S,0,S), mat_green));
    // Lumiere au plafond
    double L = 1.5;
    double cx = S/2, cz = S/2;
    scene.world.add(std::make_shared<Triangle>(
        Point3(cx-L, S-0.01, cz-L), Point3(cx+L, S-0.01, cz-L), Point3(cx+L, S-0.01, cz+L), mat_light));
    scene.world.add(std::make_shared<Triangle>(
        Point3(cx-L, S-0.01, cz-L), Point3(cx+L, S-0.01, cz+L), Point3(cx-L, S-0.01, cz+L), mat_light));
```

```

    scene.world.add(std::make_shared<Sphere>(Point3(1.8, 0.8, 1.9), 0.8, mat_metal));
    scene.world.add(std::make_shared<Sphere>(Point3(3.7, 0.6, 3.5), 0.6, mat_glass));

    scene.camera.aspect_ratio = 1.0;
    scene.camera.image_width = 600;
    scene.camera.samples_per_pixel = 300;
    scene.camera.max_depth = 50;
    scene.camera.vfov = 40.0;
    scene.camera.lookfrom = Point3(S/2, S/2, -8);
    scene.camera.lookat = Point3(S/2, S/2, 0);
    scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.0;
    scene.camera.focus_dist = 10.0;
    scene.camera.initialize();

    return scene;
}

// ============================================================
// Scene "showcase" : vitrine de toutes les features
// ============================================================
Scene Scene::build_showcase() {
    Scene scene;

    auto* mat_ground = scene.add_material<Checker>(Color(0.9, 0.9, 0.9), Color(0.3, 0.3, 0.35), 1.0);
    auto* mat_mirror   = scene.add_material<Metal>(Color(0.95, 0.95, 0.95), 0.0);
    auto* mat_gold     = scene.add_material<Metal>(Color(0.85, 0.65, 0.13), 0.05);
    auto* mat_copper   = scene.add_material<Metal>(Color(0.72, 0.45, 0.20), 0.15);
    auto* mat_glass    = scene.add_material<Dielectric>(1.5);
    auto* mat_diamond  = scene.add_material<Dielectric>(2.42);
    auto* mat_red      = scene.add_material<Lambertian>(Color(0.9, 0.15, 0.15));
    auto* mat_blue     = scene.add_material<Lambertian>(Color(0.15, 0.35, 0.9));
    auto* mat_green    = scene.add_material<Lambertian>(Color(0.2, 0.85, 0.3));
    auto* mat_orange   = scene.add_material<Lambertian>(Color(0.95, 0.55, 0.1));
    auto* mat_purple   = scene.add_material<Lambertian>(Color(0.6, 0.2, 0.8));
    auto* mat_light    = scene.add_material<Emissive>(Color(1.0, 0.95, 0.9), 4.0);

    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), mat_ground));

    scene.world.add(std::make_shared<Sphere>(Point3(0, 1.2, 0), 1.2, mat_mirror));
    scene.world.add(std::make_shared<Sphere>(Point3(-2.5, 0.8, 0.5), 0.8, mat_glass));
    scene.world.add(std::make_shared<Sphere>(Point3(-2.5, 0.8, 0.5), -0.75, mat_glass));
    scene.world.add(std::make_shared<Sphere>(Point3(2.5, 0.9, -0.3), 0.9, mat_gold));
    scene.world.add(std::make_shared<Sphere>(Point3(-1.0, 0.35, 2.0), 0.35, mat_red));
    scene.world.add(std::make_shared<Sphere>(Point3(-0.2, 0.25, 2.5), 0.25, mat_orange));
    scene.world.add(std::make_shared<Sphere>(Point3(0.7, 0.30, 2.2), 0.30, mat_blue));
    scene.world.add(std::make_shared<Sphere>(Point3(1.5, 0.28, 2.3), 0.28, mat_green));
    scene.world.add(std::make_shared<Sphere>(Point3(1.0, 0.5, -2.0), 0.5, mat_diamond));
    scene.world.add(std::make_shared<Sphere>(Point3(-1.5, 0.4, -1.8), 0.4, mat_copper));
    scene.world.add(std::make_shared<Sphere>(Point3(3.5, 0.35, 1.5), 0.35, mat_purple));
    scene.world.add(std::make_shared<Cylinder>(Point3(-3.5, 0, -1.0), 0.3, 2.0, mat_copper));

    scene.world.add(std::make_shared<Triangle>(Point3(-2,4,-2), Point3(2,4,-2), Point3(2,4,2),
mat_light));
    scene.world.add(std::make_shared<Triangle>(Point3(-2,4,-2), Point3(2,4,2), Point3(-2,4,2),
mat_light));

    scene.camera.aspect_ratio = 16.0 / 9.0;
    scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 200;
    scene.camera.max_depth = 50;
    scene.camera.vfov = 35.0;
    scene.camera.lookfrom = Point3(4, 2.5, 6);
    scene.camera.lookat = Point3(0, 0.5, 0);
    scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.4;
    scene.camera.focus_dist = 7.0;
    scene.camera.initialize();
    return scene;
}

// ============================================================
// Scene "hall" : salle avec colonnes
// ============================================================
Scene Scene::build_hall() {
    Scene scene;
    auto* mat_floor = scene.add_material<Checker>(Color(0.95, 0.95, 0.95), Color(0.15, 0.15, 0.2),
2.0);
    auto* mat_wall  = scene.add_material<Lambertian>(Color(0.85, 0.82, 0.78));
    auto* mat_chrome = scene.add_material<Metal>(Color(0.9, 0.9, 0.9), 0.02);
    auto* mat_gold  = scene.add_material<Metal>(Color(0.85, 0.65, 0.13), 0.0);
    auto* mat_glass = scene.add_material<Dielectric>(1.5);
    auto* mat_ruby  = scene.add_material<Lambertian>(Color(0.7, 0.05, 0.1));
    auto* mat_emerald = scene.add_material<Lambertian>(Color(0.05, 0.6, 0.15));
    auto* mat_light = scene.add_material<Emissive>(Color(1.0, 0.98, 0.95), 5.0);

```

```
    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), mat_floor));
    scene.world.add(std::make_shared<Triangle>(Point3(-20,0,-8), Point3(20,0,-8), Point3(20,10,-8), mat_wall));
    scene.world.add(std::make_shared<Triangle>(Point3(-20,0,-8), Point3(20,10,-8), Point3(-20,10,-8), mat_wall));

    for (int i = -3; i <= 3; i++) {
        scene.world.add(std::make_shared<Cylinder>(Point3(i*3.0, 0, -5), 0.25, 4.0, mat_chrome));
        scene.world.add(std::make_shared<Cylinder>(Point3(i*3.0, 0, 3), 0.25, 4.0, mat_chrome));
    }
    scene.world.add(std::make_shared<Sphere>(Point3(0, 1.5, -1), 1.5, mat_gold));
    for (int i = -2; i <= 2; i++)
        scene.world.add(std::make_shared<Sphere>(Point3(i*2.5, 0.4, 1.0), 0.4, mat_glass));
    scene.world.add(std::make_shared<Sphere>(Point3(-4, 0.3, 0), 0.3, mat_ruby));
    scene.world.add(std::make_shared<Sphere>(Point3(4, 0.3, 0), 0.3, mat_emerald));
    scene.world.add(std::make_shared<Triangle>(Point3(-3,4,-3), Point3(3,4,-3), Point3(3,4,1), mat_light));
    scene.world.add(std::make_shared<Triangle>(Point3(-3,4,-3), Point3(3,4,1), Point3(-3,4,1), mat_light));

    scene.camera.aspect_ratio = 16.0 / 9.0; scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 200; scene.camera.max_depth = 50;
    scene.camera.vfov = 50.0; scene.camera.lookfrom = Point3(0, 2.0, 8);
    scene.camera.lookat = Point3(0, 1.0, -2); scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.0; scene.camera.focus_dist = 10.0;
    scene.camera.initialize();
    return scene;
}

// ============================================================
// Scene "galaxy" : ~1000 spheres en spirale (stress test)
// ============================================================
Scene Scene::build_galaxy() {
    Scene scene;
    auto* mat_ground = scene.add_material<Checker>(Color(0.1, 0.1, 0.15), Color(0.05, 0.05, 0.08), 3.0);
    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), mat_ground));

    for (int arm = 0; arm < 4; arm++) {
        double arm_off = arm * 3.14159265 / 2.0;
        for (int i = 0; i < 250; i++) {
            double t = i * 0.04, r = 1.0 + t * 2.5, angle = t * 2.5 + arm_off;
            double x = r * std::cos(angle) + random_double(-0.3, 0.3);
            double z = r * std::sin(angle) + random_double(-0.3, 0.3);
            double y = 0.1 + random_double(0.0, 0.3), radius = 0.08 + random_double(0.0, 0.12);
            double mc = random_double();
            const Material* mat;
            if (mc < 0.5) mat = scene.add_material<Lambertian>(random_vec3() * random_vec3());
            else if (mc < 0.8) mat = scene.add_material<Metal>(random_vec3(0.5, 1.0), random_double(0, 0.3));
            else mat = scene.add_material<Dielectric>(1.5);
            scene.world.add(std::make_shared<Sphere>(Point3(x, y, z), radius, mat));
        }
    }
    auto* mat_c = scene.add_material<Metal>(Color(0.95, 0.95, 0.98), 0.0);
    scene.world.add(std::make_shared<Sphere>(Point3(0, 1.5, 0), 1.5, mat_c));
    scene.world.add(std::make_shared<Sphere>(Point3(-3, 0.8, 2), 0.8, scene.add_material<Dielectric>(1.5)));
    scene.world.add(std::make_shared<Sphere>(Point3(3, 0.7, -2), 0.7, scene.add_material<Metal>(Color(0.85, 0.65, 0.13), 0.0)));

    scene.camera.aspect_ratio = 16.0 / 9.0; scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 100; scene.camera.max_depth = 30;
    scene.camera.vfov = 30.0; scene.camera.lookfrom = Point3(15, 8, 15);
    scene.camera.lookat = Point3(0, 0.5, 0); scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.3; scene.camera.focus_dist = 20.0;
    scene.camera.initialize();
    return scene;
}

// ============================================================
// Scene "minimal" : noir blanc miroir verre, DOF
// ============================================================
Scene Scene::build_minimal() {
    Scene scene;
    auto* mat_white = scene.add_material<Lambertian>(Color(0.95, 0.95, 0.95));
    auto* mat_black = scene.add_material<Lambertian>(Color(0.02, 0.02, 0.02));
    auto* mat_mirror = scene.add_material<Metal>(Color(0.98, 0.98, 0.98), 0.0);
    auto* mat_glass = scene.add_material<Dielectric>(1.5);

    scene.world.add(std::make_shared<Plane>(Point3(0, 0, 0), Vec3(0, 1, 0), mat_white));
    scene.world.add(std::make_shared<Triangle>(Point3(-10,0,-5), Point3(10,0,-5), Point3(10,10,-5), mat_white));
    scene.world.add(std::make_shared<Triangle>(Point3(-10,0,-5), Point3(10,10,-5), Point3(-10,10,-5), mat_white));

    scene.world.add(std::make_shared<Sphere>(Point3(-2, 1, 0), 1.0, mat_black));
```

```
    scene.world.add(std::make_shared<Sphere>(Point3(0, 1, 0), 1.0, mat_mirror));
    scene.world.add(std::make_shared<Sphere>(Point3(2, 1, 0), 1.0, mat_glass));
    scene.world.add(std::make_shared<Sphere>(Point3(2, 1, 0), -0.95, mat_glass));

    scene.camera.aspect_ratio = 16.0 / 9.0; scene.camera.image_width = 800;
    scene.camera.samples_per_pixel = 300; scene.camera.max_depth = 50;
    scene.camera.vfov = 30.0; scene.camera.lookfrom = Point3(0, 2.5, 8);
    scene.camera.lookat = Point3(0, 0.8, 0); scene.camera.vup = Vec3(0, 1, 0);
    scene.camera.defocus_angle = 0.8; scene.camera.focus_dist = 8.0;
    scene.camera.initialize();
    return scene;
}
```

7.51 src/scene/scene.h File Reference

```
#include "scene/primitives.cuh"
#include "scene/camera.cuh"
#include "scene/material.cuh"
#include <memory>
#include <vector>
```

Classes

- class Scene

7.52 scene.h

Go to the documentation of this file.

```
#ifndef RT_SCENE_SCENE_H
#define RT_SCENE_SCENE_H

#include "scene/primitives.cuh"
#include "scene/camera.cuh"
#include "scene/material.cuh"
#include <memory>
#include <vector>

class Scene {
public:
    Camera camera;
    HittableList world;

    // Scene owns all materials
    template<typename T, typename... Args>
    T* add_material(Args&&... args) {
        auto mat = std::make_unique<T>(std::forward<Args>(args)...);
        T* ptr = mat.get();
        materials_.push_back(std::move(mat));
        return ptr;
    }

    // Factory: three spheres + ground plane
    static Scene build_three_spheres();

    // Factory: final scene with many random spheres
    static Scene build_final_scene();

    // Factory: scene with triangles (pyramide + spheres)
    static Scene build_triangle_scene();

    // Factory: Cornell Box (classique)
    static Scene build_cornell_box();

    // Factory: scene showcase (lumineuse, variee)
    static Scene build_showcase();

    // Factory: colonnes de verre et metal dans une salle
```

```
    static Scene build_hall();

    // Factory: galaxie de spheres (stress test ~1000 objets)
    static Scene build_galaxy();

    // Factory: scene minimaliste noir/blanc
    static Scene build_minimal();

private:
    std::vector<std::unique_ptr<Material» materials_;
};

#endif // RT_SCENE_SCENE_H
```

7.53 src/utils/cuda_utils.h File Reference

7.54 cuda_utils.h

Go to the documentation of this file.

```
#ifndef RT_UTILS_CUDA_UTILS_H
#define RT_UTILS_CUDA_UTILS_H

#ifdef __CUDACC__
#include <cstdio>

#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        return; \
    } \
} while(0)

#endif // __CUDACC__

#endif // RT_UTILS_CUDA_UTILS_H
```

7.55 src/utils/denoise.cpp File Reference

```
#include "utils/denoise.h"
#include <cmath>
#include <vector>
```

Functions

- void denoise_bilateral (ImageBuffer &image, int radius, double sigma_color)

7.55.1 Function Documentation

7.55.1.1 denoise_bilateral()

```
void denoise_bilateral (
            ImageBuffer & image,
            int radius,
            double sigma_color)
```

Definition at line 5 of file denoise.cpp.

7.56 denoise.cpp

Go to the documentation of this file.

```
#include "utils/denoise.h"
#include <cmath>
#include <vector>

void denoise_bilateral(ImageBuffer& image, int radius, double sigma_color) {
    int w = image.width(), h = image.height();
    std::vector<Color> output(w * h);
    double sigma_spatial = (double)radius;

    for (int y = 0; y < h; y++) {
        for (int x = 0; x < w; x++) {
            Color center = image.get_pixel(x, y);
            Color sum(0, 0, 0);
            double weight_sum = 0;

            for (int dy = -radius; dy <= radius; dy++) {
                for (int dx = -radius; dx <= radius; dx++) {
                    int nx = x + dx, ny = y + dy;
                    if (nx < 0 || nx >= w || ny < 0 || ny >= h) continue;

                    Color neighbor = image.get_pixel(nx, ny);

                    // Distance spatiale
                    double spatial_dist = std::sqrt((double)(dx*dx + dy*dy));
                    double ws = std::exp(-spatial_dist / (2.0 * sigma_spatial * sigma_spatial));

                    // Difference de couleur
                    double dr = center.x() - neighbor.x();
                    double dg = center.y() - neighbor.y();
                    double db = center.z() - neighbor.z();
                    double color_dist = dr*dr + dg*dg + db*db;
                    double wc = std::exp(-color_dist / (2.0 * sigma_color * sigma_color));

                    double w_total = ws * wc;
                    sum += neighbor * w_total;
                    weight_sum += w_total;
                }
            }

            if (weight_sum > 0)
                output[y * w + x] = sum / weight_sum;
            else
                output[y * w + x] = center;
        }
    }

    // Ecrire les resultats
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
            image.set_pixel(x, y, output[y * w + x]);
}
```

7.57 src/utils/denoise.h File Reference

```
#include "utils/image.h"
```

Functions

- void denoise_bilateral (ImageBuffer &image, int radius=3, double sigma_color=0.1)

7.57.1 Function Documentation

7.57.1.1 denoise_bilateral()

```
void denoise_bilateral (
            ImageBuffer & image,
            int radius = 3,
            double sigma_color = 0.1)
```

Definition at line 5 of file denoise.cpp.

7.58 denoise.h

Go to the documentation of this file.

```
00001 #ifndef RT_UTILS_DENOISE_H
00002 #define RT_UTILS_DENOISE_H
00003
00004 #include "utils/image.h"
00005
00006 // Simple bilateral denoise : lisse le bruit tout en preservant les contours
00007 // sigma_spatial : rayon du filtre (en pixels)
00008 // sigma_color : sensibilite aux differences de couleur
00009 void denoise_bilateral(ImageBuffer& image, int radius = 3, double sigma_color = 0.1);
00010
00011 #endif
```

7.59 src/utils/image.cpp File Reference

```
#include "utils/image.h"
#include <fstream>
#include <algorithm>
#include <cmath>
#include <stdexcept>
```

Functions

- static double aces_tonemap (double x)

7.59.1 Function Documentation

7.59.1.1 aces_tonemap()

```
double aces_tonemap (
            double x)  [static]
```

Definition at line 19 of file image.cpp.

7.60 image.cpp

Go to the documentation of this file.

```
#include "utils/image.h"
#include <fstream>
#include <algorithm>
#include <cmath>
#include <stdexcept>

ImageBuffer::ImageBuffer(int width, int height)
    : width_(width), height_(height), pixels_(width * height, Color(0, 0, 0)) {}

void ImageBuffer::set_pixel(int x, int y, const Color& color) {
    pixels_[index(x, y)] = color;
}

Color ImageBuffer::get_pixel(int x, int y) const {
    return pixels_[index(x, y)];
}

// ACES filmic tone mapping + sRGB gamma
static double aces_tonemap(double x) {
    // ACES filmic curve: meilleur rendu que simple sqrt gamma
    double a = x * (2.51 * x + 0.03);
    double b = x * (2.43 * x + 0.59) + 0.14;
    return std::max(0.0, std::min(1.0, a / b));
}

int ImageBuffer::to_byte(double linear) {
    double tonemapped = aces_tonemap(std::max(0.0, linear));
    // sRGB gamma approximation
    double corrected = std::sqrt(tonemapped);
    return static_cast<int>(255.999 * corrected);
}

void ImageBuffer::write_ppm(std::ostream& out) const {
    out « "P3\n" « width_ « ' ' « height_ « "\n255\n";
    for (int y = 0; y < height_; ++y) {
        for (int x = 0; x < width_; ++x) {
            const Color& c = pixels_[index(x, y)];
            out « to_byte(c.x()) « ' '
                « to_byte(c.y()) « ' '
                « to_byte(c.z()) « '\n';
        }
    }
}

void ImageBuffer::write_ppm(const std::string& filename) const {
    std::ofstream file(filename);
    if (!file)
        throw std::runtime_error("Cannot open file: " + filename);
    write_ppm(file);
}

void ImageBuffer::write_ppm_binary(const std::string& filename) const {
    std::ofstream file(filename, std::ios::binary);
    if (!file)
        throw std::runtime_error("Cannot open file: " + filename);

    // Header P6 (toujours en texte)
    file « "P6\n" « width_ « ' ' « height_ « "\n255\n";

    // Pixel data en binaire (3 octets par pixel)
    for (int y = 0; y < height_; ++y) {
        for (int x = 0; x < width_; ++x) {
            const Color& c = pixels_[index(x, y)];
            unsigned char rgb[3] = {
                (unsigned char)to_byte(c.x()),
                (unsigned char)to_byte(c.y()),
                (unsigned char)to_byte(c.z())
            };
            file.write(reinterpret_cast<char*>(rgb), 3);
        }
    }
}
```

7.61 src/utils/image.h File Reference

```
#include "math/vec3.cuh"
#include <vector>
```

```
#include <string>
#include <ostream>
```

Classes

- class ImageBuffer

7.62 image.h

Go to the documentation of this file.

```
#ifndef RT_UTILS_IMAGE_H
#define RT_UTILS_IMAGE_H

#include "math/vec3.cuh"
#include <vector>
#include <string>
#include <ostream>

class ImageBuffer {
public:
    ImageBuffer(int width, int height);

    int width() const { return width_; }
    int height() const { return height_; }

    void set_pixel(int x, int y, const Color& color);
    Color get_pixel(int x, int y) const;

    // Write PPM P3 (text) format
    void write_ppm(std::ostream& out) const;
    void write_ppm(const std::string& filename) const;

    // Write PPM P6 (binary) format - plus compact, plus rapide
    void write_ppm_binary(const std::string& filename) const;

private:
    int width_;
    int height_;
    std::vector<Color> pixels_;

    int index(int x, int y) const { return y * width_ + x; }

    // Apply gamma correction (gamma 2.0 = sqrt)
    static int to_byte(double linear);
};

#endif // RT_UTILS_IMAGE_H
```

7.63 src/utils/timer.h File Reference

```
#include <chrono>
```

Classes

- class Timer

7.64 timer.h

Go to the documentation of this file.

```
#ifndef RT_UTILS_TIMER_H
#define RT_UTILS_TIMER_H

#include <chrono>

class Timer {
public:
    Timer() : start_(std::chrono::high_resolution_clock::now()) {}

    void reset() {
        start_ = std::chrono::high_resolution_clock::now();
    }

    // Returns elapsed time in seconds
    double elapsed() const {
        auto now = std::chrono::high_resolution_clock::now();
        return std::chrono::duration<double>(now - start_).count();
    }

    // Returns elapsed time in milliseconds
    double elapsed_ms() const {
        return elapsed() * 1000.0;
    }

private:
    std::chrono::high_resolution_clock::time_point start_;
};

#endif // RT_UTILS_TIMER_H
```